



Horizon Rendering Library

C & C++ API - Detailed Reference Manual

API version: 26.6

Language: C / C ABI compatible

Source basis: Horizon Rendering Library public header supplied for this documentation

Documentation approach. Every public C symbol is described in English. Where the header already contains a comment, that material is preserved and expanded. Where it only contains a signature, the behavior is described by inference from the function name, parameter types, enums, resource relationships, and neighboring APIs. Those inferred descriptions are intended to explain the design and expected usage; they are not a substitute for implementation-specific guarantees.

Copyright (c) 2025-2026 Oscar Soirey
<https://github.com/oscar-soirey/Horizon-Rendering-Library>

Generated by AI, based on the project source code.

1. Library Overview

Horizon Rendering Library (HRL) exposes a handle-based C API for real-time rendering. The public surface is organized around scenes, resources, rendering passes, effects, debugging helpers, and UI. Objects are generally referenced by HRL_id values, while imported CPU-side data uses ordinary C structs.

Subsystem	Scope
Initialization & lifecycle	Backend selection, context creation, frame boundaries and resize handling.
Errors	Error codes, severities, diagnostics and callbacks.
Meshes, sprites & FBX	Static meshes, skeletal meshes, FBX extraction, resources and materials.
Lights	Point, directional, spot and ambient/sky-style illumination controls.
Textures & fonts	Synchronous/asynchronous image resources, text rasterization and filtering.
Scenes & rendering	Scenes, GI, sky/environment, picking and render targets.
Shaders & materials	Shader creation, async compilation and named material parameters.
Viewports & cameras	Viewport rectangles and camera projection/transform controls.
Gizmos	Interactive transform manipulators and callbacks.
VFX / particles	Emitters, curves, simulation, rendering and collisions.
Fog, god rays & AO	Global/local volumetrics and screen-space lighting effects.
Matrices & debug	Matrix extraction, debug primitives, captures and antialiasing.
Input & widgets	Mouse input and button/label/image/slider/checkbox/progress UI.

2. Core Concepts and Conventions

2.1 Opaque handles

HRL_id is the common object identifier used throughout the API. Creation functions generally return an HRL_id; Delete functions invalidate it; IsValid functions test whether a handle still refers to a live object. HRL_INVALID_ID is the reserved invalid value.

```
typedef uint32_t HRL_id;
#define HRL_INVALID_ID ((HRL_id)-1)
```

2.2 Boolean values

The C interface uses integer booleans rather than the C99 _Bool type. HRL_TRUE is 1 and HRL_FALSE is 0. Functions with parameters such as enable, visible, enabled, clickable, loop, and checked follow this convention.

```
#define HRL_FALSE ((int)0)
#define HRL_TRUE ((int)1)
```

2.3 Ownership and lifetime

A useful mental model is that HRL owns GPU/runtime objects represented by integer handles. Creation transfers no ownership of caller-provided CPU buffers unless a function explicitly returns allocated data. Conversely, importer functions returning pointers allocate HRL-managed CPU data and expose a matching Free function.

2.4 Coordinates and units

The API uses world-space values for scene objects, normalized 0..1 values for many colors, pixels for framebuffer/UI dimensions, degrees for Euler angles where the function names explicitly refer to rotation/cutoffs, and seconds for animation/simulation durations.

2.5 Error handling

Creation and lookup functions that return HRL_id use HRL_INVALID_ID as the natural sentinel for failure or "not found" cases. Void operations instead communicate failures through the library error state and optional error callback.

2.6 Asynchronous resources

Asynchronous texture and shader creation is explicitly modeled as a two-stage lifecycle: create the resource, poll with Is*Ready or block with WaitFor*, then use the resource once ready. HRL_WaitForAllAsyncResources provides a global synchronization point.

3. Data Structures and Enumerations

HRL_Vertex3D

@brief Vertex format used by HRL_CreateMesh3D.

All vectors are expressed in object/local space. UV coordinates use the same convention as HRL_CreateMeshSprite and HRL_SetSpriteRegion. Tangent and bitangent are used with the normal map for the built-in OpenGL 3.3 shader.

Member	Type	Description
position	float[3]	Per-vertex geometric attribute used by the standard 3D vertex format.
normal	float[3]	Per-vertex geometric attribute used by the standard 3D vertex format.
uv	float[2]	Per-vertex geometric attribute used by the standard 3D vertex format.
tangent	float[3]	Per-vertex geometric attribute used by the standard 3D vertex format.
bitangent	float[3]	Per-vertex geometric attribute used by the standard 3D vertex format.

HRL_SkeletalVertex

Vertex format used by skeletal meshes. The first five attributes share the HRL_Vertex3D layout, followed by four bone indices and four normalized weights.

Member	Type	Description
vertex	HRL_Vertex3D	Base vertex attributes shared with HRL_Vertex3D.
boneIndices	HRL_uint[HRL_SKELETAL_MAX_INFLUENCES]	Indices of the bones contributing to this vertex. Up to HRL_SKELETAL_MAX_INFLUENCES entries are stored.
boneWeights	float[HRL_SKELETAL_MAX_INFLUENCES]	Normalized influence weights corresponding to boneIndices.

HRL_SkeletalBoneTransform

Local transform sampled for one skeletal bone at one animation frame.

Member	Type	Description
translation	float[3]	Local translation component of the transform.
rotation	float[4]	Quaternion rotation, stored as x, y, z, w.
scale	float[3]	Local scale component of the transform.

HRL_SkeletalBone

Public skeletal bone metadata.

Member	Type	Description
name	const char*	Null-terminated name identifying the imported object.
parentIndex	HRL_uint	Index of the parent bone in the bones array, defining the hierarchy.
bindTransform	HRL_SkeletalBoneTransform	Local bind-pose transform used to reconstruct the bone hierarchy.
bindWorldMatrix	float[16]	Evaluated bind-pose node-to-world matrix.
inverseBindMatrix	float[16]	Inverse bind matrix used to transform vertices from bind space into skinning space.

HRL_SkeletalAnimation

Baked animation data owned by HRL_SkeletalMeshData until freed.

Member	Type	Description
name	const char*	Null-terminated name identifying the imported object.
duration	float	Animation duration in seconds.
frameRate	float	Nominal sampling rate of the baked animation.
frameCount	size_t	Number of baked samples in the animation.

Member	Type	Description
frames	HRL_SkeletalBoneTransform*	Array of bone transforms indexed by frame and bone.
poseMatrices	float*	Optional evaluated skin matrices: frameCount * boneCount * 16 floats.

HRL_SkeletalMeshData

CPU-side skeletal mesh data produced by an importer.

Member	Type	Description
vertices	HRL_SkeletalVertex*	Pointer to the mesh vertex array.
vertexCount	size_t	Number of vertices in the array.
indices	HRL_uint*	Optional triangle index array.
indexCount	size_t	Number of indices in the index array.
bones	HRL_SkeletalBone*	Pointer to skeletal bone metadata.
boneCount	size_t	Number of skeletal bones.
animations	HRL_SkeletalAnimation*	Pointer to baked skeletal animations.
animationCount	size_t	Number of animations in the array.
geometryToWorldMatrix	float[16]	Geometry-to-world transform of the FBX mesh instance. New data should set this to identity unless imported from FBX.

HRL_FBXResources

Opaque handle owning a decoded FBX resource scene.

Member	Type	Description
--------	------	-------------

HRL_FBXTextureInfo

Description of one texture resource in an imported FBX scene.
The data pointer remains valid until HRL_FreeFBXResources().

Member	Type	Description
name	const char*	Null-terminated name identifying the imported object.
filename	const char*	Public data member used by the corresponding HRL subsystem.
data	const unsigned char*	Public data member used by the corresponding HRL subsystem.
size	size_t	Public data member used by the corresponding HRL subsystem.
type	HRL_EFBXTextureType	Public data member used by the corresponding HRL subsystem.
embedded	int	Public data member used by the corresponding HRL subsystem.

HRL_FBXMaterialInfo

Material information extracted from the FBX PBR/legacy material model.
textureIndices contains HRL_INVALID_ID where no matching texture exists.

Member	Type	Description
name	const char*	Null-terminated name identifying the imported object.
baseColor	float[4]	Public data member used by the corresponding HRL subsystem.
roughness	float	Public data member used by the corresponding HRL subsystem.
metallic	float	Public data member used by the corresponding HRL subsystem.
specular	float	Public data member used by the corresponding HRL subsystem.
opacity	float	Public data member used by the corresponding HRL subsystem.
textureIndices	HRL_uint[HRL_FBX_MATERIAL_TEXTURE_SLOT_COUNT]	Public data member used by the corresponding HRL subsystem.

HRL_E_APIs

Enumerator	Value	Meaning
HRL_OPENGL_33	0x0001	Named enumeration value for the corresponding HRL subsystem.
HRL_OPENGL_45		Named enumeration value for the corresponding HRL subsystem.
HRL_VULKAN		Named enumeration value for the corresponding HRL subsystem.
HRL_D3D11		Named enumeration value for the corresponding HRL subsystem.
HRL_D3D12		Named enumeration value for the corresponding HRL subsystem.
HRL_METAL		Named enumeration value for the corresponding HRL subsystem.
HRL_NVN		Named enumeration value for the corresponding HRL subsystem.
HRL_GNM		Named enumeration value for the corresponding HRL subsystem.

HRL_ELightType

Enumerator	Value	Meaning
HRL_POINT_LIGHT	0x0011	Point-shaped source/emitter/light or center part, depending on enum family.
HRL_DIRECTIONAL_LIGHT		Direction-based, effectively distant, source.
HRL_SPOT_LIGHT		Cone-shaped spotlight source.
HRL_SKY_LIGHT		Low-intensity ambient light coming uniformly from all directions. Does not cast shadows.

HRL_EMeshType

Enumerator	Value	Meaning
HRL_SPRITE	0x0021	Named enumeration value for the corresponding HRL subsystem.
HRL_2D_MESH		Renders particles using an existing mesh.
HRL_3D_MESH		Renders particles using an existing mesh.
HRL_3D_SKELETAL_MESH		Renders particles using an existing mesh.

HRL_EDebugRenderingType

Enumerator	Value	Meaning
HRL_DEBUG_HOLLOW	0x0031	Named enumeration value for the corresponding HRL subsystem.
HRL_DEBUG_SOLID		Named enumeration value for the corresponding HRL subsystem.

HRL_ECameraType

Enumerator	Value	Meaning
HRL_ORTHO	0x0041	Uses orthographic projection.
HRL_PERSPECTIVE		Uses perspective projection.

HRL_EFilterType

Enumerator	Value	Meaning
HRL_FILTER_NEAREST	0x0050	Nearest-neighbor filtering.
HRL_FILTER_LINEAR		Selects the linear interpretation/mode.
HRL_FILTER_BILINEAR		Selects the linear interpretation/mode.
HRL_FILTER_TRILINEAR		Selects the linear interpretation/mode.
HRL_FILTER_ANISOTROPIC		Anisotropic texture filtering.

Enumerator	Value	Meaning
HRL_FILTER_SUPERSAMPLING		Not available with OpenGL backends

HRL_EGlobalIlluminationMethod

Global illumination methods.

Backend support in this release:

- OpenGL 3.3: DDGI-style world-space irradiance probes
- OpenGL 4.5: not implemented in this repository
- Vulkan: not implemented in this repository
- D3D11: not implemented in this repository
- D3D12: not implemented in this repository
- Metal: not implemented in this repository
- NVN: not implemented in this repository
- GNM: not implemented in this repository

VCT, LPV, DDGI, path tracing and ray tracing are exposed now so the public API does not need to change when future backends implement them.

Enumerator	Value	Meaning
HRL_GI_NONE	0	Disables or represents the absence of the corresponding feature/state.
HRL_GI_SSGI		Named enumeration value for the corresponding HRL subsystem.
HRL_GI_VCT		Named enumeration value for the corresponding HRL subsystem.
HRL_GI_LPV		Named enumeration value for the corresponding HRL subsystem.
HRL_GI_DDGI		Named enumeration value for the corresponding HRL subsystem.
HRL_GI_PATH_TRACING		Named enumeration value for the corresponding HRL subsystem.
HRL_GI_RAY_TRACING		Named enumeration value for the corresponding HRL subsystem.

HRL_EAntialiasingMode

Enumerator	Value	Meaning
HRL_ANTIALIASING_OFF	0	Named enumeration value for the corresponding HRL subsystem.
HRL_ANTIALIASING_2X	2	Named enumeration value for the corresponding HRL subsystem.
HRL_ANTIALIASING_4X	4	Named enumeration value for the corresponding HRL subsystem.
HRL_ANTIALIASING_8X	8	Named enumeration value for the corresponding HRL subsystem.

HRL_EDebugView

Enumerator	Value	Meaning
HRL_DEBUG_VIEW_NONE	0x0060	Disables or represents the absence of the corresponding feature/state.
HRL_DEBUG_VIEW_UNLIT		Displays geometry without lighting contribution.
HRL_DEBUG_VIEW_NORMAL		Visualizes surface normals.
HRL_DEBUG_VIEW_LIGHTS		Visualizes scene lighting.
HRL_DEBUG_VIEW_LIGHTING	HRL_DEBUG_VIEW_LIGHTS	Visualizes scene lighting.
HRL_DEBUG_VIEW_WIREFRAME		Renders geometry as wireframe.
HRL_DEBUG_VIEW_LOD		Visualizes/queries level-of-detail behavior.

HRL_ELODMode

Enumerator	Value	Meaning
HRL_LOD_DISTANCE	0	Visualizes/queries level-of-detail behavior.
HRL_LOD_SCREEN_SIZE		Visualizes/queries level-of-detail behavior.

HRL_EError

Enumerator	Value	Meaning
HRL_NO_ERROR	0x0070	Named enumeration value for the corresponding HRL subsystem.
HRL_ERROR_INVALID_ID		Named enumeration value for the corresponding HRL subsystem.
HRL_INVALID_ENUM		Named enumeration value for the corresponding HRL subsystem.
HRL_INVALID_VALUE		Named enumeration value for the corresponding HRL subsystem.
HRL_INVALID_OPERATION		Named enumeration value for the corresponding HRL subsystem.
HRL_INVALID_BACKEND_OPERATION		Named enumeration value for the corresponding HRL subsystem.
HRL_SHADER_COMPILE_FAIL		Named enumeration value for the corresponding HRL subsystem.
HRL_OUT_OF_MEMORY		Named enumeration value for the corresponding HRL subsystem.
HRL_INVALID_FILE_FORMAT		Named enumeration value for the corresponding HRL subsystem.

HRL_ESeverity

Enumerator	Value	Meaning
HRL_SEVERITY_WEAK_WARNING	0x0080	Named enumeration value for the corresponding HRL subsystem.
HRL_SEVERITY_WARNING		Named enumeration value for the corresponding HRL subsystem.
HRL_SEVERITY_ERROR		Named enumeration value for the corresponding HRL subsystem.
HRL_SEVERITY_FATAL		Named enumeration value for the corresponding HRL subsystem.

HRL_EFogType

Enumerator	Value	Meaning
HRL_FOG_LINEAR	0x0090	Selects the linear interpretation/mode.
HRL_FOG_EXPONENTIAL		Selects the exponential interpretation/mode.
HRL_FOG_EXP_SQUARED		Selects the squared exponential interpretation/mode.

HRL_EWidgetState

Enumerator	Value	Meaning
HRL_WIDGET_STATE_IDLE	(1 << 0)	Named enumeration value for the corresponding HRL subsystem.
HRL_WIDGET_STATE_HOVERED	(1 << 1)	Named enumeration value for the corresponding HRL subsystem.
HRL_WIDGET_STATE_PRESSED	(1 << 2)	Named enumeration value for the corresponding HRL subsystem.

HRL_EWidgetType

Enumerator	Value	Meaning
HRL_WIDGET_BUTTON	0x00A0	Named enumeration value for the corresponding HRL subsystem.
HRL_WIDGET_LABEL		Named enumeration value for the corresponding HRL subsystem.
HRL_WIDGET_IMAGE		Named enumeration value for the corresponding HRL subsystem.
HRL_WIDGET_SLIDER		Named enumeration value for the corresponding HRL subsystem.

Enumerator	Value	Meaning
HRL_WIDGET_CHECKBOX		Named enumeration value for the corresponding HRL subsystem.
HRL_WIDGET_PROGRESSBAR		Named enumeration value for the corresponding HRL subsystem.

HRL_EGizmoAxis

Enumerator	Value	Meaning
HRL_GIZMO_AXIS_NONE	0	Disables or represents the absence of the corresponding feature/state.
HRL_GIZMO_AXIS_X	1 << 0	Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_AXIS_Y	1 << 1	Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_AXIS_Z	1 << 2	Named enumeration value for the corresponding HRL subsystem.

HRL_EGizmoMode

Enumerator	Value	Meaning
HRL_GIZMO_MODE_TRANSLATE	1 << 0	Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_MODE_ROTATE	1 << 1	Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_MODE_SCALE	1 << 2	Named enumeration value for the corresponding HRL subsystem.

HRL_EGizmoSpace

Enumerator	Value	Meaning
HRL_GIZMO_SPACE_WORLD	0	Interprets the quantity in world space.
HRL_GIZMO_SPACE_LOCAL		Interprets the quantity in local/object space.

HRL_EGizmoPart

Enumerator	Value	Meaning
HRL_GIZMO_PART_NONE	0	Disables or represents the absence of the corresponding feature/state.
HRL_GIZMO_PART_X		Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_PART_Y		Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_PART_Z		Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_PART_CENTER		Named enumeration value for the corresponding HRL subsystem.

HRL_EGizmoOperation

Enumerator	Value	Meaning
HRL_GIZMO_OPERATION_NONE	0	Disables or represents the absence of the corresponding feature/state.
HRL_GIZMO_OPERATION_TRANSLATE	HRL_GIZMO_MODE_TRANSLATE	Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_OPERATION_ROTATE	HRL_GIZMO_MODE_ROTATE	Named enumeration value for the corresponding HRL subsystem.
HRL_GIZMO_OPERATION_SCALE	HRL_GIZMO_MODE_SCALE	Named enumeration value for the corresponding HRL subsystem.

HRL_EVFXSimulationSpace

=====

=====

VFX / PARTICLE SYSTEMS

=====

=====

Enumerator	Value	Meaning
HRL_VFX_SIMULATION_LOCAL	0	Interprets the quantity in local/object space.
HRL_VFX_SIMULATION_WORLD		Interprets the quantity in world space.

HRL_EVFXRenderMode

Enumerator	Value	Meaning
HRL_VFX_RENDER_BILLBOARD	0	Renders particles as camera-facing quads.
HRL_VFX_RENDER_STRETCHED_BILLBOARD		Renders particles as camera-facing quads.
HRL_VFX_RENDER_MESH		Renders particles using an existing mesh.

HRL_EVFXBlendMode

Enumerator	Value	Meaning
HRL_VFX_BLEND_ALPHA	0	Standard alpha blending/compositing.
HRL_VFX_BLEND_ADDITIVE		Additive blending/compositing.
HRL_VFX_BLEND_MULTIPLY		Multiplicative blending/compositing.

HRL_EVFXSpawnShape

Enumerator	Value	Meaning
HRL_VFX_SHAPE_POINT	0	Point-shaped source/emitter/light or center part, depending on enum family.
HRL_VFX_SHAPE_SPHERE		Named enumeration value for the corresponding HRL subsystem.
HRL_VFX_SHAPE_BOX		Named enumeration value for the corresponding HRL subsystem.
HRL_VFX_SHAPE_CYLINDER		Named enumeration value for the corresponding HRL subsystem.
HRL_VFX_SHAPE_CONE		Named enumeration value for the corresponding HRL subsystem.

HRL_ESliderOrientation

Enumerator	Value	Meaning
HRL_SLIDER_HORIZONTAL	0	Named enumeration value for the corresponding HRL subsystem.
HRL_SLIDER_VERTICAL		Named enumeration value for the corresponding HRL subsystem.

HRL_EFBXTextureType

Resource texture kind as reported by ufbx.

Enumerator	Value	Meaning
HRL_FBX_TEXTURE_FILE	0	Named enumeration value for the corresponding HRL subsystem.
HRL_FBX_TEXTURE_LAYERED		Named enumeration value for the corresponding HRL subsystem.
HRL_FBX_TEXTURE_PROCEDURAL		Named enumeration value for the corresponding HRL subsystem.
HRL_FBX_TEXTURE_SHADER		Named enumeration value for the corresponding HRL subsystem.

HRL_EFBXMaterialTextureSlot

Texture slots understood by HRL's built-in 3D material shader.

Enumerator	Value	Meaning
HRL_FBX_MATERIAL_ALBEDO	0	Named enumeration value for the corresponding HRL subsystem.

Enumerator	Value	Meaning
HRL_FBX_MATERIAL_NORMAL		Visualizes surface normals.
HRL_FBX_MATERIAL_SPECULAR		Named enumeration value for the corresponding HRL subsystem.
HRL_FBX_MATERIAL_ROUGHNESS		Named enumeration value for the corresponding HRL subsystem.
HRL_FBX_MATERIAL_METALLIC		Named enumeration value for the corresponding HRL subsystem.
HRL_FBX_MATERIAL_ALPHA		Standard alpha blending/compositing.
HRL_FBX_MATERIAL_TEXTURE_SLOT_COUNT		Named enumeration value for the corresponding HRL subsystem.

4. Constants and Built-in Identifiers

Macro	Value	Purpose
HRL_SKELETAL_MAX_INFLUENCES	4	Maximum number of bone influences stored per skeletal vertex.
HRL_MAX_SKELETAL_BONES	128	Maximum number of bones supported by the OpenGL 3.3 skeletal renderer.
HRL_MAX_VOLUMETRIC_FOGS	64	Maximum number of localized volumetric fog objects rendered per scene.
HRL_FALSE	((int)0)	Public HRL compile-time constant.
HRL_TRUE	((int)1)	Public HRL compile-time constant.
HRL_INVALID_ID	((HRL_id)-1)	Public HRL compile-time constant.
HRL_T_ALBEDO	"T_Albedo"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_NORMAL	"T_Normal"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_SPECULAR	"T_Specular"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_ROUGHNESS	"T_Roughness"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_METALLIC	"T_Metallic"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_AMBIENT_OCCLUSION	"T_AO"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_ALPHA	"T_Alpha"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_EMISSIVE	"T_Emissive"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_SHADOW_MAP	"T_ShadowMap"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_T_CUBE_MAP	"T_CubeMap"	Built-in material texture/uniform semantic name used by HRL's standard shaders.
HRL_MATERIAL_PARAM_TWO_SIDED	"TwoSided"	Material parameter or texture-slot key consumed by HRL material functions.
HRL_MOUSE_BUTTON_LEFT	0	Mouse callback constants. The callback receives window-relative pixel coordinates through HRL_MouseMovedCallback and button transitions through HRL_MouseButtonCallback.
HRL_MOUSE_BUTTON_RIGHT	1	Mouse input constant used by HRL_MouseButtonCallback.
HRL_MOUSE_BUTTON_MIDDLE	2	Mouse input constant used by HRL_MouseButtonCallback.
HRL_MOUSE_RELEASE	0	Mouse input constant used by HRL_MouseButtonCallback.
HRL_MOUSE_PRESS	1	Mouse input constant used by HRL_MouseButtonCallback.
HRL_SPRITE_SHADER	(UINT32_MAX)	Default Shaders (HRL reserve theses ID)
HRL_MESH_2D_SHADER	(UINT32_MAX - 1)	Reserved shader identifier for a built-in HRL rendering program.
HRL_MESH_3D_SHADER	(UINT32_MAX - 2)	Reserved shader identifier for a built-in HRL rendering program.
HRL_DEBUG_SHADER	(UINT32_MAX - 3)	Reserved shader identifier for a built-in HRL rendering program.
HRL_DEFAULT_POST_PROCESS_SHADER	(UINT32_MAX - 4)	Reserved shader identifier for a built-in HRL rendering program.
HRL_SKINNED_3D_MESH_SHADER	(UINT32_MAX - 5)	Reserved shader identifier for a built-in HRL rendering program.
HRL_MATERIAL_TEXTURE_SS_DISPLACEMENT_MAPPING	"SS_DISPLACEMENT_MAPPING"	Material parameter or texture-slot key consumed by HRL material functions.

Macro	Value	Purpose
HRL_MATERIAL_PARAM_SS_DISPLACEMENT_ENABLED	"ss_displacement_enabled"	Opt-in switch for screen-space displacement. Defaults to disabled.
HRL_MATERIAL_PARAM_SS_DISPLACEMENT_STRENGTH	"SSDisplacementStrength"	Material parameter or texture-slot key consumed by HRL material functions.
HRL_MATERIAL_PARAM_SS_DISPLACEMENT_SCALE	"SSDisplacementScale"	Material parameter or texture-slot key consumed by HRL material functions.
HRL_MATERIAL_PARAM_SS_DISPLACEMENT_OPACITY	"SSDisplacementOpacity"	Material parameter or texture-slot key consumed by HRL material functions.

5. Complete API Index

Function	Subsystem	Source line
HRL_Init	INITIALIZATION / LIFECYCLE	388
HRL_InitContext	INITIALIZATION / LIFECYCLE	396
HRL_Shutdown	INITIALIZATION / LIFECYCLE	402
HRL_BeginFrame	INITIALIZATION / LIFECYCLE	408
HRL_EndFrame	INITIALIZATION / LIFECYCLE	414
HRL_WindowResizeCallback	INITIALIZATION / LIFECYCLE	423
HRL_GetLastError	ERROR HANDLING	436
HRL_ErrorEnumToString	ERROR HANDLING	443
HRL_SeverityEnumToString	ERROR HANDLING	450
HRL_RegisterErrorCallback	ERROR HANDLING	458
HRL_CreateMeshSprite	MESHES / SPRITES	471
HRL_SetMeshPivotPoint	MESHES / SPRITES	478
HRL_SetSpriteRegion	MESHES / SPRITES	485
HRL_CreateMesh3D	MESHES / SPRITES	504
HRL_CreateSkeletalMesh	MESHES / SPRITES	512
HRL_GetVertex3DFromFBX	MESHES / SPRITES	533
HRL_FreeVertex3DFromFBX	MESHES / SPRITES	542
HRL_GetSkeletalMeshFromFBX	MESHES / SPRITES	551
HRL_FreeSkeletalMeshData	MESHES / SPRITES	556
HRL_LoadFBXResources	FBX RESOURCES / MATERIALS	613
HRL_FreeFBXResources	FBX RESOURCES / MATERIALS	616
HRL_GetFBXMaterialCount	FBX RESOURCES / MATERIALS	618
HRL_GetFBXMaterial	FBX RESOURCES / MATERIALS	620
HRL_FindFBXMaterial	FBX RESOURCES / MATERIALS	622
HRL_GetFBXTextureCount	FBX RESOURCES / MATERIALS	624
HRL_GetFBXTexture	FBX RESOURCES / MATERIALS	626
HRL_FindFBXTexture	FBX RESOURCES / MATERIALS	628
HRL_GetFBXMaterialTexture	FBX RESOURCES / MATERIALS	629
HRL_CreateTextureFromFBX	FBX RESOURCES / MATERIALS	636
HRL_CreateMaterialFromFBX	FBX RESOURCES / MATERIALS	639
HRL_CreateMaterialFromFBXWithShader	FBX RESOURCES / MATERIALS	642
HRL_CreateMaterialFromFBXIndexed	FBX RESOURCES / MATERIALS	647
HRL_CreateMaterialFromFBXIndexedWithShader	FBX RESOURCES / MATERIALS	652
HRL_CreateMesh	FBX RESOURCES / MATERIALS	663
HRL_CreateMeshFromFile	FBX RESOURCES / MATERIALS	671
HRL_DeleteMesh	FBX RESOURCES / MATERIALS	677
HRL_IsValidMesh	FBX RESOURCES / MATERIALS	684
HRL_SetMeshMaterial	FBX RESOURCES / MATERIALS	691
HRL_SetMeshLocation	FBX RESOURCES / MATERIALS	696
HRL_SetMeshRotation	FBX RESOURCES / MATERIALS	704
HRL_SetMeshScale	FBX RESOURCES / MATERIALS	709
HRL_GetMeshCameraDistance	FBX RESOURCES / MATERIALS	712
HRL_SetMeshLODAutomatic	FBX RESOURCES / MATERIALS	715
HRL_SetMeshLODMode	FBX RESOURCES / MATERIALS	717
HRL_SetMeshLODLevels	FBX RESOURCES / MATERIALS	719
HRL_SetMeshLODDistance	FBX RESOURCES / MATERIALS	721

Function	Subsystem	Source line
HRL_SetMeshLODScale	FBX RESOURCES / MATERIALS	723
HRL_SetMeshLODMinDistance	FBX RESOURCES / MATERIALS	724
HRL_SetMeshLODMaxDistance	FBX RESOURCES / MATERIALS	725
HRL_SetMeshLODScreenThreshold	FBX RESOURCES / MATERIALS	727
HRL_SetMeshLODScreenScale	FBX RESOURCES / MATERIALS	729
HRL_SetMeshLODHysteresis	FBX RESOURCES / MATERIALS	731
HRL_SetMeshLODOVERRIDE	FBX RESOURCES / MATERIALS	733
HRL_ForceMeshLODRebuild	FBX RESOURCES / MATERIALS	735
HRL_GetMeshLODCount	FBX RESOURCES / MATERIALS	736
HRL_GetMeshLODVertexCount	FBX RESOURCES / MATERIALS	737
HRL_GetMeshLODTriangleCount	FBX RESOURCES / MATERIALS	738
HRL_GetMeshLODLevel	FBX RESOURCES / MATERIALS	740
HRL_GetSkeletalBoneCount	SKELETAL MESHES	746
HRL_GetSkeletalBone	SKELETAL MESHES	747
HRL_FindSkeletalBone	SKELETAL MESHES	748
HRL_GetSkeletalAnimationCount	SKELETAL MESHES	750
HRL_GetSkeletalAnimation	SKELETAL MESHES	751
HRL_FindSkeletalAnimation	SKELETAL MESHES	752
HRL_PlaySkeletalAnimation	SKELETAL MESHES	754
HRL_StopSkeletalAnimation	SKELETAL MESHES	755
HRL_SetSkeletalAnimationTime	SKELETAL MESHES	756
HRL_SetSkeletalAnimationSpeed	SKELETAL MESHES	757
HRL_SetSkeletalAnimationLoop	SKELETAL MESHES	758
HRL_GetCurrentSkeletalAnimation	SKELETAL MESHES	759
HRL_GetSkeletalAnimationTime	SKELETAL MESHES	760
HRL_IsSkeletalAnimationPlaying	SKELETAL MESHES	761
HRL_UpdateSkeletalAnimations	SKELETAL MESHES	764
HRL_SetSpriteDrawOrder	SKELETAL MESHES	771
HRL_CreateLight	LIGHTS	783
HRL_DeleteLight	LIGHTS	789
HRL_IsValidLight	LIGHTS	796
HRL_SetLightColor	LIGHTS	802
HRL_SetLightIntensity	LIGHTS	808
HRL_SetLightAttenuation	LIGHTS	815
HRL_SetLightLocation	LIGHTS	821
HRL_SetLightRotation	LIGHTS	827
HRL_SetLightCastShadows	LIGHTS	833
HRL_SetLightShadowBias	LIGHTS	838
HRL_SetLightShadowStrength	LIGHTS	845
HRL_SetLightShadowResolution	LIGHTS	852
HRL_SetSpotLightInnerCutoff	LIGHTS	859
HRL_SetSpotLightOuterCutoff	LIGHTS	861
HRL_CreateTexture	TEXTURES	875
HRL_CreateTextureAsync	TEXTURES	878
HRL_IsTextureReady	TEXTURES	882
HRL_WaitForTexture	TEXTURES	885
HRL_DeleteTexture	TEXTURES	891
HRL_IsValidTexture	TEXTURES	898

Function	Subsystem	Source line
HRL_ReloadTexture	TEXTURES	907
HRL_GetTextureSize	TEXTURES	914
HRL_SetTextureMinFilter	TEXTURES	920
HRL_SetTextureMagFilter	TEXTURES	926
HRL_CreateTextureFromText	TEXTURES	938
HRL_ClearScreen	SCENES	953
HRL_CreateScene	SCENES	961
HRL_DeleteScene	SCENES	967
HRL_IsValidScene	SCENES	974
HRL_SetGlobalIlluminationEnabled	SCENES	977
HRL_SetGlobalIlluminationMethod	SCENES	979
HRL_IsGlobalIlluminationEnabled	SCENES	981
HRL_GetGlobalIlluminationMethod	SCENES	983
HRL_IsGlobalIlluminationMethodSupported	SCENES	985
HRL_GetGlobalIlluminationSupportedMethods	SCENES	987
HRL_ResizeSceneTexture	SCENES	993
HRL_SetSkySphereEnabled	SCENES	1001
HRL_SetSkySphereColors	SCENES	1007
HRL_SetSkySphereRotation	SCENES	1017
HRL_SetSkySphereTexture	SCENES	1023
HRL_SetEnvironmentMappingEnabled	SCENES	1029
HRL_SetEnvironmentMap	SCENES	1035
HRL_EnableColorPickingBuffer	SCENES	1043
HRL_GetHoveredObject	SCENES	1053
HRL_CreatePostProcess	POST PROCESSING	1066
HRL_DeletePostProcess	POST PROCESSING	1072
HRL_IsValidPostProcess	POST PROCESSING	1079
HRL_CreateShader	SHADERS	1094
HRL_CreateShaderAsync	SHADERS	1097
HRL_IsShaderReady	SHADERS	1101
HRL_WaitForShader	SHADERS	1104
HRL_WaitForAllAsyncResources	SHADERS	1107
HRL_DeleteShader	SHADERS	1113
HRL_IsValidShader	SHADERS	1120
HRL_CreateMaterial	MATERIALS	1133
HRL_DeleteMaterial	MATERIALS	1139
HRL_IsValidMaterial	MATERIALS	1146
HRL_MaterialSetInt	MATERIALS	1151
HRL_MaterialSetTexture	MATERIALS	1156
HRL_MaterialSetBool	MATERIALS	1181
HRL_MaterialSetFloat	MATERIALS	1186
HRL_MaterialSetVec2	MATERIALS	1191
HRL_MaterialSetVec3	MATERIALS	1196
HRL_MaterialSetVec4	MATERIALS	1201
HRL_MaterialSetEmissiveColor	MATERIALS	1208
HRL_CreateViewport	VIEWPORTS	1222
HRL_DeleteViewport	VIEWPORTS	1228
HRL_IsValidViewport	VIEWPORTS	1235

Function	Subsystem	Source line
HRL_SetViewportCamera	VIEWPORTS	1241
HRL_SetViewportRect	VIEWPORTS	1247
HRL_CreateCamera	CAMERA	1259
HRL_DeleteCamera	CAMERA	1265
HRL_IsValidCamera	CAMERA	1272
HRL_SetCameraType	CAMERA	1278
HRL_SetCameraOrthoVertical	CAMERA	1284
HRL_SetCameraPerspectiveFov	CAMERA	1290
HRL_SetCameraNearPlane	CAMERA	1296
HRL_SetCameraFarPlane	CAMERA	1302
HRL_SetCameraLocation	CAMERA	1307
HRL_SetCameraRotation	CAMERA	1313
HRL_CreateGizmo	GIZMOS	1325
HRL_DeleteGizmo	GIZMOS	1326
HRL_IsValidGizmo	GIZMOS	1327
HRL_SetGizmoPosition	GIZMOS	1329
HRL_GetGizmoPosition	GIZMOS	1330
HRL_SetGizmoRotation	GIZMOS	1331
HRL_GetGizmoRotation	GIZMOS	1332
HRL_SetGizmoScale	GIZMOS	1333
HRL_GetGizmoScale	GIZMOS	1334
HRL_SetGizmoMode	GIZMOS	1336
HRL_GetGizmoMode	GIZMOS	1337
HRL_SetGizmoSpace	GIZMOS	1338
HRL_GetGizmoSpace	GIZMOS	1339
HRL_SetGizmoTranslateVisible	GIZMOS	1341
HRL_SetGizmoRotateVisible	GIZMOS	1342
HRL_SetGizmoScaleVisible	GIZMOS	1343
HRL_SetGizmoTranslateAxes	GIZMOS	1344
HRL_SetGizmoRotateAxes	GIZMOS	1345
HRL_SetGizmoScaleAxes	GIZMOS	1346
HRL_SetGizmoSize	GIZMOS	1347
HRL_SetGizmoScreenSize	GIZMOS	1348
HRL_SetGizmoUseScreenSize	GIZMOS	1349
HRL_SetGizmoVisible	GIZMOS	1350
HRL_SetGizmoEnabled	GIZMOS	1351
HRL_SetGizmoAxisColor	GIZMOS	1353
HRL_SetGizmoCenterColor	GIZMOS	1354
HRL_SetGizmoHoverColor	GIZMOS	1355
HRL_GetGizmoHoveredPart	GIZMOS	1356
HRL_GetGizmoActivePart	GIZMOS	1357
HRL_GetGizmoHoveredOperation	GIZMOS	1358
HRL_GetGizmoActiveOperation	GIZMOS	1359
HRL_SetGizmoRotateArcDegrees	GIZMOS	1360
HRL_SetGizmoChangedCallback	GIZMOS	1361
HRL_CreateVFXSystem	VFX / PARTICLE SYSTEMS	1367
HRL_DeleteVFXSystem	VFX / PARTICLE SYSTEMS	1368
HRL_IsValidVFXSystem	VFX / PARTICLE SYSTEMS	1369

Function	Subsystem	Source line
HRL_SetVFXSystemPosition	VFX / PARTICLE SYSTEMS	1371
HRL_GetVFXSystemPosition	VFX / PARTICLE SYSTEMS	1372
HRL_SetVFXSystemRotation	VFX / PARTICLE SYSTEMS	1373
HRL_GetVFXSystemRotation	VFX / PARTICLE SYSTEMS	1374
HRL_SetVFXSystemScale	VFX / PARTICLE SYSTEMS	1375
HRL_GetVFXSystemScale	VFX / PARTICLE SYSTEMS	1376
HRL_PlayVFXSystem	VFX / PARTICLE SYSTEMS	1378
HRL_StopVFXSystem	VFX / PARTICLE SYSTEMS	1379
HRL_PauseVFXSystem	VFX / PARTICLE SYSTEMS	1380
HRL_ResetVFXSystem	VFX / PARTICLE SYSTEMS	1381
HRL_SetVFXSystemLooping	VFX / PARTICLE SYSTEMS	1382
HRL_SetVFXSystemTimeScale	VFX / PARTICLE SYSTEMS	1383
HRL_SetVFXSystemEnabled	VFX / PARTICLE SYSTEMS	1384
HRL_SetVFXSystemAutoUpdate	VFX / PARTICLE SYSTEMS	1385
HRL_SetVFXSystemDuration	VFX / PARTICLE SYSTEMS	1386
HRL_CreateVFXEmitter	VFX / PARTICLE SYSTEMS	1388
HRL_DeleteVFXEmitter	VFX / PARTICLE SYSTEMS	1389
HRL_IsValidVFXEmitter	VFX / PARTICLE SYSTEMS	1390
HRL_SetVFXEmitterEnabled	VFX / PARTICLE SYSTEMS	1392
HRL_SetVFXEmitterPosition	VFX / PARTICLE SYSTEMS	1393
HRL_SetVFXEmitterRotation	VFX / PARTICLE SYSTEMS	1394
HRL_SetVFXEmitterMaxParticles	VFX / PARTICLE SYSTEMS	1395
HRL_SetVFXEmitterSpawnRate	VFX / PARTICLE SYSTEMS	1396
HRL_SetVFXEmitterBurst	VFX / PARTICLE SYSTEMS	1397
HRL_AddVFXBurst	VFX / PARTICLE SYSTEMS	1398
HRL_ClearVFXBursts	VFX / PARTICLE SYSTEMS	1399
HRL_SetVFXEmitterLifetime	VFX / PARTICLE SYSTEMS	1400
HRL_SetVFXEmitterSpawnShape	VFX / PARTICLE SYSTEMS	1401
HRL_SetVFXEmitterShapeRadius	VFX / PARTICLE SYSTEMS	1402
HRL_SetVFXEmitterShapeSize	VFX / PARTICLE SYSTEMS	1403
HRL_SetVFXEmitterShapeAngle	VFX / PARTICLE SYSTEMS	1404
HRL_SetVFXEmitterInitialVelocity	VFX / PARTICLE SYSTEMS	1405
HRL_SetVFXEmitterInitialSpeed	VFX / PARTICLE SYSTEMS	1407
HRL_SetVFXEmitterInitialRotation	VFX / PARTICLE SYSTEMS	1408
HRL_SetVFXEmitterAngularVelocity	VFX / PARTICLE SYSTEMS	1410
HRL_SetVFXGravity	VFX / PARTICLE SYSTEMS	1412
HRL_SetVFXDrag	VFX / PARTICLE SYSTEMS	1413
HRL_SetVFXForce	VFX / PARTICLE SYSTEMS	1414
HRL_SetVFXNoise	VFX / PARTICLE SYSTEMS	1415
HRL_SetVFXEmitterRenderMode	VFX / PARTICLE SYSTEMS	1416
HRL_SetVFXEmitterTexture	VFX / PARTICLE SYSTEMS	1417
HRL_SetVFXEmitterMaterial	VFX / PARTICLE SYSTEMS	1418
HRL_SetVFXEmitterMesh	VFX / PARTICLE SYSTEMS	1419
HRL_SetVFXEmitterMeshScale	VFX / PARTICLE SYSTEMS	1420
HRL_SetVFXEmitterMeshRotation	VFX / PARTICLE SYSTEMS	1421
HRL_SetVFXEmitterBlendMode	VFX / PARTICLE SYSTEMS	1422
HRL_SetVFXEmitterParticleSize	VFX / PARTICLE SYSTEMS	1423
HRL_SetVFXEmitterStretch	VFX / PARTICLE SYSTEMS	1424

Function	Subsystem	Source line
HRL_SetVFXEmitterSimulationSpace	VFX / PARTICLE SYSTEMS	1425
HRL_CreateVFXColorCurve	VFX / PARTICLE SYSTEMS	1427
HRL_CreateVFXFloatCurve	VFX / PARTICLE SYSTEMS	1428
HRL_DeleteVFXCurve	VFX / PARTICLE SYSTEMS	1429
HRL_IsValidVFXCurve	VFX / PARTICLE SYSTEMS	1430
HRL_AddVFXColorKey	VFX / PARTICLE SYSTEMS	1431
HRL_AddVFXFloatKey	VFX / PARTICLE SYSTEMS	1432
HRL_ClearVFXColorKeys	VFX / PARTICLE SYSTEMS	1433
HRL_ClearVFXFloatKeys	VFX / PARTICLE SYSTEMS	1434
HRL_SetVFXEmitterColorCurve	VFX / PARTICLE SYSTEMS	1435
HRL_SetVFXEmitterSizeCurve	VFX / PARTICLE SYSTEMS	1436
HRL_SetVFXEmitterRotationCurve	VFX / PARTICLE SYSTEMS	1437
HRL_SetVFXCollisionEnabled	VFX / PARTICLE SYSTEMS	1439
HRL_SetVFXCollisionRestitution	VFX / PARTICLE SYSTEMS	1440
HRL_SetVFXCollisionFriction	VFX / PARTICLE SYSTEMS	1441
HRL_SetVFXCollisionScene	VFX / PARTICLE SYSTEMS	1442
HRL_UpdateVFX	VFX / PARTICLE SYSTEMS	1445
HRL_SetFogEnabled	FOG */	1457
HRL_SetFogMode	FOG */	1464
HRL_SetFogColor	FOG */	1471
HRL_SetFogDensity	FOG */	1479
HRL_SetFogLinearRange	FOG */	1488
HRL_CreateVolumetricFog	FOG */	1499
HRL_DeleteVolumetricFog	FOG */	1502
HRL_IsValidVolumetricFog	FOG */	1505
HRL_SetVolumetricFogEnabled	FOG */	1508
HRL_SetVolumetricFogPosition	FOG */	1510
HRL_SetVolumetricFogRadius	FOG */	1512
HRL_SetVolumetricFogDensity	FOG */	1514
HRL_SetVolumetricFogColor	FOG */	1516
HRL_SetVolumetricFogSteps	FOG */	1518
HRL_SetGlobalVolumetricFogEnabled	FOG */	1522
HRL_SetGlobalVolumetricFogDensity	FOG */	1524
HRL_SetGlobalVolumetricFogColor	FOG */	1526
HRL_SetGlobalVolumetricFogSteps	FOG */	1528
HRL_SetGodRaysEnabled	FOG */	1531
HRL_SetGodRaysPosition	FOG */	1533
HRL_SetGodRaysColor	FOG */	1535
HRL_SetGodRaysDensity	FOG */	1537
HRL_SetGodRaysDecay	FOG */	1539
HRL_SetGodRaysWeight	FOG */	1541
HRL_SetGodRaysSamples	FOG */	1543
HRL_SetAmbientOcclusionEnabled	FOG */	1547
HRL_SetAmbientOcclusionStrength	FOG */	1548
HRL_SetAmbientOcclusionRadius	FOG */	1549
HRL_SetAmbientOcclusionBias	FOG */	1550
HRL_SetAmbientOcclusionPower	FOG */	1551
HRL_GetProjectionMatrix	MATRICES	1561

Function	Subsystem	Source line
HRL_GetViewMatrix	MATRICES	1567
HRL_GetModelMatrix	MATRICES	1574
HRL_DrawSceneAsDebugMode	DEBUG VIEWS	1586
HRL_SetDebugLineThickness	DEBUG GEOMETRY	1597
HRL_DrawDebugSegment	DEBUG GEOMETRY	1606
HRL_DrawDebugPolygon	DEBUG GEOMETRY	1618
HRL_DrawDebugCircle	DEBUG GEOMETRY	1629
HRL_DrawDebugCapsule	DEBUG GEOMETRY	1642
HRL_DrawDebugPoint	DEBUG GEOMETRY	1653
HRL_TakeScreenshot	UTILITY	1670
HRL_SetAntialiasingMode	UTILITY	1677
HRL_CreateFont	FONTS	1690
HRL_DeleteFont	FONTS	1696
HRL_IsValidFont	FONTS	1703
HRL_MouseMovedCallback	UI	1712
HRL_MouseButtonCallback	UI	1713
HRL_CreateWidget	UI	1718
HRL_DeleteWidget	UI	1719
HRL_IsValidWidget	UI	1720
HRL_SetWidgetPosition	UI	1722
HRL_SetWidgetSize	UI	1723
HRL_SetWidgetAlpha	UI	1724
HRL_SetWidgetAnchor	UI	1725
HRL_SetWidgetVisible	UI	1726
HRL_SetWidgetEnabled	UI	1727
HRL_SetWidgetZIndex	UI	1728
HRL_IsWidgetHovered	UI	1729
HRL_SetButtonClickable	UI	1732
HRL_SetButtonText	UI	1733
HRL_SetButtonTextSize	UI	1734
HRL_SetButtonTextTintColor	UI	1735
HRL_SetButtonTextFont	UI	1736
HRL_SetButtonBackgroundTexture	UI	1737
HRL_SetButtonBackgroundTintColor	UI	1738
HRL_SetButtonPressedCallback	UI	1740
HRL_SetLabelText	UI	1743
HRL_SetLabelTextSize	UI	1744
HRL_SetLabelFont	UI	1745
HRL_SetLabelTintColor	UI	1746
HRL_SetImageTexture	UI	1749
HRL_SetImageTintColor	UI	1750
HRL_SetSliderRange	UI	1754
HRL_SetSliderValue	UI	1755
HRL_GetSliderValue	UI	1756
HRL_SetSliderOrientation	UI	1757
HRL_SetSliderClickable	UI	1758
HRL_SetSliderBackgroundColor	UI	1759
HRL_SetSliderFillColor	UI	1760

Function	Subsystem	Source line
HRL_SetSliderHandleColor	UI	1761
HRL_SetSliderChangedCallback	UI	1762
HRL_SetCheckboxChecked	UI	1766
HRL_IsCheckboxChecked	UI	1767
HRL_SetCheckboxClickable	UI	1768
HRL_SetCheckboxBackgroundColor	UI	1769
HRL_SetCheckboxCheckedColor	UI	1770
HRL_SetCheckboxChangedCallback	UI	1771
HRL_SetProgressBarValue	UI	1774
HRL_GetProgressBarValue	UI	1775
HRL_SetProgressBarBackgroundColor	UI	1776
HRL_SetProgressBarFillColor	UI	1777

Initialization & Lifecycle

HRL Init

```
void HRL_Init(HRL_E_APIs _api);
```

Selects the graphics backend to use. Must be called before HRL_InitContext.

Selects the graphics backend that subsequent HRL rendering operations will use. It establishes the global backend selection and is intended to precede context creation.

Parameters

Name	Declaration	Description
_api	HRL_E_APIs _api	One of the HRL_* API constants (e.g. HRL_OPENGL_45, HRL_VULKAN).

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 388.

HRL InitContext

```
void HRL_InitContext(HRL_uint _width, HRL_uint _height, void* _loader);
```

Initializes the rendering context for the given window dimensions.

Initializes the active rendering context for the supplied framebuffer dimensions and platform-specific graphics-function loader. The loader is expected to expose the procedures required by the selected backend.

Parameters

Name	Declaration	Description
_width	HRL_uint _width	Initial framebuffer width in pixels.
_height	HRL_uint _height	Initial framebuffer height in pixels.
_loader	void* _loader	Platform-specific function loader (eg. glfwGetProcAddress).

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 396.

HRL Shutdown

```
void HRL_Shutdown();
```

Releases all resources and shuts down the rendering context.

Shuts down the active HRL runtime and releases resources owned by the rendering context. Call it during application teardown, before destroying the native window/context that HRL depends on.

Must be called before the window is destroyed.

Parameters

This function takes no parameters.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 402.

HRL BeginFrame

```
void HRL_BeginFrame();
```

Prepares the renderer for a new frame. Call at the start of your render loop.

Starts a frame. HRL resets per-frame rendering state and prepares its command/rendering pipeline for scene, widget, debug, and effect submission.

Clears internal per-frame state and begins command recording.

Parameters

This function takes no parameters.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 408.

HRL EndFrame

```
void HRL_EndFrame();
```

Finalizes and submits the current frame. Call at the end of your render loop.

Completes the current frame, submits the accumulated rendering work, and performs the backend-specific end-of-frame operation. In a windowed renderer this is the natural companion to HRL_BeginFrame.

Flushes all draw commands and swaps buffers if applicable.

Parameters

This function takes no parameters.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 414.

HRL WindowResizeCallback

```
void HRL_WindowResizeCallback(int _width, int _height);
```

Notifies HRL of a window/framebuffer resize. Call from your framebuffer-size callback.

Updates HRL with the drawable framebuffer size after a window resize. Supplying the actual framebuffer dimensions keeps viewport and projection calculations correct on HiDPI systems.

The dimensions must be the drawable framebuffer size in pixels (for example,

GLFW framebuffer size), not the logical window size on HiDPI displays.

Parameters

Name	Declaration	Description
_width	int _width	New framebuffer width in pixels.
_height	int _height	New framebuffer height in pixels.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 423.

Error Handling

HRL CErrorCallback

```
typedef void (*HRL_CErrorCallback)(HRL_EError code, HRL_ESeverity severity, const char* detail); HRL_API void HRL_RegisterErrorCallback(HRL_CErrorCallback _callback);
```

@brief Registers a callback invoked whenever an error is raised internally.

Useful for integrating HRL errors into a custom logging or assertion system.

@param _callback Function pointer with signature: void(HRL_Error, HRL_Severity, const char*).

HRL GetLastError

```
HRL_EError HRL_GetLastError(const char** _detail, HRL_ESeverity* _severity);
```

Retrieves the last recorded error, if any.

Returns the most recently recorded HRL error and optionally exposes its human-readable detail string and severity through output pointers. It is the primary post-call diagnostic query for the C API.

Parameters

Name	Declaration	Description
_detail	const char** _detail	Output pointer to a human-readable description string.
_severity	HRL_ESeverity* _severity	Output severity level of the error.

Return value

The HRL_Error code. Returns HRL_NO_ERROR if no error occurred.

Header declaration location: line 436.

HRL ErrorEnumToString

```
const char* HRL_ErrorEnumToString(HRL_EError err);
```

Converts an HRL_Error enum value to its string representation.

Converts an HRL_EError code into a stable, null-terminated symbolic string suitable for logs, diagnostics, or UI.

Parameters

Name	Declaration	Description
err	HRL_EError err	The error code to convert.

Return value

A null-terminated string literal (e.g. "HRL_INVALID_ID").

Header declaration location: line 443.

HRL SeverityEnumToString

```
const char* HRL_SeverityEnumToString(HRL_ESeverity sev);
```

Converts an HRL_Severity enum value to its string representation.

Converts an HRL_ESeverity value into its symbolic textual name.

Parameters

Name	Declaration	Description
sev	HRL_ESeverity sev	The severity level to convert.

Return value

A null-terminated string literal (e.g. "HRL_SEVERITY_FATAL").

Header declaration location: line 450.

HRL RegisterErrorCallback

```
void HRL_RegisterErrorCallback(HRL_CErrorCallback _callback);
```

Installs the application callback used when HRL reports an internal error. The callback receives the error code, severity, and diagnostic detail.

Parameters

Name	Declaration	Description
_callback	HRL_CErrorCallback _callback	Function pointer invoked by the corresponding subsystem when the associated event/state change occurs.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 458.

Meshes, Sprites & FBX

HRL CreateMeshSprite

```
HRL_id HRL_CreateMeshSprite(HRL_id _sceneid);
```

Creates a sprite mesh in the given scene.

Creates a camera-facing sprite mesh attached to the specified scene. The returned handle identifies the new mesh resource.

A sprite is a textured quad that always faces the camera.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	ID of the target scene.

Return value

HRL_id of the new sprite, or HRL_InvalidID on failure.

Header declaration location: line 471.

HRL SetMeshPivotPoint

```
void HRL_SetMeshPivotPoint(HRL_id _meshid, float x, float y, float z);
```

Sets the pivot (origin) point of a mesh or sprite.

Updates the named property of the supplied HRL resource or rendering subsystem.

Coordinates are normalized: (0,0,0) is center, (-0.5,-0.5,0) is top-left.

Affects how translation and rotation are applied to the object.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 478.

HRL SetSpriteRegion

```
void HRL_SetSpriteRegion(HRL_id _meshid, float min_u, float min_v, float max_u, float max_v);
```

Defines the UV region of the texture displayed on a sprite.

Updates the named property of the supplied HRL resource or rendering subsystem.

Useful for sprite atlases. (min_u, min_v) is the top-left corner,

(max_u, max_v) is the bottom-right corner, in normalized [0..1] coordinates.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
min_u	float min_u	Input value controlling the named operation.
min_v	float min_v	Input value controlling the named operation.
max_u	float max_u	Input value controlling the named operation.
max_v	float max_v	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 485.

HRL CreateMesh3D

```
HRL_id HRL_CreateMesh3D( HRL_id _sceneid, const HRL_Vertex3D* _vertices, size_t _vertexCount, const HRL_uint* _indices, size_t _indexCount );
```

Creates a static 3D mesh from caller-owned vertex/index arrays.

Creates a non-skinned 3D triangle mesh from caller-supplied vertex data and optional triangle indices. HRL consumes/copies the submitted geometry so the caller does not need to keep the source arrays alive after the call returns.

The mesh data is copied by HRL and may be released by the caller as soon as the function returns. When `_indexCount` is zero, the vertices are drawn sequentially as triangles. Otherwise `_indices` must contain triangle indices and `_indexCount` must be a multiple of 3.

No standard 3D file format importer is involved in this API.

Parameters

Name	Declaration	Description
<code>_sceneid</code>	<code>HRL_id _sceneid</code>	ID of the target scene.
<code>_vertices</code>	<code>const HRL_Vertex3D* _vertices</code>	Pointer to <code>_vertexCount</code> <code>HRL_Vertex3D</code> values.
<code>_vertexCount</code>	<code>size_t _vertexCount</code>	Number of vertices.
<code>_indices</code>	<code>const HRL_uint* _indices</code>	Optional pointer to <code>_indexCount</code> <code>HRL_uint</code> indices.
<code>_indexCount</code>	<code>size_t _indexCount</code>	Number of indices, or 0 for non-indexed rendering.

Return value

`HRL_id` of the new mesh, or `HRL_INVALID_ID` on failure.

Header declaration location: line 504.

HRL CreateSkeletalMesh

```
HRL_id HRL_CreateSkeletalMesh( HRL_id _sceneid, const HRL_SkeletalMeshData* _data );
```

Creates a skinned 3D mesh from an `HRL_SkeletalMeshData` structure, including bone hierarchy and baked animation data.

Creates a skeletal mesh from CPU-side data returned by the FBX importer.

HRL copies the data and the caller may free the source structure afterwards.

Parameters

Name	Declaration	Description
<code>_sceneid</code>	<code>HRL_id _sceneid</code>	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
<code>_data</code>	<code>const HRL_SkeletalMeshData* _data</code>	Pointer to the caller-provided binary/text data consumed by the operation.

Return value

Returns `HRL_id`. For resource creation and name-lookup functions, `HRL_INVALID_ID` is the failure/not-found sentinel.

Header declaration location: line 512.

HRL GetVertex3DFromFBX

```
HRL_Vertex3D* HRL_GetVertex3DFromFBX( const char* _data, size_t _bufferSize, size_t* _vertexCount );
```

Converts an FBX mesh scene to a flat `HRL_Vertex3D` array.

Decodes an in-memory FBX buffer and returns a newly allocated flat `HRL_Vertex3D` array representing triangle geometry with imported transforms/tangent space applied.

The FBX file is decoded internally by HRL and the resulting geometry is returned as non-indexed triangles. Node transforms are baked into the returned vertex positions and tangent space. The returned buffer must be released with `HRL_FreeVertex3DFromFBX`.

HRL does not expose ufbx types through this API. The FBX implementation is an internal dependency of hrl.cpp.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the complete FBX data in memory.
_bufferSize	size_t _bufferSize	Size of the FBX data buffer in bytes.
_vertexCount	size_t* _vertexCount	Receives the number of returned vertices.

Return value

Newly allocated HRL_Vertex3D array, or NULL on failure.

Header declaration location: line 533.

HRL FreeVertex3DFromFBX

```
void HRL_FreeVertex3DFromFBX(HRL_Vertex3D* _vertices);
```

Frees the buffer returned by HRL_GetVertex3DFromFBX.

Releases the vertex buffer returned by HRL_GetVertex3DFromFBX.

Parameters

Name	Declaration	Description
_vertices	HRL_Vertex3D* _vertices	Caller-provided pointer used as input or output according to the function signature.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 542.

HRL GetSkeletalMeshFromFBX

```
HRL_SkeletalMeshData* HRL_GetSkeletalMeshFromFBX( const char* _data, size_t _bufferSize );
```

Converts the first skinned FBX mesh in an in-memory FBX buffer.

Decodes skeletal mesh information from an in-memory FBX buffer and returns HRL-owned CPU data containing vertices, bones, weights, and baked animation samples.

ufbx is an internal dependency of hrl.cpp. Geometry, bones, weights and baked animation samples are copied into an HRL-owned CPU data structure.

No file path is accepted by this API.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Header declaration location: line 551.

HRL FreeSkeletalMeshData

```
void HRL_FreeSkeletalMeshData(HRL_SkeletalMeshData* _data);
```

Releases the complete nested HRL_SkeletalMeshData allocation returned by the FBX skeletal importer.

Frees all nested allocations returned by HRL_GetSkeletalMeshFromFBX.

Parameters

Name	Declaration	Description
_data	HRL_SkeletalMeshData* _data	Pointer to the caller-provided binary/text data consumed by the operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 556.

FBX Resources & Materials

HRL_LoadFBXResources

```
HRL_FBXResources* HRL_LoadFBXResources(const char* _data, size_t _bufferSize);
```

Loads FBX materials and textures without importing geometry.

Loads FBX material and texture resources into an opaque HRL_FBXResources container without requiring geometry import.

Embedded texture bytes are exposed directly through HRL_FBXTextureInfo.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Header declaration location: line 613.

HRL_FreeFBXResources

```
void HRL_FreeFBXResources(HRL_FBXResources* _resources);
```

Releases an opaque FBX resource container and all metadata/embedded data owned by it.

Releases a resource scene returned by HRL_LoadFBXResources().

Parameters

Name	Declaration	Description
_resources	HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 616.

HRL_GetFBXMaterialCount

```
size_t HRL_GetFBXMaterialCount(const HRL_FBXResources* _resources);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.

Return value

Returns the requested size_t value.

Header declaration location: line 618.

HRL_GetFBXMaterial

```
const HRL_FBXMaterialInfo* HRL_GetFBXMaterial(const HRL_FBXResources* _resources, size_t _index);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Returns read-only material metadata; the pointer is valid while _resources is alive.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.
_index	size_t _index	Count/index value controlling how many elements are processed or which element is addressed.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Pointer lifetime: the returned pointer is a read-only view into HRL-owned/import-owned data and remains usable only while the owning resource/mesh remains alive.

Header declaration location: line 620.

HRL FindFBXMaterial

```
HRL_id HRL_FindFBXMaterial(const HRL_FBXResources* _resources, const char* _name);
```

Searches the associated collection by name and returns the matching resource/index, or HRL_INVALID_ID when no matching entry exists.

Finds a material by its FBX name and returns its material index, or HRL_INVALID_ID.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.
_name	const char* _name	Null-terminated resource or symbol name used for lookup/association.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 622.

HRL GetFBXTextureCount

```
size_t HRL_GetFBXTextureCount(const HRL_FBXResources* _resources);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.

Return value

Returns the requested size_t value.

Header declaration location: line 624.

HRL GetFBXTexture

```
const HRL_FBXTextureInfo* HRL_GetFBXTexture(const HRL_FBXResources* _resources, size_t _index);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Returns read-only texture metadata and embedded bytes; valid while _resources is alive.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.
_index	size_t _index	Count/index value controlling how many elements are processed or which element is addressed.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Pointer lifetime: the returned pointer is a read-only view into HRL-owned/import-owned data and remains usable only while the owning resource/mesh remains alive.

Header declaration location: line 626.

HRL FindFBXTexture

```
HRL_id HRL_FindFBXTexture(const HRL_FBXResources* _resources, const char* _name);
```

Searches the associated collection by name and returns the matching resource/index, or HRL_INVALID_ID when no matching entry exists.

Finds a texture by FBX name or filename and returns its texture index, or HRL_INVALID_ID.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.
_name	const char* _name	Null-terminated resource or symbol name used for lookup/association.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 628.

HRL GetFBXMaterialTexture

```
const HRL_FBXTextureInfo* HRL_GetFBXMaterialTexture( const HRL_FBXResources* _resources, size_t _materialIndex, HRL_EFBXMaterialTextureSlot _slot );
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.
_materialIndex	size_t _materialIndex	Count/index value controlling how many elements are processed or which element is addressed.
_slot	HRL_EFBXMaterialTextureSlot _slot	Input value controlling the named operation.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Pointer lifetime: the returned pointer is a read-only view into HRL-owned/import-owned data and remains usable only while the owning resource/mesh remains alive.

Header declaration location: line 629.

HRL CreateTextureFromFBX

```
HRL_id HRL_CreateTextureFromFBX(const HRL_FBXResources* _resources, size_t _textureIndex);
```

Creates a normal HRL GPU texture from an embedded FBX texture entry.

Creates an HRL GPU texture from an embedded FBX texture resource.

Parameters

Name	Declaration	Description
_resources	const HRL_FBXResources* _resources	Caller-provided pointer used as input or output according to the function signature.
_textureIndex	size_t _textureIndex	Count/index value controlling how many elements are processed or which element is addressed.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 636.

HRL CreateMaterialFromFBX

```
HRL_id HRL_CreateMaterialFromFBX(const char* _data, size_t _bufferSize);
```

Builds a material from the first material described by an FBX buffer and chooses HRL's built-in static or skinned 3D shader based on the imported contents.

Creates a material from the first FBX material. Automatically selects the built-in static or skinned 3D shader from the FBX contents.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 639.

HRL CreateMaterialFromFBXWithShader

```
HRL_id HRL_CreateMaterialFromFBXWithShader( const char* _data, size_t _bufferSize, HRL_id _shaderid );
```

Builds a material from the first FBX material while forcing the caller-selected shader handle.

Creates a material from the first FBX material using a caller-selected shader.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.
_shaderid	HRL_id _shaderid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 642.

HRL CreateMaterialFromFBXIndexed

```
HRL_id HRL_CreateMaterialFromFBXIndexed( const char* _data, size_t _bufferSize, size_t _materialIndex );
```

Builds a material from the indexed FBX material entry, automatically selecting the appropriate built-in 3D shader.

Creates one indexed FBX material and automatically selects the built-in static or skinned 3D shader from the FBX contents.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.
_materialIndex	size_t _materialIndex	Count/index value controlling how many elements are processed or which element is addressed.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 647.

HRL CreateMaterialFromFBXIndexedWithShader

```
HRL_id HRL_CreateMaterialFromFBXIndexedWithShader( const char* _data, size_t _bufferSize, size_t _materialIndex, HRL_id _shaderid );
```

Builds a material from the indexed FBX material entry with a caller-selected shader.

Creates one indexed FBX material using a caller-selected shader.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.
_materialIndex	size_t _materialIndex	Count/index value controlling how many elements are processed or which element is addressed.
_shaderid	HRL_id _shaderid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 652.

HRL CreateMesh

```
HRL_id HRL_CreateMesh(HRL_id _sceneid, HRL_EMeshType _type, const float* _vertices);
```

Legacy generic mesh entry point.

Legacy generic mesh constructor retained for compatibility. Because the signature does not provide a bounded vertex-buffer description, it should be treated as a compatibility/low-level entry point rather than the preferred static 3D creation path.

This prototype does not carry enough information to describe a bounded vertex buffer and therefore remains reserved. Use `HRL_CreateMesh3D` for static 3D geometry.

Parameters

Name	Declaration	Description
<code>_sceneid</code>	<code>HRL_id_sceneid</code>	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
<code>_type</code>	<code>HRL_EMeshType_type</code>	Enumeration value selecting the requested operating mode, type, space, method, or category.
<code>_vertices</code>	<code>const float* _vertices</code>	Caller-provided pointer used as input or output according to the function signature.

Return value

Returns `HRL_id`. For resource creation and name-lookup functions, `HRL_INVALID_ID` is the failure/not-found sentinel.

Header declaration location: line 663.

HRL CreateMeshFromFile

```
HRL_id HRL_CreateMeshFromFile(HRL_id_sceneid, HRL_EMeshType_type, const char* _data, size_t _bufferSize);
```

Reserved generic file-based mesh entry point.

Reserved generic file-based mesh constructor. The public API exposes dedicated FBX conversion/import helpers instead of requiring a file-format-specific implementation here.

This entry point remains reserved. FBX conversion is intentionally exposed separately through `HRL_GetVertex3DFromFBX`.

Parameters

Name	Declaration	Description
<code>_sceneid</code>	<code>HRL_id_sceneid</code>	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
<code>_type</code>	<code>HRL_EMeshType_type</code>	Enumeration value selecting the requested operating mode, type, space, method, or category.
<code>_data</code>	<code>const char* _data</code>	Pointer to the caller-provided binary/text data consumed by the operation.
<code>_bufferSize</code>	<code>size_t _bufferSize</code>	Size of the associated data buffer in bytes.

Return value

Returns `HRL_id`. For resource creation and name-lookup functions, `HRL_INVALID_ID` is the failure/not-found sentinel.

Header declaration location: line 671.

HRL DeleteMesh

```
void HRL_DeleteMesh(HRL_id_meshid);
```

Destroys a mesh and frees its associated GPU resources.

Destroys the referenced eMesh object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
<code>_meshid</code>	<code>HRL_id_meshid</code>	ID of the mesh to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 677.

HRL IsValidMesh

```
int HRL_IsValidMesh(HRL_id _id);
```

Returns whether the given ID refers to a live mesh object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 684.

HRL SetMeshMaterial

```
void HRL_SetMeshMaterial(HRL_id _meshid, HRL_id _matid);
```

Assigns a material to a mesh, controlling how it is shaded.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	ID of the target mesh.
_matid	HRL_id _matid	ID of the material to apply.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 691.

HRL SetMeshLocation

```
void HRL_SetMeshLocation(HRL_id _meshid, float x, float y, float z);
```

Sets the world-space position of a mesh.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 696.

HRL SetMeshRotation

```
void HRL_SetMeshRotation(HRL_id _meshid, float pitch, float yaw, float roll);
```

Sets the rotation of a mesh using Euler angles (in degrees).

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Rotation around the X axis.
yaw	float yaw	Rotation around the Y axis.
roll	float roll	Rotation around the Z axis.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 704.

HRL SetMeshScale

```
void HRL_SetMeshScale(HRL_id _meshid, float x, float y, float z);
```

Sets the scale of a mesh along each local axis.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 709.

HRL GetMeshCameraDistance

```
float HRL_GetMeshCameraDistance(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Returns the minimum world-space distance from the mesh LOD center to any viewport camera.

Uses the same object-center and camera-position metric as distance-based LOD.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested float value.

Header declaration location: line 712.

HRL SetMeshLODAutomatic

```
void HRL_SetMeshLODAutomatic(HRL_id _meshid, int _enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Enables or disables automatic LOD selection for a 3D mesh.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Name	Declaration	Description
_enabled	int_enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 715.

HRL SetMeshLODMode

```
void HRL_SetMeshLODMode(HRL_id _meshid, HRL_ELODMode _mode);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Selects distance-based or projected screen-size LOD selection.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_mode	HRL_ELODMode _mode	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 717.

HRL SetMeshLODLevels

```
void HRL_SetMeshLODLevels(HRL_id _meshid, HRL_uint _levels);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Sets the number of LOD levels, including LOD 0. Range: 1..8.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_levels	HRL_uint _levels	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 719.

HRL SetMeshLODDistance

```
void HRL_SetMeshLODDistance(HRL_id _meshid, float _baseDistance);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Sets the first transition distance in world units.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_baseDistance	float _baseDistance	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 721.

HRL SetMeshLODScale

```
void HRL_SetMeshLODScale(HRL_id _meshid, float _distanceScale);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Multiplies the distance threshold for each next LOD.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_distanceScale	float _distanceScale	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 723.

HRL SetMeshLODMinDistance

```
void HRL_SetMeshLODMinDistance(HRL_id _meshid, float _distance);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_distance	float _distance	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 724.

HRL SetMeshLODMaxDistance

```
void HRL_SetMeshLODMaxDistance(HRL_id _meshid, float _distance);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_distance	float _distance	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 725.

HRL SetMeshLODScreenThreshold

```
void HRL_SetMeshLODScreenThreshold(HRL_id _meshid, float _threshold);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Sets the LOD 0 screen-height threshold in [0,1].

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_threshold	float _threshold	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 727.

HRL SetMeshLODScreenScale

```
void HRL_SetMeshLODScreenScale(HRL_id _meshid, float _scale);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Multiplies the screen-size threshold for each next LOD.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_scale	float _scale	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 729.

HRL SetMeshLODHysteresis

```
void HRL_SetMeshLODHysteresis(HRL_id _meshid, float _hysteresis);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Adds transition hysteresis in [0,0.49] to reduce LOD popping.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_hysteresis	float _hysteresis	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 731.

HRL SetMeshLODOVERRIDE

```
void HRL_SetMeshLODOVERRIDE(HRL_id _meshid, int _level);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Forces a fixed LOD level. Pass -1 to return to automatic selection.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_level	int _level	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 733.

HRL ForceMeshLODRebuild

```
void HRL_ForceMeshLODRebuild(HRL_id _meshid);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Regenerates the internal LOD geometry from LOD 0.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 735.

HRL GetMeshLODCount

```
HRL_uint HRL_GetMeshLODCount(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested HRL_uint value.

Header declaration location: line 736.

HRL GetMeshLODVertexCount

```
size_t HRL_GetMeshLODVertexCount(HRL_id _meshid, HRL_uint _level);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_level	HRL_uint _level	Input value controlling the named operation.

Return value

Returns the requested size_t value.

Header declaration location: line 737.

HRL GetMeshLODTriangleCount

```
size_t HRL_GetMeshLODTriangleCount(HRL_id _meshid, HRL_uint _level);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_level	HRL_uint _level	Input value controlling the named operation.

Return value

Returns the requested size_t value.

Header declaration location: line 738.

HRL GetMeshLODLevel

```
int HRL_GetMeshLODLevel(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Returns the LOD chosen by the most recently rendered viewport.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 740.

Skeletal Meshes

HRL GetSkeletalBoneCount

```
HRL_uint HRL_GetSkeletalBoneCount(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

```
=====
=====
```

SKELETAL MESHES

```
=====
=====
```

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested HRL_uint value.

Header declaration location: line 746.

HRL GetSkeletalBone

```
const HRL_SkeletalBone* HRL_GetSkeletalBone(HRL_id _meshid, HRL_uint _index);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_index	HRL_uint _index	Count/index value controlling how many elements are processed or which element is addressed.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Pointer lifetime: the returned pointer is a read-only view into HRL-owned/import-owned data and remains usable only while the owning resource/mesh remains alive.

Header declaration location: line 747.

HRL FindSkeletalBone

```
HRL_uint HRL_FindSkeletalBone(HRL_id _meshid, const char* _name);
```

Searches the associated collection by name and returns the matching resource/index, or HRL_INVALID_ID when no matching entry exists.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_name	const char* _name	Null-terminated resource or symbol name used for lookup/association.

Return value

Returns HRL_uint. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 748.

HRL GetSkeletalAnimationCount

```
HRL_uint HRL_GetSkeletalAnimationCount(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested HRL_uint value.

Header declaration location: line 750.

HRL GetSkeletalAnimation

```
const HRL_SkeletalAnimation* HRL_GetSkeletalAnimation(HRL_id _meshid, HRL_uint _index);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_index	HRL_uint _index	Count/index value controlling how many elements are processed or which element is addressed.

Return value

Returns a pointer to HRL-managed or newly allocated data as described by the function. The corresponding Free/Release function controls its lifetime.

Pointer lifetime: the returned pointer is a read-only view into HRL-owned/import-owned data and remains usable only while the owning resource/mesh remains alive.

Header declaration location: line 751.

HRL FindSkeletalAnimation

```
HRL_uint HRL_FindSkeletalAnimation(HRL_id _meshid, const char* _name);
```

Searches the associated collection by name and returns the matching resource/index, or HRL_INVALID_ID when no matching entry exists.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_name	const char* _name	Null-terminated resource or symbol name used for lookup/association.

Return value

Returns HRL_uint. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 752.

HRL PlaySkeletalAnimation

```
void HRL_PlaySkeletalAnimation(HRL_id _meshid, HRL_uint _animation);
```

Begins playback/simulation for the referenced resource.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_animation	HRL_uint _animation	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 754.

HRL StopSkeletalAnimation

```
void HRL_StopSkeletalAnimation(HRL_id _meshid);
```

Stops playback/simulation for the referenced resource.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 755.

HRL SetSkeletalAnimationTime

```
void HRL_SetSkeletalAnimationTime(HRL_id _meshid, float _time);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_time	float _time	Time value in seconds within the relevant timeline unless the function name indicates another unit.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 756.

HRL SetSkeletalAnimationSpeed

```
void HRL_SetSkeletalAnimationSpeed(HRL_id _meshid, float _speed);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_speed	float _speed	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 757.

HRL SetSkeletalAnimationLoop

```
void HRL_SetSkeletalAnimationLoop(HRL_id _meshid, int _loop);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_loop	int _loop	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 758.

HRL GetCurrentSkeletalAnimation

```
int HRL_GetCurrentSkeletalAnimation(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 759.

HRL GetSkeletalAnimationTime

```
float HRL_GetSkeletalAnimationTime(HRL_id _meshid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested float value.

Header declaration location: line 760.

HRL IsSkeletalAnimationPlaying

```
int HRL_IsSkeletalAnimationPlaying(HRL_id _meshid);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 761.

HRL UpdateSkeletalAnimations

```
void HRL_UpdateSkeletalAnimations(float _deltaSeconds);
```

Advances every currently active skeletal animation by the supplied simulation delta. Call once per frame when driving animations from the global update loop.

Advances all playing skeletal mesh animations by _deltaSeconds.

Parameters

Name	Declaration	Description
_deltaSeconds	float _deltaSeconds	Elapsed simulation time for the update, in seconds.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 764.

HRL SetSpriteDrawOrder

```
void HRL_SetSpriteDrawOrder(HRL_id _meshid, float _draworder);
```

Controls the rendering order of a sprite on the Z axis.

Updates the named property of the supplied HRL resource or rendering subsystem.

Only relevant when two or more sprites share the same Z depth.

Higher values are drawn on top.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_draworder	float _draworder	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 771.

Lights

HRL CreateLight

```
HRL_id HRL_CreateLight(HRL_id _sceneid, HRL_ELightType _type);
```

Creates a light source in the given scene.

Creates a light of the requested ELightType in a scene.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_type	HRL_ELightType _type	One of HRL_PointLight, HRL_DirectionalLight, HRL_SpotLight.

Return value

HRL_id of the new light, or HRL_INVALID_ID on failure.

Header declaration location: line 783.

HRL DeleteLight

```
void HRL_DeleteLight(HRL_id _lightid);
```

Destroys a light and removes it from its scene.

Destroys the referenced eLight object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_lightid	HRL_id _lightid	ID of the light to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 789.

HRL IsValidLight

```
int HRL_IsValidLight(HRL_id _id);
```

Returns whether the given ID refers to a live light object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 796.

HRL SetLightColor

```
void HRL_SetLightColor(HRL_id _lightid, float x, float y, float z);
```

Sets the RGB color emitted by a light.

Updates the named property of the supplied HRL resource or rendering subsystem.

Values are typically in [0..1] but may exceed 1 for HDR workflows.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 802.

HRL SetLightIntensity

```
void HRL_SetLightIntensity(HRL_id_lightid, float i);
```

Sets the intensity (brightness multiplier) of a light.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
i	float i	Intensity value. 1.0 is the default, higher values produce brighter results.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 808.

HRL SetLightAttenuation

```
void HRL_SetLightAttenuation(HRL_id_lightid, float a);
```

Sets the attenuation (falloff) factor of a light.

Updates the named property of the supplied HRL resource or rendering subsystem.

Controls how quickly the light fades with distance.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
a	float a	Attenuation coefficient.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 815.

HRL SetLightLocation

```
void HRL_SetLightLocation(HRL_id_lightid, float x, float y, float z);
```

Sets the world-space position of a light.

Updates the named property of the supplied HRL resource or rendering subsystem.

Relevant for point lights and spot lights.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 821.

HRL SetLightRotation

```
void HRL_SetLightRotation(HRL_id_lightid, float pitch, float yaw, float roll);
```

Sets the orientation of a light using Euler angles (in degrees).

Updates the named property of the supplied HRL resource or rendering subsystem.

Primarily relevant for directional and spot lights.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 827.

HRL SetLightCastShadows

```
void HRL_SetLightCastShadows(HRL_id_lightid, int _enable);
```

Enables or disables shadow casting for a light.

Updates the named property of the supplied HRL resource or rendering subsystem.

OpenGL 3.3 supports shadows for point, directional and spot lights.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_enable	int _enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 833.

HRL SetLightShadowBias

```
void HRL_SetLightShadowBias(HRL_id_lightid, float _bias);
```

Sets the shadow bias used to reduce self-shadowing artifacts.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_bias	float_bias	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 838.

HRL SetLightShadowStrength

```
void HRL_SetLightShadowStrength(HRL_id_lightid, float_strength);
```

Controls how dark a cast shadow is for this light.

Updates the named property of the supplied HRL resource or rendering subsystem.

1 applies the full shadow term.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_strength	float_strength	Value in [0..1]. 0 keeps the surface fully lit,

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 845.

HRL SetLightShadowResolution

```
void HRL_SetLightShadowResolution(HRL_id_lightid, int_resolution);
```

Sets the resolution of the private shadow map for a light.

Updates the named property of the supplied HRL resource or rendering subsystem.

Supported values are positive powers of two; the OpenGL backend may clamp the requested value to the implementation limits.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_resolution	int_resolution	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 852.

HRL SetSpotLightInnerCutoff

```
void HRL_SetSpotLightInnerCutoff(HRL_id_lightid, float_inner_cutoff);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_lightid	HRL_id_lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
inner_cutoff	float_inner_cutoff	Degrees

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 859.

HRL SetSpotLightOuterCutoff

```
void HRL_SetSpotLightOuterCutoff(HRL_id _lightid, float outer_cutoff);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_lightid	HRL_id _lightid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
outer_cutoff	float outer_cutoff	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 861.

Textures

HRL CreateTexture

```
HRL_id HRL_CreateTexture(const char* _data, size_t _bufferSize);
```

Creates a GPU texture from a raw file buffer.

Decodes texture data from an in-memory image buffer and creates a GPU texture resource. The supported built-in formats are the image formats identified by the header comments.

Supported formats: png, jpeg, jpg, bmp, tga, gif (first frame), hdr, psd (partial).

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the file contents (opened in binary mode).
_bufferSize	size_t _bufferSize	Size of the buffer in bytes.

Return value

HRL_id of the new texture, or HRL_INVALID_ID on failure.

Header declaration location: line 875.

HRL CreateTextureAsync

```
HRL_id HRL_CreateTextureAsync(const char* _data, size_t _bufferSize);
```

Starts asynchronous texture decoding/upload work and returns the texture handle before the resource is necessarily ready for immediate use.

Starts an asynchronous texture load/decoding job. GPU upload is completed automatically on the HRL context thread.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_bufferSize	size_t _bufferSize	Size of the associated data buffer in bytes.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Asynchronous-use note: creation returns a handle before work necessarily completes. Pair the function with its Is*Ready query when polling is appropriate, or the matching WaitFor* function when deterministic readiness is required.

Header declaration location: line 878.

HRL IsTextureReady

```
int HRL_IsTextureReady(HRL_id _textureid);
```

Reports whether an asynchronously created texture has completed its loading/upload work and can be treated as ready by dependent rendering operations.

Returns HRL_TRUE when an asynchronous texture is fully uploaded and usable.

Parameters

Name	Declaration	Description
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 882.

HRL WaitForTexture

```
void HRL_WaitForTexture(HRL_id _textureid);
```

Blocks the calling thread until the specified asynchronous texture has finished loading/uploading.

Blocks until an asynchronous texture is fully uploaded.

Parameters

Name	Declaration	Description
_textureid	HRL_id_textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 885.

HRL DeleteTexture

```
void HRL_DeleteTexture(HRL_id_textureid);
```

Destroys a texture and frees its GPU memory.

Destroys the referenced eTexture object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_textureid	HRL_id_textureid	ID of the texture to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 891.

HRL IsValidTexture

```
int HRL_IsValidTexture(HRL_id_id);
```

Returns whether the given ID refers to a live texture object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id_id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 898.

HRL ReloadTexture

```
void HRL_ReloadTexture(HRL_id_textureid, const char* _data, size_t _bufferSize);
```

Replaces the pixel data of an existing texture from a new file buffer.

Performs the operation represented by the function name on the supplied HRL objects and parameters.

The texture ID remains valid; any material referencing it will use the updated image.

Parameters

Name	Declaration	Description
_textureid	HRL_id_textureid	ID of the texture to update.
_data	const char* _data	Pointer to the new file contents (opened in binary mode).
_bufferSize	size_t _bufferSize	Size of the buffer in bytes.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 907.

HRL_GetTextureSize

```
void HRL_GetTextureSize(HRL_id _textureid, int* _width, int* _height);
```

Retrieves the current dimensions of a texture in pixels.

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_width	int* _width	Output width.
_height	int* _height	Output height.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 914.

HRL SetTextureMinFilter

```
void HRL_SetTextureMinFilter(HRL_id _textureid, HRL_EFilterType _filter);
```

Sets the minification filter used when the texture appears smaller than its native size.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_filter	HRL_EFilterType _filter	One of HRL_Filter_Nearest, HRL_Filter_Linear, HRL_Filter_Trilinear, etc.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 920.

HRL SetTextureMagFilter

```
void HRL_SetTextureMagFilter(HRL_id _textureid, HRL_EFilterType _filter);
```

Sets the magnification filter used when the texture appears larger than its native size.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_filter	HRL_EFilterType _filter	One of HRL_Filter_Nearest, HRL_Filter_Linear, etc.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 926.

HRL CreateTextureFromText

```
HRL_id HRL_CreateTextureFromText(const char* _text, HRL_id _fontid, float _font_size, float _wrap_width, float r, float g, float b, float bg_r, float bg_g, float bg_b, float bg_a );
```

Rasterizes a UTF-8 text string into a new texture using the given font.

Rasterizes a text string using an HRL font and creates a GPU texture containing the resulting glyphs.

Parameters

Name	Declaration	Description
_text	const char* _text	Null-terminated UTF-8 string to render.
_fontid	HRL_id _fontid	ID of a font created with HRL_CreateFont.
_font_size	float _font_size	Glyph height in pixels.

Name	Declaration	Description
_wrap_width	float _wrap_width	Line wrap threshold in pixels. Pass 0 to disable wrapping.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
bg_r	float bg_r	Input value controlling the named operation.
bg_g	float bg_g	Input value controlling the named operation.
bg_b	float bg_b	Input value controlling the named operation.
bg_a	float bg_a	Input value controlling the named operation.

Return value

HRL_id of the newly created texture, or HRL_INVALID_ID on failure.

Header declaration location: line 938.

Scenes & Rendering

HRL ClearScreen

```
void HRL_ClearScreen();
```

Clears the entire screen to its default clear color.

Clears the active framebuffer using HRL's configured/default clear color before scene or UI rendering.

Useful when no scene covers the full framebuffer.

Parameters

This function takes no parameters.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 953.

HRL CreateScene

```
HRL_id HRL_CreateScene(int _renderOnScreen);
```

Creates a new scene that acts as a container for meshes, lights and cameras.

Creates a render scene. When renderOnScreen is true, the scene targets the screen framebuffer; otherwise it maintains an off-screen render texture.

If HRL_False, it renders into an off-screen texture buffer (default 480x480).

Parameters

Name	Declaration	Description
_renderOnScreen	int _renderOnScreen	If HRL_True, the scene renders directly to the screen.

Return value

HRL_id of the new scene, or HRL_INVALID_ID on failure.

Header declaration location: line 961.

HRL DeleteScene

```
void HRL_DeleteScene(HRL_id _sceneid);
```

Destroys a scene and all objects it owns.

Destroys the referenced eScene object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	ID of the scene to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 967.

HRL IsValidScene

```
int HRL_IsValidScene(HRL_id _id);
```

Returns whether the given ID refers to a live scene object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 974.

HRL_SetGlobalIlluminationEnabled

```
void HRL_SetGlobalIlluminationEnabled(HRL_id _sceneid, int _enable);
```

Enables or disables the scene-level global illumination pass.

Enable/disable global illumination. Disabled by default.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_enable	int _enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 977.

HRL_SetGlobalIlluminationMethod

```
void HRL_SetGlobalIlluminationMethod(HRL_id _sceneid, HRL_EGlobalIlluminationMethod _method);
```

Selects the global illumination method used by the scene. The public enumeration deliberately exposes future techniques in addition to those implemented by the current backend.

Select the GI method. HRL_GI_NONE disables GI for the scene.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_method	HRL_EGlobalIlluminationMethod _method	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 979.

HRL_IsGlobalIlluminationEnabled

```
int HRL_IsGlobalIlluminationEnabled(HRL_id _sceneid);
```

Reports whether scene-level global illumination is currently enabled.

Returns the scene GI enabled state.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 981.

HRL_GetGlobalIlluminationMethod

```
HRL_EGlobalIlluminationMethod HRL_GetGlobalIlluminationMethod(HRL_id _sceneid);
```

Returns the scene's currently selected global illumination method.

Returns the scene's selected GI method.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.

Return value

Returns HRL_EGlobalIlluminationMethod representing the requested value/state.

Header declaration location: line 983.

HRL_IsGlobalIlluminationMethodSupported

```
int HRL_IsGlobalIlluminationMethodSupported(HRL_EGlobalIlluminationMethod _method);
```

Checks whether the current backend/runtime implements the requested global illumination method.

Returns whether the active backend implements this GI method.

Parameters

Name	Declaration	Description
_method	HRL_EGlobalIlluminationMethod_method	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 985.

HRL_GetGlobalIlluminationSupportedMethods

```
uint32_t HRL_GetGlobalIlluminationSupportedMethods();
```

Returns a bitmask describing the global illumination methods supported by the current runtime/backend.

Bitmask of methods supported by the active backend: (1u << method).

Parameters

This function takes no parameters.

Return value

Returns the requested uint32_t value.

Header declaration location: line 987.

HRL_ResizeSceneTexture

```
void HRL_ResizeSceneTexture(HRL_id_sceneid, int _width, int _height);
```

Resizes the off-screen render texture of a scene.

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Has no effect if the scene was created with _renderOnScreen = HRL_True.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_width	int_width	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.
_height	int_height	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 993.

HRL_SetSkySphereEnabled

```
void HRL_SetSkySphereEnabled(HRL_id_sceneid, int _enable);
```

Enables or disables the procedural sky sphere for a scene.

Updates the named property of the supplied HRL resource or rendering subsystem.

The sky follows the camera translation, so it behaves as an infinitely

distant environment while remaining a true sphere in the OpenGL backend.

Disabled by default for backward compatibility.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_enable	int_enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1001.

HRL SetSkySphereColors

```
void HRL_SetSkySphereColors( HRL_id_sceneid, float top_r, float top_g, float top_b, float horizon_r, float horizon_g, float horizon_b, float bottom_r, float bottom_g, float bottom_b );
```

Sets the top, horizon and bottom colors of the procedural sky.

Updates the named property of the supplied HRL resource or rendering subsystem.

Values are linear RGB and may exceed 1 for HDR rendering.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
top_r	float top_r	Input value controlling the named operation.
top_g	float top_g	Input value controlling the named operation.
top_b	float top_b	Input value controlling the named operation.
horizon_r	float horizon_r	Input value controlling the named operation.
horizon_g	float horizon_g	Input value controlling the named operation.
horizon_b	float horizon_b	Input value controlling the named operation.
bottom_r	float bottom_r	Input value controlling the named operation.
bottom_g	float bottom_g	Input value controlling the named operation.
bottom_b	float bottom_b	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1007.

HRL SetSkySphereRotation

```
void HRL_SetSkySphereRotation(HRL_id_sceneid, float pitch, float yaw, float roll);
```

Rotates the sky sphere using Euler angles in degrees.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1017.

HRL SetSkySphereTexture

```
void HRL_SetSkySphereTexture(HRL_id _sceneid, HRL_id _textureid);
```

Uses an equirectangular 2D texture as the sky-sphere image.

Updates the named property of the supplied HRL resource or rendering subsystem.

Pass HRL_INVALID_ID to return to the procedural color sky.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1023.

HRL SetEnvironmentMappingEnabled

```
void HRL_SetEnvironmentMappingEnabled(HRL_id _sceneid, int _enable);
```

Enables or disables equirectangular environment mapping for 3D materials.

Updates the named property of the supplied HRL resource or rendering subsystem.

The environment texture is independent from the sky texture.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_enable	int _enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1029.

HRL SetEnvironmentMap

```
void HRL_SetEnvironmentMap(HRL_id _sceneid, HRL_id _textureid);
```

Sets an equirectangular texture used for environment reflections.

Updates the named property of the supplied HRL resource or rendering subsystem.

Pass HRL_INVALID_ID to remove the environment map.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1035.

HRL EnableColorPickingBuffer

```
void HRL_EnableColorPickingBuffer(HRL_id _scene, int _enable);
```

Enables or disables the GPU color-picking buffer for a scene.

Performs the operation represented by the function name on the supplied HRL objects and parameters.

When enabled, each rendered object is assigned a unique color ID, allowing CPU-side object picking by reading pixel values.

Parameters

Name	Declaration	Description
_scene	HRL_id _scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_enable	int _enable	HRL_True to enable, HRL_False to disable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1043.

HRL_GetHoveredObject

```
HRL_id HRL_GetHoveredObject(HRL_id _scene, int mouseX, int mouseY, HRL_EMeshType* mesh_type);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_scene	HRL_id _scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
mouseX	int mouseX	Relative mouse cursor position to the window
mouseY	int mouseY	Input value controlling the named operation.
mesh_type	HRL_EMeshType* mesh_type	Reference to the type of hovered mesh, can be nullptr

Return value

HRL id of the hovered mesh, if none, it will return HRL_INVALID_ID

Header declaration location: line 1053.

Post Processing

HRL CreatePostProcess

```
HRL_id HRL_CreatePostProcess(HRL_id _viewport, HRL_id _matid, int priority);
```

Attaches a post-process pass to a viewport using a custom material.

Creates a post-processing stage associated with a viewport and material. Priority controls ordering among multiple post-process entries.

Passes are applied in creation order after the scene is rendered.

Parameters

Name	Declaration	Description
_viewport	HRL_id _viewport	Input value controlling the named operation.
_matid	HRL_id _matid	Material containing the full-screen shader to apply.
priority	int priority	Input value controlling the named operation.

Return value

HRL_id of the new post-process object, or HRL_INVALID_ID on failure.

Header declaration location: line 1066.

HRL DeletePostProcess

```
void HRL_DeletePostProcess(HRL_id _postid);
```

Removes and destroys a post-process pass.

Destroys the referenced ePostProcess object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_postid	HRL_id _postid	ID of the post-process to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1072.

HRL IsValidPostProcess

```
int HRL_IsValidPostProcess(HRL_id _id);
```

Returns whether the given ID refers to a live post-process object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1079.

Shaders

HRL CreateShader

```
HRL_id HRL_CreateShader(const char* _vertData, size_t _vertSize, const char* _fragData, size_t _fragSize);
```

Compiles and links a shader program from GLSL vertex and fragment source.

Compiles/creates a shader program from caller-provided vertex and fragment shader source/data buffers.

Parameters

Name	Declaration	Description
_vertData	const char* _vertData	Pointer to the vertex shader source buffer.
_vertSize	size_t _vertSize	Size of the vertex shader source in bytes.
_fragData	const char* _fragData	Pointer to the fragment shader source buffer.
_fragSize	size_t _fragSize	Size of the fragment shader source in bytes.

Return value

HRL_id of the compiled shader, or HRL_INVALID_ID on compilation failure.

Header declaration location: line 1094.

HRL CreateShaderAsync

```
HRL_id HRL_CreateShaderAsync(const char* _vertData, size_t _vertSize, const char* _fragData, size_t _fragSize);
```

Starts asynchronous shader creation and returns the resource handle before compilation is guaranteed to be finished.

Starts asynchronous shader source loading. Compilation/linking is performed automatically on the HRL context thread.

Parameters

Name	Declaration	Description
_vertData	const char* _vertData	Pointer to the caller-provided binary/text data consumed by the operation.
_vertSize	size_t _vertSize	Size of the associated data buffer in bytes.
_fragData	const char* _fragData	Pointer to the caller-provided binary/text data consumed by the operation.
_fragSize	size_t _fragSize	Size of the associated data buffer in bytes.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Asynchronous-use note: creation returns a handle before work necessarily completes. Pair the function with its Is*Ready query when polling is appropriate, or the matching WaitFor* function when deterministic readiness is required.

Header declaration location: line 1097.

HRL IsShaderReady

```
int HRL_IsShaderReady(HRL_id _shaderid);
```

Reports whether an asynchronously created shader has completed compilation and setup.

Returns HRL_TRUE when an asynchronous shader is compiled and linked.

Parameters

Name	Declaration	Description
_shaderid	HRL_id _shaderid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 1101.

HRL WaitForShader

```
void HRL_WaitForShader(HRL_id _shaderid);
```

Blocks until the specified asynchronous shader creation operation completes.

Blocks until an asynchronous shader is compiled and linked.

Parameters

Name	Declaration	Description
_shaderid	HRL_id_shaderid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1104.

HRL WaitForAllAsyncResources

```
void HRL_WaitForAllAsyncResources();
```

Blocks until all currently outstanding asynchronous shader/texture resource jobs have completed.

Waits for all queued asynchronous resources and uploads their completed GPU objects.

Parameters

This function takes no parameters.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Asynchronous-use note: creation returns a handle before work necessarily completes. Pair the function with its Is*Ready query when polling is appropriate, or the matching WaitFor* function when deterministic readiness is required.

Header declaration location: line 1107.

HRL DeleteShader

```
void HRL_DeleteShader(HRL_id_shaderid);
```

Destroys a shader program and frees its GPU resources.

Destroys the referenced eShader object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_shaderid	HRL_id_shaderid	ID of the shader to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1113.

HRL IsValidShader

```
int HRL_IsValidShader(HRL_id_id);
```

Returns whether the given ID refers to a live shader object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id_id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1120.

Materials

HRL CreateMaterial

```
HRL_id HRL_CreateMaterial(HRL_id _shaderid);
```

Creates a material instance backed by the given shader.

Creates a material instance associated with the supplied shader.

A material stores the uniform values (textures, floats, etc.) passed to its shader.

Parameters

Name	Declaration	Description
_shaderid	HRL_id _shaderid	ID of the shader this material uses.

Return value

HRL_id of the new material, or HRL_INVALID_ID on failure.

Header declaration location: line 1133.

HRL DeleteMaterial

```
void HRL_DeleteMaterial(HRL_id _matid);
```

Destroys a material and its stored uniform data.

Destroys the referenced eMaterial object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	ID of the material to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1139.

HRL IsValidMaterial

```
int HRL_IsValidMaterial(HRL_id _id);
```

Returns whether the given ID refers to a live material object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1146.

HRL MaterialSetInt

```
void HRL_MaterialSetInt(HRL_id _matid, const char* _uniformName, int a);
```

Sets an integer uniform on a material.

Assigns an integer value to a material parameter/uniform by name.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Name	Declaration	Description
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.
a	int a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1151.

HRL MaterialSetTexture

```
void HRL_MaterialSetTexture(HRL_id _matid, const char* _uniformName, HRL_id _textureid);
```

Binds a texture to a named sampler uniform on a material.

Binds a texture handle to a named material texture parameter/uniform.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1156.

HRL MaterialSetBool

```
void HRL_MaterialSetBool(HRL_id _matid, const char* _uniformName, int a);
```

Sets a boolean uniform on a material (internally stored as int 0 or 1).

Assigns a boolean-style value to a named material parameter, using an integer 0/1 representation for the C ABI.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.
a	int a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1181.

HRL MaterialSetFloat

```
void HRL_MaterialSetFloat(HRL_id _matid, const char* _uniformName, float a);
```

Sets a float uniform on a material.

Assigns a floating-point value to a named material parameter/uniform.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.

Name	Declaration	Description
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1186.

HRL MaterialSetVec2

```
void HRL_MaterialSetVec2(HRL_id _matid, const char* _uniformName, float x, float y);
```

Sets a vec2 uniform on a material.

Assigns a 2-component floating-point vector to a named material parameter/uniform.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1191.

HRL MaterialSetVec3

```
void HRL_MaterialSetVec3(HRL_id _matid, const char* _uniformName, float x, float y, float z);
```

Sets a vec3 uniform on a material.

Assigns a 3-component floating-point vector to a named material parameter/uniform.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1196.

HRL MaterialSetVec4

```
void HRL_MaterialSetVec4(HRL_id _matid, const char* _uniformName, float x, float y, float z, float w);
```

Sets a vec4 uniform on a material.

Assigns a 4-component floating-point vector to a named material parameter/uniform.

Parameters

Name	Declaration	Description
_matid	HRL_id _matid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_uniformName	const char* _uniformName	Null-terminated resource or symbol name used for lookup/association.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
w	float w	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1201.

HRL MaterialSetEmissiveColor

```
void HRL_MaterialSetEmissiveColor(HRL_id matid, float r, float g, float b, float a);
```

Sets the emissive color tint of a material, additively blended with the emissive texture.

Sets the material emissive color using four floating-point color components.

Parameters

Name	Declaration	Description
matid	HRL_id matid	ID of the target material.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1208.

Viewports

HRL CreateViewport

```
HRL_id HRL_CreateViewport(HRL_id _sceneid, HRL_id _cameraid, float x, float y, float _width, float _height);
```

Creates a viewport that renders a scene through a camera into a screen region.

Creates a rectangular rendering viewport for a scene and camera. Its rectangle defines the portion of the target framebuffer where the viewport renders.

Useful for split-screen or picture-in-picture setups.

All coordinates are normalized [0..1]: (0,0) is top-left, (1,1) is bottom-right.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_cameraid	HRL_id _cameraid	Camera to use, or HRL_INVALID_ID to leave unassigned.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
_width	float _width	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.
_height	float _height	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.

Return value

HRL_id of the new viewport, or HRL_INVALID_ID on failure.

Header declaration location: line 1222.

HRL DeleteViewport

```
void HRL_DeleteViewport(HRL_id _viewportid);
```

Destroys a viewport.

Destroys the referenced eViewport object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_viewportid	HRL_id _viewportid	ID of the viewport to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1228.

HRL IsValidViewport

```
int HRL_IsValidViewport(HRL_id _id);
```

Returns whether the given ID refers to a live viewport object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1235.

HRL SetViewportCamera

```
void HRL_SetViewportCamera(HRL_id _viewportid, HRL_id _camid);
```

Reassigns the camera used by a viewport.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_viewportid	HRL_id _viewportid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_camid	HRL_id _camid	New camera ID, or HRL_INVALID_ID to detach.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1241.

HRL SetViewportRect

```
void HRL_SetViewportRect(HRL_id _viewportid, float x, float y, float _width, float _height);
```

Updates the screen-space rectangle of a viewport.

Updates the named property of the supplied HRL resource or rendering subsystem.

All values are normalized [0..1].

Parameters

Name	Declaration	Description
_viewportid	HRL_id _viewportid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
_width	float _width	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.
_height	float _height	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1247.

Camera

HRL CreateCamera

```
HRL_id HRL_CreateCamera(HRL_id _sceneid, HRL_ECameraType _type);
```

Creates a camera in the given scene.

Creates a camera attached to a scene with the requested orthographic or perspective type.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_type	HRL_ECameraType _type	One of HRL_Ortho or HRL_Perspective. Defaults to HRL_Ortho.

Return value

HRL_id of the new camera, or HRL_INVALID_ID on failure.

Header declaration location: line 1259.

HRL DeleteCamera

```
void HRL_DeleteCamera(HRL_id _camid);
```

Destroys a camera.

Destroys the referenced eCamera object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_camid	HRL_id _camid	ID of the camera to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1265.

HRL IsValidCamera

```
int HRL_IsValidCamera(HRL_id _id);
```

Returns whether the given ID refers to a live camera object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1272.

HRL SetCameraType

```
void HRL_SetCameraType(HRL_id _camid, HRL_ECameraType _type);
```

Changes the projection type of an existing camera at runtime.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_camid	HRL_id _camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Name	Declaration	Description
_type	HRL_ECameraType_type	HRL_Ortho or HRL_Perspective.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1278.

HRL_SetCameraOrthoVertical

```
void HRL_SetCameraOrthoVertical(HRL_id _camid, float _height);
```

Sets the vertical extent of an orthographic camera's view volume.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_camid	HRL_id_camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_height	float_height	World-space height visible on screen.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1284.

HRL_SetCameraPerspectiveFov

```
void HRL_SetCameraPerspectiveFov(HRL_id _camid, float _fov);
```

Sets the vertical field of view for a perspective camera.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_camid	HRL_id_camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_fov	float_fov	Vertical FOV in degrees.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1290.

HRL_SetCameraNearPlane

```
void HRL_SetCameraNearPlane(HRL_id _camid, float _nearPlane);
```

Sets the near clipping plane distance.

Updates the named property of the supplied HRL resource or rendering subsystem.

Objects closer than this value will not be rendered.

Parameters

Name	Declaration	Description
_camid	HRL_id_camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_nearPlane	float_nearPlane	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1296.

HRL_SetCameraFarPlane

```
void HRL_SetCameraFarPlane(HRL_id _camid, float _farPlane);
```

Sets the far clipping plane distance.

Updates the named property of the supplied HRL resource or rendering subsystem.

Objects farther than this value will not be rendered.

Parameters

Name	Declaration	Description
_camid	HRL_id _camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_farPlane	float _farPlane	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1302.

HRL SetCameraLocation

```
void HRL_SetCameraLocation(HRL_id _camid, float x, float y, float z);
```

Sets the world-space position of a camera.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_camid	HRL_id _camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1307.

HRL SetCameraRotation

```
void HRL_SetCameraRotation(HRL_id _camid, float pitch, float yaw, float roll);
```

Sets the orientation of a camera using Euler angles (in degrees).

Updates the named property of the supplied HRL resource or rendering subsystem.

Axis mapping: Pitch = X, Yaw = Y, Roll = Z.

Parameters

Name	Declaration	Description
_camid	HRL_id _camid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1313.

Gizmos

Gizmos are viewport-bound interaction helpers. Their API intentionally separates transform state, displayed operations/axes, visual state, hit-test state, and change notifications.

HRL CreateGizmo

```
HRL_id HRL_CreateGizmo(HRL_id _viewportid);
```

Creates an editor gizmo attached to a viewport, independently of HRL scene objects.

Creates an interactive transform gizmo attached to a viewport. The gizmo exposes translation, rotation, and scale editing plus hit/operation queries.

Its transform is expressed in the same world coordinate system as HRL meshes/lights/cameras.

Parameters

Name	Declaration	Description
_viewportid	HRL_id _viewportid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 1325.

HRL DeleteGizmo

```
void HRL_DeleteGizmo(HRL_id _gizmoid);
```

Destroys the referenced eGizmo object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1326.

HRL IsValidGizmo

```
int HRL_IsValidGizmo(HRL_id _gizmoid);
```

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1327.

HRL SetGizmoPosition

```
void HRL_SetGizmoPosition(HRL_id _gizmoid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1329.

HRL GetGizmoPosition

```
void HRL_GetGizmoPosition(HRL_id _gizmoid, float* x, float* y, float* z);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float* x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float* y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float* z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1330.

HRL SetGizmoRotation

```
void HRL_SetGizmoRotation(HRL_id _gizmoid, float pitch, float yaw, float roll);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1331.

HRL GetGizmoRotation

```
void HRL_GetGizmoRotation(HRL_id _gizmoid, float* pitch, float* yaw, float* roll);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float* pitch	Caller-provided pointer used as input or output according to the function signature.
yaw	float* yaw	Caller-provided pointer used as input or output according to the function signature.
roll	float* roll	Caller-provided pointer used as input or output according to the function signature.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1332.

HRL SetGizmoScale

```
void HRL_SetGizmoScale(HRL_id _gizmoid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1333.

HRL GetGizmoScale

```
void HRL_GetGizmoScale(HRL_id _gizmoid, float* x, float* y, float* z);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float* x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float* y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float* z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1334.

HRL SetGizmoMode

```
void HRL_SetGizmoMode(HRL_id _gizmoid, HRL_EGizmoMode _mode);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_mode	HRL_EGizmoMode _mode	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1336.

HRL GetGizmoMode

```
HRL_EGizmoMode HRL_GetGizmoMode(HRL_id _gizmoid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_EGizmoMode representing the requested value/state.

Header declaration location: line 1337.

HRL SetGizmoSpace

```
void HRL_SetGizmoSpace(HRL_id _gizmoid, HRL_EGizmoSpace _space);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_space	HRL_EGizmoSpace _space	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1338.

HRL GetGizmoSpace

```
HRL_EGizmoSpace HRL_GetGizmoSpace(HRL_id _gizmoid);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_EGizmoSpace representing the requested value/state.

Header declaration location: line 1339.

HRL SetGizmoTranslateVisible

```
void HRL_SetGizmoTranslateVisible(HRL_id _gizmoid, int _visible);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id_gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_visible	int_visible	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1341.

HRL SetGizmoRotateVisible

```
void HRL_SetGizmoRotateVisible(HRL_id_gizmoid, int_visible);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id_gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_visible	int_visible	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1342.

HRL SetGizmoScaleVisible

```
void HRL_SetGizmoScaleVisible(HRL_id_gizmoid, int_visible);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id_gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_visible	int_visible	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1343.

HRL SetGizmoTranslateAxes

```
void HRL_SetGizmoTranslateAxes(HRL_id_gizmoid, int_axis_mask);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id_gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_axis_mask	int_axis_mask	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1344.

HRL SetGizmoRotateAxes

```
void HRL_SetGizmoRotateAxes(HRL_id_gizmoid, int_axis_mask);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_axis_mask	int _axis_mask	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1345.

HRL SetGizmoScaleAxes

```
void HRL_SetGizmoScaleAxes(HRL_id _gizmoid, int _axis_mask);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_axis_mask	int _axis_mask	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1346.

HRL SetGizmoSize

```
void HRL_SetGizmoSize(HRL_id _gizmoid, float _world_size);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_world_size	float _world_size	Size of the associated data buffer in bytes.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1347.

HRL SetGizmoScreenSize

```
void HRL_SetGizmoScreenSize(HRL_id _gizmoid, float _pixels);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_pixels	float _pixels	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1348.

HRL SetGizmoUseScreenSize

```
void HRL_SetGizmoUseScreenSize(HRL_id _gizmoid, int _use_screen_size);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_use_screen_size	int _use_screen_size	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1349.

HRL SetGizmoVisible

```
void HRL_SetGizmoVisible(HRL_id _gizmoid, int _visible);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_visible	int _visible	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1350.

HRL SetGizmoEnabled

```
void HRL_SetGizmoEnabled(HRL_id _gizmoid, int _enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_enabled	int _enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1351.

HRL SetGizmoAxisColor

```
void HRL_SetGizmoAxisColor(HRL_id _gizmoid, int _axis, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_axis	int _axis	Enumeration value selecting the requested operating mode, type, space, method, or category.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Name	Declaration	Description
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1353.

HRL SetGizmoCenterColor

```
void HRL_SetGizmoCenterColor(HRL_id _gizmoid, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1354.

HRL SetGizmoHoverColor

```
void HRL_SetGizmoHoverColor(HRL_id _gizmoid, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1355.

HRL GetGizmoHoveredPart

```
HRL_EGizmoPart HRL_GetGizmoHoveredPart(HRL_id _gizmoid);
```

Returns the transform-gizmo sub-part currently under the pointer, or HRL_GIZMO_PART_NONE when nothing is hovered.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_EGizmoPart representing the requested value/state.

Header declaration location: line 1356.

HRL GetGizmoActivePart

```
HRL_EGizmoPart HRL_GetGizmoActivePart(HRL_id _gizmoid);
```

Returns the transform-gizmo sub-part currently engaged by an active drag/interaction.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_EGizmoPart representing the requested value/state.

Header declaration location: line 1357.

HRL GetGizmoHoveredOperation

```
HRL_EGizmoOperation HRL_GetGizmoHoveredOperation(HRL_id _gizmoid);
```

Returns the transform operation corresponding to the hovered gizmo part.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_EGizmoOperation representing the requested value/state.

Header declaration location: line 1358.

HRL GetGizmoActiveOperation

```
HRL_EGizmoOperation HRL_GetGizmoActiveOperation(HRL_id _gizmoid);
```

Returns the transform operation currently being executed by the active gizmo interaction.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_EGizmoOperation representing the requested value/state.

Header declaration location: line 1359.

HRL SetGizmoRotateArcDegrees

```
void HRL_SetGizmoRotateArcDegrees(HRL_id _gizmoid, float _degrees);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_degrees	float _degrees	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1360.

HRL SetGizmoChangedCallback

```
void HRL_SetGizmoChangedCallback(HRL_id _gizmoid, HRL_CGizmoChanged _callback, void* _user_data);
```

Registers a callback fired when a gizmo interaction changes its manipulated transform/part.

Parameters

Name	Declaration	Description
_gizmoid	HRL_id _gizmoid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_callback	HRL_CGizmoChanged _callback	Function pointer invoked by the corresponding subsystem when the associated event/state change occurs.
_user_data	void* _user_data	Opaque pointer passed unchanged to the callback, allowing application-defined state to be associated with the event.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1361.

VFX / Particle Systems

VFX is organized as a hierarchy: scene -> VFX system -> emitter -> curve. The system defines playback and transform state, emitters define particle generation/appearance/physics, and curves provide time-varying values.

HRL CreateVFXSystem

```
HRL_id HRL_CreateVFXSystem(HRL_id _sceneid);
```

Creates a particle/VFX simulation container associated with a scene.

```
=====
=====
```

VFX / PARTICLE SYSTEMS

```
=====
=====
```

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 1367.

HRL DeleteVFXSystem

```
void HRL_DeleteVFXSystem(HRL_id _systemid);
```

Destroys the referenced eVFXSystem object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1368.

HRL IsValidVFXSystem

```
int HRL_IsValidVFXSystem(HRL_id _systemid);
```

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1369.

HRL SetVFXSystemPosition

```
void HRL_SetVFXSystemPosition(HRL_id _systemid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1371.

HRL GetVFXSystemPosition

```
void HRL_GetVFXSystemPosition(HRL_id _systemid, float* x, float* y, float* z);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float* x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float* y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float* z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1372.

HRL SetVFXSystemRotation

```
void HRL_SetVFXSystemRotation(HRL_id _systemid, float pitch, float yaw, float roll);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1373.

HRL GetVFXSystemRotation

```
void HRL_GetVFXSystemRotation(HRL_id _systemid, float* pitch, float* yaw, float* roll);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float* pitch	Caller-provided pointer used as input or output according to the function signature.
yaw	float* yaw	Caller-provided pointer used as input or output according to the function signature.
roll	float* roll	Caller-provided pointer used as input or output according to the function signature.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1374.

HRL SetVFXSystemScale

```
void HRL_SetVFXSystemScale(HRL_id _systemid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1375.

HRL GetVFXSystemScale

```
void HRL_GetVFXSystemScale(HRL_id _systemid, float* x, float* y, float* z);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float* x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float* y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float* z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1376.

HRL PlayVFXSystem

```
void HRL_PlayVFXSystem(HRL_id _systemid);
```

Starts or resumes VFX system simulation.

Parameters

Name	Declaration	Description
_systemid	HRL_id_systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1378.

HRL StopVFXSystem

```
void HRL_StopVFXSystem(HRL_id_systemid);
```

Stops VFX system playback and places the system into its stopped state.

Parameters

Name	Declaration	Description
_systemid	HRL_id_systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1379.

HRL PauseVFXSystem

```
void HRL_PauseVFXSystem(HRL_id_systemid);
```

Pauses VFX simulation without discarding its current state.

Parameters

Name	Declaration	Description
_systemid	HRL_id_systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1380.

HRL ResetVFXSystem

```
void HRL_ResetVFXSystem(HRL_id_systemid);
```

Resets VFX system simulation state, including its active particle population and timeline position.

Parameters

Name	Declaration	Description
_systemid	HRL_id_systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1381.

HRL SetVFXSystemLooping

```
void HRL_SetVFXSystemLooping(HRL_id_systemid, int _looping);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id_systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Name	Declaration	Description
_looping	int _looping	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1382.

HRL SetVFXSystemTimeScale

```
void HRL_SetVFXSystemTimeScale(HRL_id _systemid, float _scale);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_scale	float _scale	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1383.

HRL SetVFXSystemEnabled

```
void HRL_SetVFXSystemEnabled(HRL_id _systemid, int _enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_enabled	int _enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1384.

HRL SetVFXSystemAutoUpdate

```
void HRL_SetVFXSystemAutoUpdate(HRL_id _systemid, int _auto_update);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_auto_update	int _auto_update	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1385.

HRL SetVFXSystemDuration

```
void HRL_SetVFXSystemDuration(HRL_id _systemid, float _duration);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_duration	float _duration	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1386.

HRL CreateVFXEmitter

```
HRL_id HRL_CreateVFXEmitter(HRL_id _systemid);
```

Creates an emitter owned by the specified VFX system.

Parameters

Name	Declaration	Description
_systemid	HRL_id _systemid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 1388.

HRL DeleteVFXEmitter

```
void HRL_DeleteVFXEmitter(HRL_id _emitterid);
```

Destroys the referenced eVFXEmitter object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1389.

HRL IsValidVFXEmitter

```
int HRL_IsValidVFXEmitter(HRL_id _emitterid);
```

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1390.

HRL SetVFXEmitterEnabled

```
void HRL_SetVFXEmitterEnabled(HRL_id _emitterid, int _enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id_emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_enabled	int_enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1392.

HRL SetVFXEmitterPosition

```
void HRL_SetVFXEmitterPosition(HRL_id_emitterid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id_emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1393.

HRL SetVFXEmitterRotation

```
void HRL_SetVFXEmitterRotation(HRL_id_emitterid, float pitch, float yaw, float roll);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id_emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1394.

HRL SetVFXEmitterMaxParticles

```
void HRL_SetVFXEmitterMaxParticles(HRL_id_emitterid, HRL_uint_max_particles);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id_emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_max_particles	HRL_uint_max_particles	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1395.

HRL SetVFXEmitterSpawnRate

```
void HRL_SetVFXEmitterSpawnRate(HRL_id _emitterid, float _particles_per_second);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_particles_per_second	float _particles_per_second	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1396.

HRL SetVFXEmitterBurst

```
void HRL_SetVFXEmitterBurst(HRL_id _emitterid, HRL_uint _count);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_count	HRL_uint _count	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1397.

HRL AddVFXBurst

```
void HRL_AddVFXBurst(HRL_id _emitterid, float _time, HRL_uint _count);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_time	float _time	Time value in seconds within the relevant timeline unless the function name indicates another unit.
_count	HRL_uint _count	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1398.

HRL ClearVFXBursts

```
void HRL_ClearVFXBursts(HRL_id _emitterid);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1399.

HRL SetVFXEmitterLifetime

```
void HRL_SetVFXEmitterLifetime(HRL_id _emitterid, float _min_lifetime, float _max_lifetime);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_min_lifetime	float _min_lifetime	Time value in seconds within the relevant timeline unless the function name indicates another unit.
_max_lifetime	float _max_lifetime	Time value in seconds within the relevant timeline unless the function name indicates another unit.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1400.

HRL SetVFXEmitterSpawnShape

```
void HRL_SetVFXEmitterSpawnShape(HRL_id _emitterid, HRL_EVFXSpawnShape _shape);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_shape	HRL_EVFXSpawnShape _shape	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1401.

HRL SetVFXEmitterShapeRadius

```
void HRL_SetVFXEmitterShapeRadius(HRL_id _emitterid, float _radius);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_radius	float _radius	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1402.

HRL SetVFXEmitterShapeSize

```
void HRL_SetVFXEmitterShapeSize(HRL_id _emitterid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1403.

HRL SetVFXEmitterShapeAngle

```
void HRL_SetVFXEmitterShapeAngle(HRL_id _emitterid, float _angle_degrees);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_angle_degrees	float _angle_degrees	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1404.

HRL SetVFXEmitterInitialVelocity

```
void HRL_SetVFXEmitterInitialVelocity(HRL_id _emitterid, float minX, float minY, float minZ, float maxX, float maxY, float maxZ);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
minX	float minX	Input value controlling the named operation.
minY	float minY	Input value controlling the named operation.
minZ	float minZ	Input value controlling the named operation.
maxX	float maxX	Input value controlling the named operation.
maxY	float maxY	Input value controlling the named operation.
maxZ	float maxZ	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1405.

HRL SetVFXEmitterInitialSpeed

```
void HRL_SetVFXEmitterInitialSpeed(HRL_id _emitterid, float _min_speed, float _max_speed);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_min_speed	float _min_speed	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.
_max_speed	float _max_speed	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1407.

HRL SetVFXEmitterInitialRotation

```
void HRL_SetVFXEmitterInitialRotation(HRL_id _emitterid, float minX, float minY, float minZ, float maxX, float maxY, float maxZ);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
minX	float minX	Input value controlling the named operation.
minY	float minY	Input value controlling the named operation.
minZ	float minZ	Input value controlling the named operation.
maxX	float maxX	Input value controlling the named operation.
maxY	float maxY	Input value controlling the named operation.
maxZ	float maxZ	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1408.

HRL SetVFXEmitterAngularVelocity

```
void HRL_SetVFXEmitterAngularVelocity(HRL_id _emitterid, float minX, float minY, float minZ, float maxX, float maxY, float maxZ);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
minX	float minX	Input value controlling the named operation.
minY	float minY	Input value controlling the named operation.
minZ	float minZ	Input value controlling the named operation.
maxX	float maxX	Input value controlling the named operation.
maxY	float maxY	Input value controlling the named operation.
maxZ	float maxZ	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1410.

HRL SetVFXGravity

```
void HRL_SetVFXGravity(HRL_id _emitterid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1412.

HRL SetVFXDrag

```
void HRL_SetVFXDrag(HRL_id _emitterid, float _drag);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_drag	float _drag	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1413.

HRL SetVFXForce

```
void HRL_SetVFXForce(HRL_id _emitterid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1414.

HRL SetVFXNoise

```
void HRL_SetVFXNoise(HRL_id _emitterid, float _strength, float _frequency, float _scroll_speed);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_strength	float _strength	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.
_frequency	float _frequency	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.
_scroll_speed	float _scroll_speed	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1415.

HRL SetVFXEmitterRenderMode

```
void HRL_SetVFXEmitterRenderMode(HRL_id _emitterid, HRL_EVFXRenderMode _mode);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_mode	HRL_EVFXRenderMode _mode	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1416.

HRL SetVFXEmitterTexture

```
void HRL_SetVFXEmitterTexture(HRL_id _emitterid, HRL_id _textureid);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_textureid	HRL_id _textureid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1417.

HRL SetVFXEmitterMaterial

```
void HRL_SetVFXEmitterMaterial(HRL_id _emitterid, HRL_id _materialid);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_materialid	HRL_id _materialid	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1418.

HRL SetVFXEmitterMesh

```
void HRL_SetVFXEmitterMesh(HRL_id _emitterid, HRL_id _meshid);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_meshid	HRL_id _meshid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1419.

HRL SetVFXEmitterMeshScale

```
void HRL_SetVFXEmitterMeshScale(HRL_id _emitterid, float x, float y, float z);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1420.

HRL SetVFXEmitterMeshRotation

```
void HRL_SetVFXEmitterMeshRotation(HRL_id _emitterid, float pitch, float yaw, float roll);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
pitch	float pitch	Input value controlling the named operation.
yaw	float yaw	Input value controlling the named operation.
roll	float roll	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1421.

HRL SetVFXEmitterBlendMode

```
void HRL_SetVFXEmitterBlendMode(HRL_id _emitterid, HRL_EVFXBlendMode _mode);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_mode	HRL_EVFXBlendMode _mode	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1422.

HRL_SetVFXEmitterParticleSize

```
void HRL_SetVFXEmitterParticleSize(HRL_id _emitterid, float x, float y);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1423.

HRL_SetVFXEmitterStretch

```
void HRL_SetVFXEmitterStretch(HRL_id _emitterid, float _amount);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_amount	float _amount	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1424.

HRL_SetVFXEmitterSimulationSpace

```
void HRL_SetVFXEmitterSimulationSpace(HRL_id _emitterid, HRL_EVFXSimulationSpace _space);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_space	HRL_EVFXSimulationSpace _space	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1425.

HRL_CreateVFXColorCurve

```
HRL_id HRL_CreateVFXColorCurve(HRL_id _emitterid);
```

Creates a color animation curve owned/associated with an emitter.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 1427.

HRL_CreateVFXFloatCurve

```
HRL_id HRL_CreateVFXFloatCurve(HRL_id _emitterid);
```

Creates a scalar animation curve owned/associated with an emitter.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 1428.

HRL_DeleteVFXCurve

```
void HRL_DeleteVFXCurve(HRL_id _curveid);
```

Destroys the referenced eVFXCurve object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1429.

HRL_IsValidVFXCurve

```
int HRL_IsValidVFXCurve(HRL_id _curveid);
```

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1430.

HRL_AddVFXColorKey

```
void HRL_AddVFXColorKey(HRL_id _curveid, float _time, float r, float g, float b, float a);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_time	float _time	Time value in seconds within the relevant timeline unless the function name indicates another unit.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1431.

HRL AddVFXFloatKey

```
void HRL_AddVFXFloatKey(HRL_id _curveid, float _time, float _value);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_time	float _time	Time value in seconds within the relevant timeline unless the function name indicates another unit.
_value	float _value	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1432.

HRL ClearVFXColorKeys

```
void HRL_ClearVFXColorKeys(HRL_id _curveid);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1433.

HRL ClearVFXFloatKeys

```
void HRL_ClearVFXFloatKeys(HRL_id _curveid);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1434.

HRL SetVFXEmitterColorCurve

```
void HRL_SetVFXEmitterColorCurve(HRL_id _emitterid, HRL_id _curveid);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1435.

HRL SetVFXEmitterSizeCurve

```
void HRL_SetVFXEmitterSizeCurve(HRL_id _emitterid, HRL_id _curveid);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1436.

HRL SetVFXEmitterRotationCurve

```
void HRL_SetVFXEmitterRotationCurve(HRL_id _emitterid, HRL_id _curveid);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_curveid	HRL_id _curveid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1437.

HRL SetVFXCollisionEnabled

```
void HRL_SetVFXCollisionEnabled(HRL_id _emitterid, int _enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Name	Declaration	Description
_enabled	int_enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1439.

HRL SetVFXCollisionRestitution

```
void HRL_SetVFXCollisionRestitution(HRL_id _emitterid, float _restitution);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_restitution	float _restitution	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1440.

HRL SetVFXCollisionFriction

```
void HRL_SetVFXCollisionFriction(HRL_id _emitterid, float _friction);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_friction	float _friction	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1441.

HRL SetVFXCollisionScene

```
void HRL_SetVFXCollisionScene(HRL_id _emitterid, HRL_id _sceneid, int _enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
_emitterid	HRL_id _emitterid	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_enabled	int_enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1442.

HRL UpdateVFX

```
void HRL_UpdateVFX(float _delta_seconds);
```

Advances all automatic VFX simulations by the specified time step.

Advances VFX systems that have AutoUpdate disabled.

Parameters

Name	Declaration	Description
_delta_seconds	float_delta_seconds	Elapsed simulation time for the update, in seconds.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1445.

Fog */

HRL SetFogEnabled

```
void HRL_SetFogEnabled(HRL_id scene, int enable);
```

Enables or disables the fog effect for a scene.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	ID of the target scene.
enable	int enable	HRL_TRUE to enable, HRL_FALSE to disable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1457.

HRL SetFogMode

```
void HRL_SetFogMode(HRL_id scene, HRL_EFogType mode);
```

Sets the fog blending mode for a scene.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	ID of the target scene.
mode	HRL_EFogType mode	One of HRL_FOG_LINEAR, HRL_FOG_EXPONENTIAL, or HRL_FOG_EXP_SQUARED.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1464.

HRL SetFogColor

```
void HRL_SetFogColor(HRL_id scene, float r, float g, float b);
```

Sets the fog color for a scene.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	ID of the target scene.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1471.

HRL SetFogDensity

```
void HRL_SetFogDensity(HRL_id scene, float density);
```

Sets the fog density for exponential fog modes.

Updates the named property of the supplied HRL resource or rendering subsystem.

Has no effect when fog mode is HRL_FOG_LINEAR.

Parameters

Name	Declaration	Description
scene	HRL_id scene	ID of the target scene.
density	float density	Density coefficient. Higher values produce thicker fog.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1479.

HRL_SetFogLinearRange

```
void HRL_SetFogLinearRange(HRL_id scene, float start, float end);
```

Sets the start and end distances for linear fog.

Updates the named property of the supplied HRL resource or rendering subsystem.

Objects beyond _end are fully fogged; objects before _start are unaffected.

Parameters

Name	Declaration	Description
scene	HRL_id scene	ID of the target scene.
start	float start	Distance at which fog begins (world units).
end	float end	Distance at which fog reaches full opacity (world units).

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1488.

HRL_CreateVolumetricFog

```
HRL_id HRL_CreateVolumetricFog(HRL_id scene);
```

Creates a localized volumetric fog volume attached to a scene.

Creates a localized volumetric fog volume in a scene.

Localized volumetric fog is an HRL object, so a scene can contain several

independent volumes. The OpenGL 3.3 backend supports up to

HRL_MAX_VOLUMETRIC_FOGS active volumes per scene.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.

Return value

The new volumetric fog ID, or HRL_INVALID_ID on failure.

Header declaration location: line 1499.

HRL_DeleteVolumetricFog

```
void HRL_DeleteVolumetricFog(HRL_id fog);
```

Deletes a localized volumetric fog object.

Destroys the referenced eVolumetricFog object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1502.

HRL IsValidVolumetricFog

```
int HRL_IsValidVolumetricFog(HRL_id fog);
```

Returns HRL_TRUE if the ID refers to a live volumetric fog object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1505.

HRL SetVolumetricFogEnabled

```
void HRL_SetVolumetricFogEnabled(HRL_id fog, int enable);
```

Enables/disables a localized volumetric fog volume.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
enable	int enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1508.

HRL SetVolumetricFogPosition

```
void HRL_SetVolumetricFogPosition(HRL_id fog, float x, float y, float z);
```

Sets the center of the localized volumetric fog volume in world-space.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1510.

HRL SetVolumetricFogRadius

```
void HRL_SetVolumetricFogRadius(HRL_id fog, float radius);
```

Sets the radius of the localized volumetric fog volume.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
radius	float radius	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1512.

HRL SetVolumetricFogDensity

```
void HRL_SetVolumetricFogDensity(HRL_id fog, float density);
```

Sets the density of the localized volumetric fog volume.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
density	float density	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1514.

HRL SetVolumetricFogColor

```
void HRL_SetVolumetricFogColor(HRL_id fog, float r, float g, float b);
```

Sets the color of the localized volumetric fog volume.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1516.

HRL SetVolumetricFogSteps

```
void HRL_SetVolumetricFogSteps(HRL_id fog, HRL_uint steps);
```

Sets the ray-march sample count for localized volumetric fog (4..64).

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
fog	HRL_id fog	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
steps	HRL_uint steps	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1518.

HRL SetGlobalVolumetricFogEnabled

```
void HRL_SetGlobalVolumetricFogEnabled(HRL_id scene, int enable);
```

Enables/disables volumetric fog that fills the whole scene.

Enables or disables the scene-wide volumetric fog layer.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
enable	int enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1522.

HRL SetGlobalVolumetricFogDensity

```
void HRL_SetGlobalVolumetricFogDensity(HRL_id scene, float density);
```

Sets the density of the scene-wide volumetric fog.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
density	float density	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1524.

HRL SetGlobalVolumetricFogColor

```
void HRL_SetGlobalVolumetricFogColor(HRL_id scene, float r, float g, float b);
```

Sets the color of the scene-wide volumetric fog.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1526.

HRL_SetGlobalVolumetricFogSteps

```
void HRL_SetGlobalVolumetricFogSteps(HRL_id scene, HRL_uint steps);
```

Sets the ray-march sample count for the scene-wide volumetric fog (4..64).

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
steps	HRL_uint steps	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1528.

HRL_SetGodRaysEnabled

```
void HRL_SetGodRaysEnabled(HRL_id scene, int enable);
```

Enables/disables screen-space god rays.

Enables or disables screen-space light shafts (god rays) for the scene.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
enable	int enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1531.

HRL_SetGodRaysPosition

```
void HRL_SetGodRaysPosition(HRL_id scene, float x, float y, float z);
```

Sets the world-space position from which god rays radiate.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.

Name	Declaration	Description
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
z	float z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1533.

HRL_SetGodRaysColor

```
void HRL_SetGodRaysColor(HRL_id scene, float r, float g, float b);
```

Sets the god-ray color.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1535.

HRL_SetGodRaysDensity

```
void HRL_SetGodRaysDensity(HRL_id scene, float density);
```

Sets radial sampling density for god rays.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
density	float density	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1537.

HRL_SetGodRaysDecay

```
void HRL_SetGodRaysDecay(HRL_id scene, float decay);
```

Sets per-sample decay for god rays.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
decay	float decay	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1539.

HRL_SetGodRaysWeight

```
void HRL_SetGodRaysWeight(HRL_id scene, float weight);
```

Sets the contribution weight for god rays.

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
weight	float weight	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1541.

HRL_SetGodRaysSamples

```
void HRL_SetGodRaysSamples(HRL_id scene, HRL_uint samples);
```

Sets the radial sample count for god rays (8..96).

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
samples	HRL_uint samples	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1543.

HRL_SetAmbientOcclusionEnabled

```
void HRL_SetAmbientOcclusionEnabled(HRL_id scene, int enable);
```

Enables or disables the ambient-occlusion pass for the scene.

Screen-space ambient occlusion. Disabled by default.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
enable	int enable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1547.

HRL_SetAmbientOcclusionStrength

```
void HRL_SetAmbientOcclusionStrength(HRL_id scene, float strength);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
strength	float strength	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1548.

HRL_SetAmbientOcclusionRadius

```
void HRL_SetAmbientOcclusionRadius(HRL_id scene, float radius);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
radius	float radius	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1549.

HRL_SetAmbientOcclusionBias

```
void HRL_SetAmbientOcclusionBias(HRL_id scene, float bias);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
bias	float bias	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1550.

HRL_SetAmbientOcclusionPower

```
void HRL_SetAmbientOcclusionPower(HRL_id scene, float power);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
scene	HRL_id scene	Opaque scene handle identifying the scene that owns or contains the affected object or effect.

Name	Declaration	Description
power	float power	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1551.

Matrices, Debug Rendering & Capture

HRL_GetProjectionMatrix

```
void HRL_GetProjectionMatrix(float* aa);
```

Writes the current projection matrix into a caller-provided array.

Writes the current projection matrix into caller-provided 16-float storage.

Parameters

Name	Declaration	Description
aa	float* aa	Pointer to a float[16] array. Matrix is column-major, contiguous.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Matrix layout: the public comments identify a contiguous 16-float matrix. Treat the destination as an array of 16 float values owned by the caller; do not pass a smaller buffer.

Header declaration location: line 1561.

HRL_GetViewMatrix

```
void HRL_GetViewMatrix(float* aa);
```

Writes the current view matrix into a caller-provided array.

Writes the current view matrix into caller-provided 16-float storage.

Parameters

Name	Declaration	Description
aa	float* aa	Pointer to a float[16] array. Matrix is column-major, contiguous.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Matrix layout: the public comments identify a contiguous 16-float matrix. Treat the destination as an array of 16 float values owned by the caller; do not pass a smaller buffer.

Header declaration location: line 1567.

HRL_GetModelMatrix

```
void HRL_GetModelMatrix(HRL_id _meshid, float* aa);
```

Writes the model matrix of a specific mesh into a caller-provided array.

Writes the model/world transform matrix associated with a mesh into caller-provided 16-float storage.

Parameters

Name	Declaration	Description
_meshid	HRL_id _meshid	ID of the target mesh.
aa	float* aa	Pointer to a float[16] array. Matrix is column-major, contiguous.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Matrix layout: the public comments identify a contiguous 16-float matrix. Treat the destination as an array of 16 float values owned by the caller; do not pass a smaller buffer.

Header declaration location: line 1574.

Debug Views

HRL DrawSceneAsDebugMode

```
void HRL_DrawSceneAsDebugMode(HRL_id _sceneid, HRL_EDebugView mode);
```

Overrides the scene rendering with a diagnostic visualization mode.

Overrides normal scene shading with a diagnostic visualization such as unlit, normal, lighting, wireframe, or LOD view.

Useful for inspecting normals, lighting, or other render passes in isolation.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
mode	HRL_EDebugView mode	One of HRL_DEBUG_VIEW_NONE, HRL_DEBUG_VIEW_UNLIT, HRL_DEBUG_VIEW_NORMAL, HRL_DEBUG_VIEW_LIGHTING, HRL_DEBUG_VIEW_WIREFRAME or HRL_DEBUG_VIEW_LOD.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1586.

Debug Geometry

HRL_SetDebugLineThickness

```
void HRL_SetDebugLineThickness(float a);
```

Sets the line thickness used by all debug draw calls.

Updates the named property of the supplied HRL resource or rendering subsystem.

The exact visual result depends on the backend's line rendering support.

Parameters

Name	Declaration	Description
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1597.

HRL_DrawDebugSegment

```
void HRL_DrawDebugSegment(HRL_id _sceneid, float a_x, float a_y, float a_z, float b_x, float b_y, float b_z, float r, float g, float b );
```

Draws a debug line segment for the current frame.

Queues a world-space line segment for debug rendering in the current frame.

Must be called every frame to persist the rendering.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
a_x	float a_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
a_y	float a_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
a_z	float a_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
b_x	float b_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
b_y	float b_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
b_z	float b_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Frame lifetime: debug primitives are intended as transient draw commands. Re-submit them on later frames when persistent visualization is required.

Header declaration location: line 1606.

HRL DrawDebugPolygon

```
void HRL_DrawDebugPolygon(HRL_id_sceneid, HRL_EDebugRenderingType _mode, const float* vertices_x, const float* vertices_y, const float* vertices_z, int vertices_count, float r, float g, float b );
```

Draws a debug polygon (filled or outlined) for the current frame.

Queues a debug polygon using parallel X/Y/Z vertex arrays and the requested filled/outlined mode.

Vertices are specified as separate X, Y and Z arrays of the same length.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_mode	HRL_EDebugRenderingType _mode	HRL_DebugHollow for outline, HRL_DebugSolid for filled.
vertices_x	const float* vertices_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
vertices_y	const float* vertices_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
vertices_z	const float* vertices_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
vertices_count	int vertices_count	Number of vertices.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Frame lifetime: debug primitives are intended as transient draw commands. Re-submit them on later frames when persistent visualization is required.

Header declaration location: line 1618.

HRL DrawDebugCircle

```
void HRL_DrawDebugCircle(HRL_id_sceneid, HRL_EDebugRenderingType _mode, float center_x, float center_y, float center_z, float radius, int segments, float r, float g, float b );
```

Draws a debug circle for the current frame.

Queues a debug circle for the current frame.

Parameters

Name	Declaration	Description
_sceneid	HRL_id_sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_mode	HRL_EDebugRenderingType _mode	HRL_DebugHollow for outline, HRL_DebugSolid for filled.
center_x	float center_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
center_y	float center_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
center_z	float center_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
radius	float radius	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.
segments	int segments	Number of segments used to approximate the circle.

Name	Declaration	Description
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Frame lifetime: debug primitives are intended as transient draw commands. Re-submit them on later frames when persistent visualization is required.

Header declaration location: line 1629.

HRL DrawDebugCapsule

```
void HRL_DrawDebugCapsule(HRL_id _sceneid, HRL_EDebugRenderingType _mode, float a_x, float a_y, float a_z, float b_x, float b_y, float b_z, float radius, int segments, float r, float g, float b );
```

Draws a debug capsule (cylinder with hemispherical caps) for the current frame.

Queues a debug capsule for the current frame.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_mode	HRL_EDebugRenderingType _mode	HRL_DebugHollow or HRL_DebugSolid.
a_x	float a_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
a_y	float a_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
a_z	float a_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
b_x	float b_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
b_y	float b_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
b_z	float b_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
radius	float radius	Scalar control parameter for the named feature. Positive ranges, normalization, and exact defaults follow the renderer implementation and the selected backend where applicable.
segments	int segments	Number of segments used to approximate the capsule.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Frame lifetime: debug primitives are intended as transient draw commands. Re-submit them on later frames when persistent visualization is required.

Header declaration location: line 1642.

HRL DrawDebugPoint

```
void HRL_DrawDebugPoint(HRL_id _sceneid, float a_x, float a_y, float a_z, float size, float r, float g, float b );
```

Draws a debug point (screen-space square) for the current frame.

Queues a debug point for the current frame.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
a_x	float a_x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
a_y	float a_y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
a_z	float a_z	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
size	float size	Point size in pixels.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Frame lifetime: debug primitives are intended as transient draw commands. Re-submit them on later frames when persistent visualization is required.

Header declaration location: line 1653.

Utility

HRL TakeScreenshot

```
void HRL_TakeScreenshot(HRL_id _sceneid, const char* _target_path);
```

Captures the rendered output of a scene and saves it as a PNG file.

Captures the rendered scene output at the end of the current frame and writes it as a PNG at the requested path.

The request is queued and consumed at the end of the current frame.

On OpenGL 3.3 the capture includes the final on-screen post-process result.

Parameters

Name	Declaration	Description
_sceneid	HRL_id _sceneid	Opaque scene handle identifying the scene that owns or contains the affected object or effect.
_target_path	const char* _target_path	Absolute path where the PNG image will be written.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1670.

HRL SetAntialiasingMode

```
void HRL_SetAntialiasingMode(HRL_uint _mode);
```

Enables or changes the global multisample anti-aliasing mode.

Updates the named property of the supplied HRL resource or rendering subsystem.

HRL_ANTIALIASING_4X or HRL_ANTIALIASING_8X.

Parameters

Name	Declaration	Description
_mode	HRL_uint _mode	One of HRL_ANTIALIASING_OFF, HRL_ANTIALIASING_2X,

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1677.

Fonts

HRL CreateFont

```
HRL_id HRL_CreateFont(const char* _data, size_t _data_size);
```

Loads a TrueType font from a memory buffer for use with HRL_CreateTextureFromText.

Creates a font resource from in-memory font data.

Parameters

Name	Declaration	Description
_data	const char* _data	Pointer to the caller-provided binary/text data consumed by the operation.
_data_size	size_t _data_size	Size of the buffer in bytes.

Return value

HRL_id of the new font, or HRL_INVALID_ID on failure.

Header declaration location: line 1690.

HRL DeleteFont

```
void HRL_DeleteFont(HRL_id _fontid);
```

Destroys a font and frees its associated resources.

Destroys the referenced eFont object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
_fontid	HRL_id _fontid	ID of the font to delete.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1696.

HRL IsValidFont

```
int HRL_IsValidFont(HRL_id _id);
```

Returns whether the given ID refers to a live font object.

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
_id	HRL_id _id	ID to test.

Return value

HRL_TRUE if valid, HRL_FALSE otherwise.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1703.

Ui

HRL CButtonPressed

```
typedef void(*HRL_CButtonPressed)(HRL_id button, int clicked, int released, void* user_data); HRL_API void  
HRL_SetButtonPressedCallback(HRL_id widget, HRL_CButtonPressed callback, void* user_data);
```

Callback signature inferred from its name and subsystem role: HRL invokes it when the associated event or state transition occurs, passing the resource handle and event data.

HRL CSliderChanged

```
typedef void(*HRL_CSliderChanged)(HRL_id slider, float value, void* user_data); HRL_API void HRL_SetSliderRange(HRL_id  
widget, float minimum, float maximum); HRL_API void HRL_SetSliderValue(HRL_id widget, float value);
```

SLIDER

HRL CCheckboxChanged

```
typedef void(*HRL_CCheckboxChanged)(HRL_id checkbox, int checked, void* user_data); HRL_API void  
HRL_SetCheckboxChecked(HRL_id widget, int checked); HRL_API int HRL_IsCheckboxChecked(HRL_id widget);
```

CHECKBOX

HRL MouseMovedCallback

```
void HRL_MouseMovedCallback(float x, float y);
```

Feeds the latest window-relative mouse position into HRL input handling so widgets and gizmos can update their hover state.

Mouse input is window-relative in pixels. Call these from the application

input callbacks before `HRL_EndFrame()`.

Parameters

Name	Declaration	Description
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1712.

HRL MouseButtonCallback

```
void HRL_MouseButtonCallback(int button, int pressed);
```

Feeds a mouse button transition into HRL input handling using the `HRL_MOUSE_*` button and press/release constants.

Parameters

Name	Declaration	Description
button	int button	Input value controlling the named operation.
pressed	int pressed	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1713.

HRL CreateWidget

```
HRL_id HRL_CreateWidget(HRL_id viewport, HRL_EWidgetType type);
```

Creates a widget of the requested type and places it in the supplied viewport.

Widget positions are normalized to their owning viewport. Widget sizes use the existing normalized API as the initial size, then keep the resulting pixel size across window resizes so aspect-ratio changes do not stretch controls.

Parameters

Name	Declaration	Description
viewport	HRL_id viewport	Input value controlling the named operation.
type	HRL_EWidgetType type	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

Returns HRL_id. For resource creation and name-lookup functions, HRL_INVALID_ID is the failure/not-found sentinel.

Header declaration location: line 1718.

HRL DeleteWidget

```
void HRL_DeleteWidget(HRL_id widget);
```

Destroys the referenced eWidget object and releases the resources owned by that object. The handle should not be reused after this call.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

After successful destruction, the corresponding handle must be considered invalid and should not be passed to other object operations.

Header declaration location: line 1719.

HRL IsValidWidget

```
int HRL_IsValidWidget(HRL_id widget);
```

Checks whether the supplied opaque handle currently refers to a live resource of the expected type.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Validity queries are safe to use as a guard before invoking operations on a potentially stale handle. A valid result means the identifier still resolves to a live object of the expected class/type.

Header declaration location: line 1720.

HRL SetWidgetPosition

```
void HRL_SetWidgetPosition(HRL_id widget, float x, float y);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
x	float x	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
y	float y	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1722.

HRL SetWidgetSize

```
void HRL_SetWidgetSize(HRL_id widget, float width, float height);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
width	float width	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.
height	float height	Dimension of the target region or resource, normally expressed in pixels for framebuffer/UI dimensions.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1723.

HRL SetWidgetAlpha

```
void HRL_SetWidgetAlpha(HRL_id widget, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1724.

HRL SetWidgetAnchor

```
void HRL_SetWidgetAnchor(HRL_id widget, float ax, float ay);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
ax	float ax	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.
ay	float ay	Position, size, direction, or other component on the corresponding axis, expressed in the coordinate system used by the owning API.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1725.

HRL SetWidgetVisible

```
void HRL_SetWidgetVisible(HRL_id widget, int visible);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Name	Declaration	Description
visible	int visible	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1726.

HRL SetWidgetEnabled

```
void HRL_SetWidgetEnabled(HRL_id widget, int enabled);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
enabled	int enabled	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1727.

HRL SetWidgetZIndex

```
void HRL_SetWidgetZIndex(HRL_id widget, int z_index);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
z_index	int z_index	Count/index value controlling how many elements are processed or which element is addressed.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1728.

HRL IsWidgetHovered

```
int HRL_IsWidgetHovered(HRL_id widget);
```

Reports whether the widget is currently under the pointer according to HRL UI hit testing.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 1729.

HRL SetButtonClickable

```
void HRL_SetButtonClickable(HRL_id widget, int clickable);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

BUTTON

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
clickable	int clickable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1732.

HRL_SetButtonText

```
void HRL_SetButtonText(HRL_id widget, const char* text);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
text	const char* text	Null-terminated text string consumed by the function.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1733.

HRL_SetButtonTextSize

```
void HRL_SetButtonTextSize(HRL_id widget, float size);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
size	float size	Size of the associated data buffer in bytes.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1734.

HRL_SetButtonTextTintColor

```
void HRL_SetButtonTextTintColor(HRL_id widget, HRL_EWidgetState state, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
state	HRL_EWidgetState state	Input value controlling the named operation.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1735.

HRL SetButtonTextFont

```
void HRL_SetButtonTextFont(HRL_id widget, HRL_id font);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
font	HRL_id font	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1736.

HRL SetButtonBackgroundTexture

```
void HRL_SetButtonBackgroundTexture(HRL_id widget, HRL_EWidgetState state, HRL_id texture);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
state	HRL_EWidgetState state	Input value controlling the named operation.
texture	HRL_id texture	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1737.

HRL SetButtonBackgroundTintColor

```
void HRL_SetButtonBackgroundTintColor(HRL_id widget, HRL_EWidgetState state, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
state	HRL_EWidgetState state	Input value controlling the named operation.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1738.

HRL SetButtonPressedCallback

```
void HRL_SetButtonPressedCallback(HRL_id widget, HRL_CButtonPressed callback, void* user_data);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
callback	HRL_CButtonPressed callback	Function pointer invoked by the corresponding subsystem when the associated event/state change occurs.
user_data	void* user_data	Opaque pointer passed unchanged to the callback, allowing application-defined state to be associated with the event.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1740.

HRL SetLabelText

```
void HRL_SetLabelText(HRL_id widget, const char* text);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

LABEL

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
text	const char* text	Null-terminated text string consumed by the function.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1743.

HRL SetLabelTextSize

```
void HRL_SetLabelTextSize(HRL_id widget, float size);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
size	float size	Size of the associated data buffer in bytes.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1744.

HRL SetLabelFont

```
void HRL_SetLabelFont(HRL_id widget, HRL_id font);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
font	HRL_id font	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1745.

HRL SetLabelTintColor

```
void HRL_SetLabelTintColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1746.

HRL SetImageTexture

```
void HRL_SetImageTexture(HRL_id widget, HRL_id texture);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

IMAGE

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
texture	HRL_id texture	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1749.

HRL SetImageTintColor

```
void HRL_SetImageTintColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1750.

HRL SetSliderRange

```
void HRL_SetSliderRange(HRL_id widget, float minimum, float maximum);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
minimum	float minimum	Input value controlling the named operation.
maximum	float maximum	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1754.

HRL SetSliderValue

```
void HRL_SetSliderValue(HRL_id widget, float value);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
value	float value	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1755.

HRL GetSliderValue

```
float HRL_GetSliderValue(HRL_id widget);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested float value.

Header declaration location: line 1756.

HRL SetSliderOrientation

```
void HRL_SetSliderOrientation(HRL_id widget, HRL_ESliderOrientation orientation);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
orientation	HRL_ESliderOrientation orientation	Enumeration value selecting the requested operating mode, type, space, method, or category.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1757.

HRL SetSliderClickable

```
void HRL_SetSliderClickable(HRL_id widget, int clickable);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
clickable	int clickable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1758.

HRL SetSliderBackgroundColor

```
void HRL_SetSliderBackgroundColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1759.

HRL SetSliderFillColor

```
void HRL_SetSliderFillColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1760.

HRL_SetSliderHandleColor

```
void HRL_SetSliderHandleColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1761.

HRL_SetSliderChangedCallback

```
void HRL_SetSliderChangedCallback(HRL_id widget, HRL_CSliderChanged callback, void* user_data);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
callback	HRL_CSliderChanged callback	Function pointer invoked by the corresponding subsystem when the associated event/state change occurs.
user_data	void* user_data	Opaque pointer passed unchanged to the callback, allowing application-defined state to be associated with the event.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1762.

HRL_SetCheckboxChecked

```
void HRL_SetCheckboxChecked(HRL_id widget, int checked);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
checked	int checked	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1766.

HRL IsCheckboxChecked

```
int HRL_IsCheckboxChecked(HRL_id widget);
```

Performs the operation represented by the function name on the supplied HRL objects and parameters.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns an integer result. For Is*/check-style functions this is a boolean-style status: HRL_TRUE indicates the queried condition is satisfied and HRL_FALSE indicates that it is not.

Header declaration location: line 1767.

HRL SetCheckboxClickable

```
void HRL_SetCheckboxClickable(HRL_id widget, int clickable);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
clickable	int clickable	Boolean-style flag: use HRL_TRUE (1) to enable the feature/state and HRL_FALSE (0) to disable it.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1768.

HRL SetCheckboxBackgroundColor

```
void HRL_SetCheckboxBackgroundColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1769.

HRL SetCheckboxCheckedColor

```
void HRL_SetCheckboxCheckedColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1770.

HRL_SetCheckboxChangedCallback

```
void HRL_SetCheckboxChangedCallback(HRL_id widget, HRL_CCheckboxChanged callback, void* user_data);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
callback	HRL_CCheckboxChanged callback	Function pointer invoked by the corresponding subsystem when the associated event/state change occurs.
user_data	void* user_data	Opaque pointer passed unchanged to the callback, allowing application-defined state to be associated with the event.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1771.

HRL_SetProgressBarValue

```
void HRL_SetProgressBarValue(HRL_id widget, float value);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

PROGRESS BAR

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
value	float value	Input value controlling the named operation.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Header declaration location: line 1774.

HRL_GetProgressBarValue

```
float HRL_GetProgressBarValue(HRL_id widget);
```

Queries the current value, metadata, or state represented by the supplied HRL resource or subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.

Return value

Returns the requested float value.

Header declaration location: line 1775.

HRL SetProgressBarBackgroundColor

```
void HRL_SetProgressBarBackgroundColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1776.

HRL SetProgressBarFillColor

```
void HRL_SetProgressBarFillColor(HRL_id widget, float r, float g, float b, float a);
```

Updates the named property of the supplied HRL resource or rendering subsystem.

Parameters

Name	Declaration	Description
widget	HRL_id widget	Opaque HRL resource handle used by this operation. The handle must refer to a live resource of the expected type.
r	float r	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
g	float g	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
b	float b	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.
a	float a	Color component. The conventional HRL range is 0..1 unless the surrounding function explicitly allows another range.

Return value

No value is returned. Failures are reported through the library error state/callback mechanisms and the validity of affected handles.

Color interpretation: unless the surrounding subsystem explicitly states otherwise, the API uses normalized floating-point color components and preserves the supplied alpha component where one is present.

Header declaration location: line 1777.

Types, Constants And Callbacks

HRL CGizmoChanged

```
typedef void (*HRL_CGizmoChanged)(HRL_id gizmo, HRL_EGizmoPart part, void* user_data); /*  
=====
```

Callback signature inferred from its name and subsystem role: HRL invokes it when the associated event or state transition occurs, passing the resource handle and event data.

6. Recommended Integration Patterns

Basic application loop

```
#include "hrl.h"

HRL_Init(HRL_OPENGL_33);
HRL_InitContext(width, height, loader);

while (running) {
    HRL_BeginFrame();
    // update game state
    // render scenes / widgets / debug primitives
    HRL_EndFrame();
}

HRL_Shutdown();
```

Scene, camera and viewport

```
HRL_id scene = HRL_CreateScene(HRL_TRUE);
HRL_id camera = HRL_CreateCamera(scene, HRL_PERSPECTIVE);
HRL_SetCameraPerspectiveFov(camera, 60.0f);
HRL_SetCameraNearPlane(camera, 0.1f);
HRL_SetCameraFarPlane(camera, 1000.0f);

HRL_id viewport = HRL_CreateViewport(scene, camera,
0.0f, 0.0f,
(float)width, (float)height);
```

Static mesh and material

```
HRL_id mesh = HRL_CreateMesh3D(scene, vertices, vertexCount, indices, indexCount);
HRL_id shader = HRL_CreateShader(vs, vsSize, fs, fsSize);
HRL_id material = HRL_CreateMaterial(shader);
HRL_MaterialSetTexture(material, HRL_T_ALBEDO, albedoTexture);
HRL_MaterialSetFloat(material, HRL_T_ROUGHNESS, 0.5f);
HRL_SetMeshMaterial(mesh, material);
```

Asynchronous resource synchronization

```
HRL_id tex = HRL_CreateTextureAsync(fileBytes, fileSize);
HRL_id shader = HRL_CreateShaderAsync(vs, vsSize, fs, fsSize);

// Polling model
while (!HRL_IsTextureReady(tex) || !HRL_IsShaderReady(shader)) {
    /* run your application wait/update policy */
}

// Or use deterministic synchronization:
HRL_WaitForTexture(tex);
HRL_WaitForShader(shader);
```

Skeletal animation update

```
HRL_id skeletal = HRL_CreateSkeletalMesh(scene, &skeletalData);
HRL_PlaySkeletalAnimation(skeletal, 0);
HRL_SetSkeletalAnimationLoop(skeletal, HRL_TRUE);

// Once per simulation frame:
HRL_UpdateSkeletalAnimations(deltaSeconds);
```

VFX hierarchy

```
HRL_id system = HRL_CreateVFXSystem(scene);
HRL_id emitter = HRL_CreateVFXEmitter(system);
HRL_SetVFXEmitterSpawnRate(emitter, 80.0f);
HRL_SetVFXEmitterLifetime(emitter, 0.5f, 1.2f);
HRL_SetVFXEmitterRenderMode(emitter, HRL_VFX_RENDER_BILLBOARD);
HRL_PlayVFXSystem(system);

// Drive the simulation if auto-update is disabled or manual updates are desired.
HRL_UpdateVFX(deltaSeconds);
```

Interactive gizmo

```
HRL_id gizmo = HRL_CreateGizmo(viewport);
HRL_SetGizmoMode(gizmo, HRL_GIZMO_MODE_TRANSLATE | HRL_GIZMO_MODE_ROTATE);
HRL_SetGizmoSpace(gizmo, HRL_GIZMO_SPACE_WORLD);
HRL_SetGizmoVisible(gizmo, HRL_TRUE);

HRL_EGizmoPart part = HRL_GetGizmoHoveredPart(gizmo);
HRL_EGizmoOperation op = HRL_GetGizmoActiveOperation(gizmo);
```

UI button

```
HRL_id button = HRL_CreateWidget(viewport, HRL_WIDGET_BUTTON);
HRL_SetWidgetPosition(button, 32.0f, 32.0f);
HRL_SetWidgetSize(button, 180.0f, 48.0f);
HRL_SetButtonText(button, "Continue");
HRL_SetButtonClickable(button, HRL_TRUE);
```

7. Resource Teardown Strategy

A predictable teardown order reduces stale-handle references. At application shutdown, stop or detach high-level systems first, then delete dependent resources, and finally call `HRL_Shutdown` before destroying the native graphics/window context. For importer allocations returned as pointers, call the matching `HRL_Free*` routine exactly once.

```
/* Example conceptual teardown */
HRL_DeleteWidget(button);
HRL_DeleteGizmo(gizmo);
HRL_DeleteViewport(viewport);
HRL_DeleteCamera(camera);
HRL_DeleteMesh(mesh);
HRL_DeleteMaterial(material);
HRL_DeleteShader(shader);
HRL_DeleteTexture(texture);
HRL_DeleteScene(scene);
HRL_Shutdown();
```

8. Design Notes and Inferred Contracts

The following rules are intentionally presented as design-level interpretations derived from the API surface:

- Creation functions produce stable opaque identifiers that identify runtime objects until their matching deletion function releases them.
- Getter functions that return pointers expose data owned by HRL or an importer; callers should treat those pointers as read-only and lifetime-bound to the owning object.
- Setters are state-mutating operations and generally do not require re-creation of the underlying resource.
- Async creation separates scheduling from readiness; `Is*Ready` is a non-blocking state query, while `WaitFor*` is the synchronization point.
- Enums are designed as forward-compatible public vocabulary: the presence of an enum value does not by itself prove that every backend implements the feature.
- Debug primitives are frame-scoped rendering commands rather than persistent scene objects.
- Viewport, camera, scene and post-process handles form a rendering graph in which the viewport selects the camera/rectangle used to present a scene and post-process objects attach to that viewport.

C++ API Reference

Horizon Rendering Library - Native C++ interface

API version: 26.6

This section documents the C++ wrapper layer exposed by `hrl.hpp`. It is intentionally placed after the C reference so the two APIs can be used side by side.

C++ API model

The C++ interface is a thin, RAII-oriented wrapper over the same renderer concepts exposed by the C API. Resource classes are move-only, carry an HRL identifier internally, and use `Borrowed()` factory functions where non-owning views are required.

C++ API contents

#	Section
1	C++ type system and ownership
2	Core data structures and callbacks
3	Enumerations and constants
4	Resource classes
5	FBX and free-function utilities
6	Frame, input, debug and capability functions
7	Recommended usage patterns

1. C++ Type System and Ownership

The C++ API uses strongly typed wrappers around the HRL handle model. The underlying identifier type remains `std::uint32_t`, while wrapper classes provide lifetime management, type-specific operations, and C++ references for related resources.

```
hrl::Instance renderer(hrl::HRL_OPENGL_33, width, height, loader);  
hrl::Scene scene(true);  
hrl::Camera camera(scene, hrl::HRL_PERSPECTIVE);  
hrl::Viewport viewport(scene, &camera, 0.0f, 0.0f, (float)width, (float)height);
```

Concept	C++ rule	Practical consequence
Ownership	Constructors create/attach to resources; destructors release owned resources.	Prefer stack allocation and let scope define lifetime.
Copying	Resource wrappers delete copy construction and copy assignment.	Do not duplicate handles by copying wrapper objects.
Moving	Wrappers generally provide noexcept move construction/assignment.	Containers and return-by-value are supported without copying ownership.
Borrowing	Borrowed() creates a non-owning wrapper from an existing id.	Use only when another owner is responsible for the underlying object.
Validity	IsValid() checks the current resource state.	Check before using a wrapper after Release() or when handling optional resources.
Manual release	Release() explicitly relinquishes an owned resource.	After Release(), the wrapper is expected to behave like an empty wrapper.

2. Core Data Structures and Callbacks

Vertex3D

Member	Declaration	Meaning
position[3]	float position[3]	Public data member used by the associated renderer/importer subsystem.
normal[3]	float normal[3]	Public data member used by the associated renderer/importer subsystem.
uv[2]	float uv[2]	Public data member used by the associated renderer/importer subsystem.
tangent[3]	float tangent[3]	Public data member used by the associated renderer/importer subsystem.
bitangent[3]	float bitangent[3]	Public data member used by the associated renderer/importer subsystem.

SkeletalVertex

Member	Declaration	Meaning
vertex	Vertex3D vertex	Public data member used by the associated renderer/importer subsystem.
boneIndices[SKELETAL_MAX_INFLUENCES]	uint boneIndices[SKELETAL_MAX_INFLUENCES]	Public data member used by the associated renderer/importer subsystem.
boneWeights[SKELETAL_MAX_INFLUENCES]	float boneWeights[SKELETAL_MAX_INFLUENCES]	Public data member used by the associated renderer/importer subsystem.

SkeletalBoneTransform

Member	Declaration	Meaning
translation[3]	float translation[3]	Translation component of the local transform.
rotation[4]	float rotation[4]	Rotation component; for skeletal transforms this is a quaternion in x,y,z,w order.
scale[3]	float scale[3]	Scale component of the local transform.

SkeletalBone

Member	Declaration	Meaning
name	const char* name	Null-terminated name identifying the imported or runtime object.
parentIndex	uint parentIndex	Index into the corresponding resource array or hierarchy.
bindTransform	SkeletalBoneTransform bindTransform	Public data member used by the associated renderer/importer subsystem.
bindWorldMatrix[16]	float bindWorldMatrix[16]	4x4 transform matrix stored as 16 contiguous floating-point values.
inverseBindMatrix[16]	float inverseBindMatrix[16]	4x4 transform matrix stored as 16 contiguous floating-point values.

SkeletalAnimation

Member	Declaration	Meaning
name	const char* name	Null-terminated name identifying the imported or runtime object.
duration	float duration	Public data member used by the associated renderer/importer subsystem.
frameRate	float frameRate	Public data member used by the associated renderer/importer subsystem.
frameCount	std::size_t frameCount	Number of entries present in the corresponding array or collection.

frames	SkeletalBoneTransform* frames	Pointer to the corresponding array or byte buffer.
poseMatrices	float* poseMatrices	Pointer to the corresponding array or byte buffer.

SkeletalMeshData

Member	Declaration	Meaning
vertices	SkeletalVertex* vertices	Pointer to the corresponding array or byte buffer.
vertexCount	std::size_t vertexCount	Number of entries present in the corresponding array or collection.
indices	uint* indices	Pointer to the corresponding array or byte buffer.
indexCount	std::size_t indexCount	Number of entries present in the corresponding array or collection.
bones	SkeletalBone* bones	Pointer to the corresponding array or byte buffer.
boneCount	std::size_t boneCount	Number of entries present in the corresponding array or collection.
animations	SkeletalAnimation* animations	Pointer to the corresponding array or byte buffer.
animationCount	std::size_t animationCount	Number of entries present in the corresponding array or collection.
geometryToWorldMatrix[16]	float geometryToWorldMatrix[16]	4x4 transform matrix stored as 16 contiguous floating-point values.

FBXTextureInfo

Member	Declaration	Meaning
name	const char* name	Null-terminated name identifying the imported or runtime object.
filename	const char* filename	Null-terminated name identifying the imported or runtime object.
data	const unsigned char* data	Pointer to the corresponding array or byte buffer.
size	std::size_t size	Public data member used by the associated renderer/importer subsystem.
type	EFBXTextureType type	Public data member used by the associated renderer/importer subsystem.
embedded	int embedded	Public data member used by the associated renderer/importer subsystem.

FBXMaterialInfo

Member	Declaration	Meaning
name	const char* name	Null-terminated name identifying the imported or runtime object.
baseColor[4]	float baseColor[4]	Public data member used by the associated renderer/importer subsystem.
roughness	float roughness	Public data member used by the associated renderer/importer subsystem.
metallic	float metallic	Public data member used by the associated renderer/importer subsystem.
specular	float specular	Public data member used by the associated renderer/importer subsystem.
opacity	float opacity	Public data member used by the associated renderer/importer subsystem.
textureIndices[HRL_FB_X_MATERIAL_TEXTURE_SLOT_COUNT]	uint textureIndices[HRL_FB_X_MATERIAL_TEXTURE_SLOT_COUNT]	Number of entries present in the corresponding array or collection.

Callback types

Name	Signature	Contract
------	-----------	----------

ErrorCallback	void(*) (EError code, ESeverity severity, const char* detail)	Called when HRL reports an internal error condition.
ButtonPressedCallback	void(*) (id button, int clicked, int released, void* userData)	Receives button click/release transitions together with user-defined callback state.
SliderChangedCallback	void(*) (id slider, float value, void* userData)	Receives the current value after a slider state change.
CheckboxChangedCallback	void(*) (id checkbox, int checked, void* userData)	Receives the checkbox state after it changes.
GizmoChangedCallback	void(*) (id gizmo, EGizmoPart part, void* userData)	Receives interaction changes for a gizmo and identifies the manipulated part.

3. Enumerations and Constants

E_APIs

Enumerator	Meaning
HRL_OPENGL_33 = 0x0001	Named HRL enumeration value.
HRL_OPENGL_45	Named HRL enumeration value.
HRL_VULKAN	Named HRL enumeration value.
HRL_D3D11	Named HRL enumeration value.
HRL_D3D12	Named HRL enumeration value.
HRL_METAL	Named HRL enumeration value.
HRL_NVN	Named HRL enumeration value.
HRL_GNM	Named HRL enumeration value.

ELightType

Enumerator	Meaning
HRL_POINT_LIGHT = 0x0011	Named HRL enumeration value.
HRL_DIRECTIONAL_LIGHT	Named HRL enumeration value.
HRL_SPOT_LIGHT	Named HRL enumeration value.
HRL_SKY_LIGHT	Named HRL enumeration value.

EMeshType

Enumerator	Meaning
HRL_SPRITE = 0x0021	Named HRL enumeration value.
HRL_2D_MESH	Named HRL enumeration value.
HRL_3D_MESH	Named HRL enumeration value.
HRL_3D_SKELETAL_MESH	Named HRL enumeration value.

EDebugRenderingType

Enumerator	Meaning
HRL_DEBUG_HOLLOW = 0x0031	Named HRL enumeration value.
HRL_DEBUG_SOLID	Named HRL enumeration value.

ECameraType

Enumerator	Meaning
HRL_ORTHO = 0x0041	Named HRL enumeration value.
HRL_PERSPECTIVE	Named HRL enumeration value.

EFilterType

Enumerator	Meaning
HRL_FILTER_NEAREST = 0x0050	Named HRL enumeration value.
HRL_FILTER_LINEAR	Named HRL enumeration value.
HRL_FILTER_BILINEAR	Named HRL enumeration value.
HRL_FILTER_TRILINEAR	Named HRL enumeration value.
HRL_FILTER_ANISOTROPIC	Named HRL enumeration value.

HRL_FILTER_SUPERSAMPLING	Supersampling filter; the source notes that it is not available with OpenGL backends.
--------------------------	---

EGlobalIlluminationMethod

Enumerator	Meaning
HRL_GI_NONE = 0	Selects the global-illumination algorithm requested for a scene.
HRL_GI_SSGI	Selects the global-illumination algorithm requested for a scene.
HRL_GI_VCT	Selects the global-illumination algorithm requested for a scene.
HRL_GI_LPV	Selects the global-illumination algorithm requested for a scene.
HRL_GI_DDGI	Selects the global-illumination algorithm requested for a scene.
HRL_GI_PATH_TRACING	Selects the global-illumination algorithm requested for a scene.
HRL_GI_RAY_TRACING	Selects the global-illumination algorithm requested for a scene.

EAntialiasingMode

Enumerator	Meaning
HRL_ANTIALIASING_OFF = 0	Named HRL enumeration value.
HRL_ANTIALIASING_2X = 2	Named HRL enumeration value.
HRL_ANTIALIASING_4X = 4	Named HRL enumeration value.
HRL_ANTIALIASING_8X = 8	Named HRL enumeration value.

EDebugView

Enumerator	Meaning
HRL_DEBUG_VIEW_NONE = 0x0060	Named HRL enumeration value.
HRL_DEBUG_VIEW_UNLIT	Named HRL enumeration value.
HRL_DEBUG_VIEW_NORMAL	Named HRL enumeration value.
HRL_DEBUG_VIEW_LIGHTS	Named HRL enumeration value.
HRL_DEBUG_VIEW_LIGHTING = HRL_DEBUG_VIEW_LIGHTS	Named HRL enumeration value.
HRL_DEBUG_VIEW_WIREFRAME	Named HRL enumeration value.
HRL_DEBUG_VIEW_LOD	Named HRL enumeration value.

ELODMode

Enumerator	Meaning
HRL_LOD_DISTANCE = 0	Selects the metric used for LOD selection.
HRL_LOD_SCREEN_SIZE	Selects the metric used for LOD selection.

EError

Enumerator	Meaning
HRL_NO_ERROR = 0x0070	Named HRL enumeration value.
HRL_ERROR_INVALID_ID	Named HRL enumeration value.
HRL_INVALID_ENUM	Named HRL enumeration value.
HRL_INVALID_VALUE	Named HRL enumeration value.
HRL_INVALID_OPERATION	Named HRL enumeration value.
HRL_INVALID_BACKEND_OPERATION	Named HRL enumeration value.
HRL_SHADER_COMPILE_FAIL	Named HRL enumeration value.
HRL_OUT_OF_MEMORY	Named HRL enumeration value.

HRL_INVALID_FILE_FORMAT	Named HRL enumeration value.
-------------------------	------------------------------

ESeverity

Enumerator	Meaning
HRL_SEVERITY_WEAK_WARNING = 0x0080	Named HRL enumeration value.
HRL_SEVERITY_WARNING	Named HRL enumeration value.
HRL_SEVERITY_ERROR	Named HRL enumeration value.
HRL_SEVERITY_FATAL	Named HRL enumeration value.

EFogType

Enumerator	Meaning
HRL_FOG_LINEAR = 0x0090	Named HRL enumeration value.
HRL_FOG_EXPONENTIAL	Named HRL enumeration value.
HRL_FOG_EXP_SQUARED	Named HRL enumeration value.

EWidgetState

Enumerator	Meaning
HRL_WIDGET_STATE_IDLE = (1 << 0)	Bit flag describing a widget interaction state.
HRL_WIDGET_STATE_HOVERED = (1 << 1)	Bit flag describing a widget interaction state.
HRL_WIDGET_STATE_PRESSED = (1 << 2)	Bit flag describing a widget interaction state.

EWidgetType

Enumerator	Meaning
HRL_WIDGET_BUTTON = 0x00A0	Named HRL enumeration value.
HRL_WIDGET_LABEL	Named HRL enumeration value.
HRL_WIDGET_IMAGE	Named HRL enumeration value.
HRL_WIDGET_SLIDER	Named HRL enumeration value.
HRL_WIDGET_CHECKBOX	Named HRL enumeration value.
HRL_WIDGET_PROGRESSBAR	Named HRL enumeration value.

EGizmoAxis

Enumerator	Meaning
HRL_GIZMO_AXIS_NONE = 0	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_AXIS_X = 1 << 0	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_AXIS_Y = 1 << 1	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_AXIS_Z = 1 << 2	Identifies a gizmo axis, mode, space, part, or active operation.

EGizmoMode

Enumerator	Meaning
HRL_GIZMO_MODE_TRANSLATE = 1 << 0	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_MODE_ROTATE = 1 << 1	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_MODE_SCALE = 1 << 2	Identifies a gizmo axis, mode, space, part, or active operation.

EGizmoSpace

Enumerator	Meaning
------------	---------

HRL_GIZMO_SPACE_WORLD = 0	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_SPACE_LOCAL	Identifies a gizmo axis, mode, space, part, or active operation.

EGizmoPart

Enumerator	Meaning
HRL_GIZMO_PART_NONE = 0	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_PART_X	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_PART_Y	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_PART_Z	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_PART_CENTER	Identifies a gizmo axis, mode, space, part, or active operation.

EGizmoOperation

Enumerator	Meaning
HRL_GIZMO_OPERATION_NONE = 0	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_OPERATION_TRANSLATE = HRL_GIZMO_MODE_TRANSLATE	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_OPERATION_ROTATE = HRL_GIZMO_MODE_ROTATE	Identifies a gizmo axis, mode, space, part, or active operation.
HRL_GIZMO_OPERATION_SCALE = HRL_GIZMO_MODE_SCALE	Identifies a gizmo axis, mode, space, part, or active operation.

EVFXSimulationSpace

Enumerator	Meaning
HRL_VFX_SIMULATION_LOCAL = 0	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_SIMULATION_WORLD	Selects the particle simulation, render, blend, or spawn configuration.

EVFXRenderMode

Enumerator	Meaning
HRL_VFX_RENDER_BILLBOARD = 0	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_RENDER_STRETCHED_BILLBOARD	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_RENDER_MESH	Selects the particle simulation, render, blend, or spawn configuration.

EVFXBlendMode

Enumerator	Meaning
HRL_VFX_BLEND_ALPHA = 0	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_BLEND_ADDITIVE	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_BLEND_MULTIPLY	Selects the particle simulation, render, blend, or spawn configuration.

EVFXSpawnShape

Enumerator	Meaning
HRL_VFX_SHAPE_POINT = 0	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_SHAPE_SPHERE	Selects the particle simulation, render, blend, or spawn configuration.

HRL_VFX_SHAPE_BOX	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_SHAPE_CYLINDER	Selects the particle simulation, render, blend, or spawn configuration.
HRL_VFX_SHAPE_CONE	Selects the particle simulation, render, blend, or spawn configuration.

ESliderOrientation

Enumerator	Meaning
HRL_SLIDER_HORIZONTAL = 0	Named HRL enumeration value.
HRL_SLIDER_VERTICAL	Named HRL enumeration value.

EFBXTextureType

Enumerator	Meaning
HRL_FBX_TEXTURE_FILE = 0	Identifies how an imported FBX texture is represented.
HRL_FBX_TEXTURE_LAYERED	Identifies how an imported FBX texture is represented.
HRL_FBX_TEXTURE_PROCEDURAL	Identifies how an imported FBX texture is represented.
HRL_FBX_TEXTURE_SHADER	Identifies how an imported FBX texture is represented.

EFBXMaterialTextureSlot

Enumerator	Meaning
HRL_FBX_MATERIAL_ALBEDO = 0	Identifies a material texture channel in the built-in FBX material mapping.
HRL_FBX_MATERIAL_NORMAL	Identifies a material texture channel in the built-in FBX material mapping.
HRL_FBX_MATERIAL_SPECULAR	Identifies a material texture channel in the built-in FBX material mapping.
HRL_FBX_MATERIAL_ROUGHNESS	Identifies a material texture channel in the built-in FBX material mapping.
HRL_FBX_MATERIAL_METALLIC	Identifies a material texture channel in the built-in FBX material mapping.
HRL_FBX_MATERIAL_ALPHA	Identifies a material texture channel in the built-in FBX material mapping.
HRL_FBX_MATERIAL_TEXTURE_SLOT_COUNT	Identifies a material texture channel in the built-in FBX material mapping.

Inline constants

Name	Type	Value	Role
API_VERSION	const char*	"0.6"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
INVALID_ID	id	static_cast<id>(-1)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
SKELETAL_MAX_INFLUENCES	uint	4	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.

MAX_SKELETAL_BONES	uint	128	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MAX_VOLUMETRIC_FOGS	uint	64	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
SPRITE_SHADER	id	static_cast<id>(0xFFFFFFFFu)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MESH_2D_SHADER	id	static_cast<id>(0xFFFFFFFFEu)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MESH_3D_SHADER	id	static_cast<id>(0xFFFFFFFFDu)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
DEBUG_SHADER	id	static_cast<id>(0xFFFFFFFFCu)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
DEFAULT_POST_PROCESS_SHADER	id	static_cast<id>(0xFFFFFFFFBu)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
SKINNED_3D_MESH_SHADER	id	static_cast<id>(0xFFFFFFFFAu)	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
FALSE_VALUE	int	0	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
TRUE_VALUE	int	1	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_ALBEDO	const char*	"T_Albedo"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_NORMAL	const char*	"T_Normal"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_SPECULAR	const char*	"T_Specular"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_ROUGHNESS	const char*	"T_Roughness"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.

T_METALLIC	const char*	"T_Metallic"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_AMBIENT_OCCLUSION	const char*	"T_AO"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_ALPHA	const char*	"T_Alpha"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_EMISSIVE	const char*	"T_Emissive"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_SHADOW_MAP	const char*	"T_ShadowMap"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
T_CUBE_MAP	const char*	"T_CubeMap"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MATERIAL_PARAM_TWO_SIDED	const char*	"TwoSided"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MATERIAL_TEXTURE_SS_DISPLACEMENT_MAPPING	const char*	"SS_DISPLACEMENT_MAPPING"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MATERIAL_PARAM_SS_DISPLACEMENT_ENABLED	const char*	"ss_displacement_enabled"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MATERIAL_PARAM_SS_DISPLACEMENT_STRENGTH	const char*	"SSDisplacementStrength"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MATERIAL_PARAM_SS_DISPLACEMENT_SCALE	const char*	"SSDisplacementScale"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MATERIAL_PARAM_SS_DISPLACEMENT_OPACITY	const char*	"SSDisplacementOpacity"	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MOUSE_BUTTON_LEFT	int	0	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MOUSE_BUTTON_RIGHT	int	1	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.

MOUSE_BUTTON_MIDDLE	int	2	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MOUSE_RELEASE	int	0	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.
MOUSE_PRESS	int	1	Public C++ constant used as a sentinel, renderer limit, built-in shader identifier, input value, or material/texture semantic.

4. Resource Classes

Each class entry below follows the declaration order of `hrl.hpp`. Every public callable declaration is listed individually. Where the source comment is sparse, the contract is expanded from the declaration, enum values, resource relationships, and the corresponding C API concepts.

Instance

Global renderer context owner. Constructing an Instance selects and initializes the backend; the object controls frame lifecycle and global error reporting.

Instance::Instance

```
Instance(E_APIs api, uint width, uint height, void* loader);
```

Selects the graphics backend to use and initializes the rendering context.

Parameter	Type	Meaning
api	E_APIs	Value supplied to configure the api property or operation.
width	uint	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	uint	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
loader	void*	Value supplied to configure the loader property or operation.

Instance::~Instance

```
Instance(const Instance&) = delete;
```

Releases the rendering context.

Instance::Instance&

```
Instance& operator=(const Instance&) = delete;
```

Copy assignment is disabled for this resource wrapper.

Instance::Instance

```
Instance(Instance&&) = delete;
```

Move construction is disabled for this resource wrapper.

Instance::Instance&

```
Instance& operator=(Instance&&) = delete;
```

Move assignment is disabled for this resource wrapper.

Instance::BeginFrame

```
void BeginFrame();
```

Prepares the renderer for a new frame.

Instance::EndFrame

```
void EndFrame();
```

Finalizes and submits the current frame.

Instance::WindowResizeCallback

```
void WindowResizeCallback(int width, int height);
```

Notifies HRL of a window or framebuffer resize.

Parameter	Type	Meaning
width	int	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	int	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Instance::GetLastError

```
EError GetLastError(const char** detail = nullptr, ESeverity* severity = nullptr) const;
```

Retrieves the last recorded error, if any.

Parameter	Type	Meaning
detail	const char**	Value supplied to configure the detail property or operation.
severity	ESeverity*	Value supplied to configure the severity property or operation.

Instance::ErrorEnumToString

```
std::string ErrorEnumToString(EError err) const;
```

Converts an error enum value to its string representation.

Parameter	Type	Meaning
err	EError	Value supplied to configure the err property or operation.

Return: std::string. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Instance::SeverityEnumToString

```
std::string SeverityEnumToString(ESeverity severity) const;
```

Converts an error severity enum value to its string representation.

Parameter	Type	Meaning
severity	ESeverity	Value supplied to configure the severity property or operation.

Return: std::string. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Instance::RegisterErrorCallback

```
void RegisterErrorCallback(ErrorCallback callback);
```

Registers a callback invoked whenever an error is raised internally.

Parameter	Type	Meaning
callback	ErrorCallback	Function pointer invoked by the corresponding event or state transition.

Mesh

Base wrapper shared by renderable mesh resources. Concrete mesh types add specialized geometry, skinning, LOD, or sprite operations.

Mesh::Borrowed

```
static Mesh Borrowed(id value);
```

Creates a non-owning C++ wrapper around an existing mesh ID.

Parameter	Type	Meaning
value	id	Reference or value identifying the associated HRL resource or wrapper object.

Mesh::~Mesh

```
virtual ~Mesh();
```

Virtual destructor for the common mesh wrapper.

Mesh::Mesh

```
Mesh(const Mesh&) = delete;
```

Copy construction is disabled for this resource wrapper.

Mesh::Mesh&

```
Mesh& operator=(const Mesh&) = delete;
```

Copy assignment is disabled for this resource wrapper.

Mesh::Mesh

```
Mesh(Mesh&& other) noexcept;
```

Move-constructs a mesh wrapper.

Parameter	Type	Meaning
other	Mesh&&	Reference or value identifying the associated HRL resource or wrapper object.

Mesh::Mesh&

```
Mesh& operator=(Mesh&& other) noexcept;
```

Move-assigns a mesh wrapper.

Parameter	Type	Meaning
other	Mesh&&	Reference or value identifying the associated HRL resource or wrapper object.

Mesh::IsValid

```
bool IsValid() const;
```

Returns the underlying HRL mesh ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Mesh::Release

```
void Release();
```

Destroys the owned mesh and releases its associated GPU resources.

Mesh::SetPivotPoint

```
void SetPivotPoint(float x, float y, float z);
```

Sets the pivot (origin) point of a mesh or sprite.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Mesh::SetMaterial

```
void SetMaterial(const Material& material);
```

Assigns a material to a mesh.

Parameter	Type	Meaning
material	const Material&	Reference or value identifying the associated HRL resource or wrapper object.

Mesh::SetLocation

```
void SetLocation(float x, float y, float z);
```

Sets the world-space position of a mesh.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Mesh::SetRotation

```
void SetRotation(float pitch, float yaw, float roll);
```

Sets the rotation of a mesh using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

Mesh::SetScale

```
void SetScale(float x, float y, float z);
```

Sets the scale of a mesh along each local axis.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Mesh3D

3D mesh base class adding camera distance and level-of-detail control to Mesh-derived resources.

Mesh3D::Borrowed

```
static Mesh3D Borrowed(id value);
```

Creates a non-owning wrapper around an existing 3D mesh ID.

Parameter	Type	Meaning
value	id	Reference or value identifying the associated HRL resource or wrapper object.

Mesh3D::GetCameraDistance

```
float GetCameraDistance() const;
```

Returns the minimum world-space distance from the mesh LOD center to any viewport camera.

Return: float. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Mesh3D::SetLODAutomatic

```
void SetLODAutomatic(bool enabled);
```

Enables or disables automatic LOD selection for a 3D mesh.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Mesh3D::SetLODMode

```
void SetLODMode(ELODMode mode);
```

Selects distance-based or projected screen-size LOD selection.

Parameter	Type	Meaning
mode	ELODMode	Value supplied to configure the mode property or operation.

Mesh3D::SetLODLevels

```
void SetLODLevels(uint levels);
```

Sets the number of LOD levels, including LOD 0.

Parameter	Type	Meaning
levels	uint	Index/count selector for an element or quality/iteration level in the associated collection.

Mesh3D::SetLODDistance

```
void SetLODDistance(float baseDistance);
```

Sets the first transition distance in world units.

Parameter	Type	Meaning
baseDistance	float	Value supplied to configure the baseDistance property or operation.

Mesh3D::SetLODScale

```
void SetLODScale(float distanceScale);
```

Multiplies the distance threshold for each next LOD.

Parameter	Type	Meaning
distanceScale	float	Value supplied to configure the distanceScale property or operation.

Mesh3D::SetLODMinDistance

```
void SetLODMinDistance(float distance);
```

Sets the minimum distance used by distance-based LOD selection.

Parameter	Type	Meaning
distance	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Mesh3D::SetLODMaxDistance

```
void SetLODMaxDistance(float distance);
```

Sets the maximum distance used by distance-based LOD selection.

Parameter	Type	Meaning
distance	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Mesh3D::SetLODScreenThreshold

```
void SetLODScreenThreshold(float threshold);
```

Sets the LOD 0 screen-height threshold.

Parameter	Type	Meaning
threshold	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Mesh3D::SetLODScreenScale

```
void SetLODScreenScale(float scale);
```

Multiplies the screen-size threshold for each next LOD.

Parameter	Type	Meaning
scale	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Mesh3D::SetLODHysteresis

```
void SetLODHysteresis(float hysteresis);
```

Adds transition hysteresis to reduce LOD popping.

Parameter	Type	Meaning
hysteresis	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Mesh3D::SetLODOVERRIDE

```
void SetLODOVERRIDE(int level);
```

Forces a fixed LOD level.

Parameter	Type	Meaning
-----------	------	---------

level	int	Index/count selector for an element or quality/iteration level in the associated collection.
-------	-----	--

Mesh3D::ForceLODRebuild

```
void ForceLODRebuild();
```

Regenerates the internal LOD geometry from LOD 0.

Mesh3D::GetLODCount

```
uint GetLODCount() const;
```

Returns the number of available LOD levels.

Return: uint. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Mesh3D::GetLODVertexCount

```
std::size_t GetLODVertexCount(uint level) const;
```

Returns the vertex count of a specific LOD level.

Parameter	Type	Meaning
level	uint	Index/count selector for an element or quality/iteration level in the associated collection.

Return: std::size_t. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Mesh3D::GetLODTriangleCount

```
std::size_t GetLODTriangleCount(uint level) const;
```

Returns the triangle count of a specific LOD level.

Parameter	Type	Meaning
level	uint	Index/count selector for an element or quality/iteration level in the associated collection.

Return: std::size_t. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Mesh3D::GetLODLevel

```
int GetLODLevel() const;
```

Returns the LOD chosen by the most recently rendered viewport.

Return: int. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

StaticMesh

Non-skinned 3D geometry resource created from an array of Vertex3D values and optional triangle indices.

StaticMesh::StaticMesh

```
explicit StaticMesh( Scene& scene, const Vertex3D* vertices, std::size_t vertexCount, const uint* indices = nullptr,
std::size_t indexCount = 0);
```

Creates a static 3D mesh from caller-owned vertex and index arrays.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
vertices	const Vertex3D*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
vertexCount	std::size_t	Value supplied to configure the vertexCount property or operation.
indices	const uint*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
indexCount	std::size_t	Value supplied to configure the indexCount property or operation.

StaticMesh::Borrowed

```
static StaticMesh Borrowed(id value);
```

Creates a non-owning wrapper around an existing static mesh ID.

Parameter	Type	Meaning
value	id	Reference or value identifying the associated HRL resource or wrapper object.

SkeletalMesh

Skinned 3D mesh with bone metadata, baked animation data, and runtime playback controls.

SkeletalMesh::SkeletalMesh

```
explicit SkeletalMesh(Scene& scene, const SkeletalMeshData& data);
```

Creates a skeletal mesh from CPU-side skeletal data.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
data	const SkeletalMeshData&	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.

SkeletalMesh::Borrowed

```
static SkeletalMesh Borrowed(id value);
```

Creates a non-owning wrapper around an existing skeletal mesh ID.

Parameter	Type	Meaning
value	id	Reference or value identifying the associated HRL resource or wrapper object.

SkeletalMesh::GetSkeletalBoneCount

```
uint GetSkeletalBoneCount() const;
```

Returns the number of skeletal bones.

Return: uint. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

SkeletalMesh::GetSkeletalBone

```
const SkeletalBone* GetSkeletalBone(uint index) const;
```

Returns read-only skeletal bone metadata.

Parameter	Type	Meaning
index	uint	Index/count selector for an element or quality/iteration level in the associated collection.

SkeletalMesh::FindSkeletalBone

```
uint FindSkeletalBone(const char* name) const;
```

Finds a skeletal bone by name.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.

Return: uint. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

SkeletalMesh::GetSkeletalAnimationCount

```
uint GetSkeletalAnimationCount() const;
```

Returns the number of skeletal animations.

Return: uint. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

SkeletalMesh::GetSkeletalAnimation

```
const SkeletalAnimation* GetSkeletalAnimation(uint index) const;
```

Returns read-only skeletal animation metadata.

Parameter	Type	Meaning
index	uint	Index/count selector for an element or quality/iteration level in the associated collection.

SkeletalMesh::FindSkeletalAnimation

```
uint FindSkeletalAnimation(const char* name) const;
```

Finds a skeletal animation by name.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.

Return: uint. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

SkeletalMesh::PlaySkeletalAnimation

```
void PlaySkeletalAnimation(uint animation);
```

Starts playback of a skeletal animation by index.

Parameter	Type	Meaning
animation	uint	Index/count selector for an element or quality/iteration level in the associated collection.

SkeletalMesh::StopSkeletalAnimation

```
void StopSkeletalAnimation();
```

Stops skeletal animation playback.

SkeletalMesh::SetSkeletalAnimationTime

```
void SetSkeletalAnimationTime(float time);
```

Sets the current skeletal animation time.

Parameter	Type	Meaning
time	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

SkeletalMesh::SetSkeletalAnimationSpeed

```
void SetSkeletalAnimationSpeed(float speed);
```

Sets skeletal animation playback speed.

Parameter	Type	Meaning
speed	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

SkeletalMesh::SetSkeletalAnimationLoop

```
void SetSkeletalAnimationLoop(bool loop);
```

Enables or disables skeletal animation looping.

Parameter	Type	Meaning
-----------	------	---------

loop	bool	Boolean state controlling whether the associated feature is enabled or active.
------	------	--

SkeletalMesh::GetCurrentSkeletalAnimation

```
int GetCurrentSkeletalAnimation() const;
```

Returns the current skeletal animation index.

Return: int. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

SkeletalMesh::GetSkeletalAnimationTime

```
float GetSkeletalAnimationTime() const;
```

Returns the current skeletal animation time.

Return: float. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

SkeletalMesh::IsSkeletalAnimationPlaying

```
bool IsSkeletalAnimationPlaying() const;
```

Returns whether skeletal animation is currently playing.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Sprite

Camera-facing 2D mesh with atlas-region and draw-order controls.

Sprite::Sprite

```
explicit Sprite(Scene& scene);
```

Creates a sprite mesh in a scene.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.

Sprite::Borrowed

```
static Sprite Borrowed(id value);
```

Creates a non-owning wrapper around an existing sprite ID.

Parameter	Type	Meaning
value	id	Reference or value identifying the associated HRL resource or wrapper object.

Sprite::SetSpriteRegion

```
void SetSpriteRegion(float minU, float minV, float maxU, float maxV);
```

Defines the UV region of the texture displayed on the sprite.

Parameter	Type	Meaning
minU	float	Value supplied to configure the minU property or operation.
minV	float	Value supplied to configure the minV property or operation.
maxU	float	Value supplied to configure the maxU property or operation.
maxV	float	Value supplied to configure the maxV property or operation.

Sprite::SetDrawOrder

```
void SetDrawOrder(float drawOrder);
```

Controls the rendering order of a sprite on the Z axis.

Parameter	Type	Meaning
drawOrder	float	Value supplied to configure the drawOrder property or operation.

Sprite::SetSpriteDrawOrder

```
void SetSpriteDrawOrder(float drawOrder);
```

Alias for SetDrawOrder().

Parameter	Type	Meaning
drawOrder	float	Value supplied to configure the drawOrder property or operation.

Light

Scene light resource supporting point, directional, spot, and other enum-defined light modes plus shadow controls.

Light::Light

```
explicit Light(Scene& scene, ELightType type);
```

Creates a light source in a scene.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
type	ELightType	Value supplied to configure the type property or operation.

Light::Light

```
Light(const Light&) = delete;
```

Destroys the owned light.

Light::Light&

```
Light& operator=(const Light&) = delete;
```

Copy assignment is disabled for this resource wrapper.

Light::Light

```
Light(Light&& other) noexcept;
```

Move-constructs a light wrapper.

Parameter	Type	Meaning
other	Light&&	Reference or value identifying the associated HRL resource or wrapper object.

Light::Light&

```
Light& operator=(Light&& other) noexcept;
```

Move-assigns a light wrapper.

Parameter	Type	Meaning
other	Light&&	Reference or value identifying the associated HRL resource or wrapper object.

Light::IsValid

```
bool IsValid() const;
```

Returns the underlying light ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Light::Release

```
void Release();
```

Destroys the owned light.

Light::SetColor

```
void SetColor(float x, float y, float z);
```

Sets the RGB color emitted by a light.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Light::SetIntensity

```
void SetIntensity(float intensity);
```

Sets the intensity (brightness multiplier) of a light.

Parameter	Type	Meaning
intensity	float	Value supplied to configure the intensity property or operation.

Light::SetAttenuation

```
void SetAttenuation(float attenuation);
```

Sets the attenuation (falloff) factor of a light.

Parameter	Type	Meaning
attenuation	float	Value supplied to configure the attenuation property or operation.

Light::SetLocation

```
void SetLocation(float x, float y, float z);
```

Sets the world-space position of a light.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Light::SetRotation

```
void SetRotation(float pitch, float yaw, float roll);
```

Sets the orientation of a light using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

Light::SetCastShadows

```
void SetCastShadows(bool enabled);
```

Enables or disables shadow casting for a light.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Light::SetShadowBias

```
void SetShadowBias(float bias);
```

Sets the shadow bias used to reduce self-shadowing artifacts.

Parameter	Type	Meaning
bias	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Light::SetShadowStrength

```
void SetShadowStrength(float strength);
```

Controls how dark a cast shadow is for this light.

Parameter	Type	Meaning
strength	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Light::SetShadowResolution

```
void SetShadowResolution(int resolution);
```

Sets the resolution of the private shadow map for a light.

Parameter	Type	Meaning
resolution	int	Value supplied to configure the resolution property or operation.

Light::SetSpotLightInnerCutoff

```
void SetSpotLightInnerCutoff(float degrees);
```

Sets the inner cutoff angle of a spot light.

Parameter	Type	Meaning
degrees	float	Value supplied to configure the degrees property or operation.

Light::SetSpotLightOuterCutoff

```
void SetSpotLightOuterCutoff(float degrees);
```

Sets the outer cutoff angle of a spot light.

Parameter	Type	Meaning
degrees	float	Value supplied to configure the degrees property or operation.

Texture

GPU texture resource created from encoded image bytes or text rasterization; also exposes asynchronous creation and sampler filtering.

Texture::Texture

```
Texture(const void* data, std::size_t size);
```

Creates a GPU texture from a raw file buffer.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

Texture::CreateAsync

```
static Texture CreateAsync(const void* data, std::size_t size);
```

Starts an asynchronous texture load from a raw file buffer.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

Texture::Texture

```
Texture(const char* text, const Font& font, float fontSize, float wrapWidth, float r, float g, float b, float bgR, float bgG, float bgB, float bgA);
```

Rasterizes a UTF-8 text string into a new texture using a font.

Parameter	Type	Meaning
text	const char*	Value supplied to configure the text property or operation.
font	const Font&	Reference or value identifying the associated HRL resource or wrapper object.
fontSize	float	Value supplied to configure the fontSize property or operation.
wrapWidth	float	Value supplied to configure the wrapWidth property or operation.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
bgR	float	Value supplied to configure the bgR property or operation.
bgG	float	Value supplied to configure the bgG property or operation.
bgB	float	Value supplied to configure the bgB property or operation.
bgA	float	Value supplied to configure the bgA property or operation.

Texture::Texture

```
Texture(const Texture&) = delete;
```

Destroys the texture.

Texture::Texture&

```
Texture& operator=(const Texture&) = delete;
```

Copy assignment is disabled for this resource wrapper.

Texture::Texture

```
Texture(Texture&& other) noexcept;
```

Move-constructs a texture wrapper.

Parameter	Type	Meaning
other	Texture&&	Reference or value identifying the associated HRL resource or wrapper object.

Texture::Texture&

```
Texture& operator=(Texture&& other) noexcept;
```

Move-assigns a texture wrapper.

Parameter	Type	Meaning
other	Texture&&	Reference or value identifying the associated HRL resource or wrapper object.

Texture::IsValid

```
bool IsValid() const;
```

Returns the underlying texture ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Texture::Release

```
void Release();
```

Destroys the owned texture and releases its GPU memory.

Texture::Reload

```
void Reload(const void* data, std::size_t size);
```

Starts or performs a texture data reload from a new file buffer.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

Texture::GetSize

```
void GetSize(int& width, int& height) const;
```

Returns the current texture dimensions in pixels.

Parameter	Type	Meaning
width	int&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	int&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Texture::SetMinFilter

```
void SetMinFilter(EFilterType filter);
```

Sets the minification filter used when the texture appears smaller than its native size.

Parameter	Type	Meaning
filter	EFilterType	Value supplied to configure the filter property or operation.

Texture::SetMagFilter

```
void SetMagFilter(EFilterType filter);
```

Sets the magnification filter used when the texture appears larger than its native size.

Parameter	Type	Meaning
filter	EFilterType	Value supplied to configure the filter property or operation.

Texture::IsReady

```
bool IsReady() const;
```

Returns whether an asynchronous texture has finished decoding and GPU upload.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Texture::Wait

```
void Wait();
```

Blocks until an asynchronous texture is fully uploaded.

Scene

Top-level rendering scene containing meshes, lights, cameras, effects, picking, environment settings, and post-processing inputs.

Scene::Scene

```
explicit Scene(bool renderOnScreen);
```

Creates a scene.

Parameter	Type	Meaning
renderOnScreen	bool	Value supplied to configure the renderOnScreen property or operation.

Scene::~Scene

```
Scene(const Scene&) = delete;
```

Destroys the scene and its owned objects.

Scene::Scene&

```
Scene& operator=(const Scene&) = delete;
```

Copy assignment is disabled for this resource wrapper.

Scene::Scene

```
Scene(Scene&& other) noexcept;
```

Move-constructs a scene wrapper.

Parameter	Type	Meaning
other	Scene&&	Reference or value identifying the associated HRL resource or wrapper object.

Scene::Scene&

```
Scene& operator=(Scene&& other) noexcept;
```

Move-assigns a scene wrapper.

Parameter	Type	Meaning
other	Scene&&	Reference or value identifying the associated HRL resource or wrapper object.

Scene::IsValid

```
bool IsValid() const;
```

Returns the underlying scene ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Scene::Release

```
void Release();
```

Destroys the scene and releases its resources.

Scene::SetGlobalIlluminationEnabled

```
void SetGlobalIlluminationEnabled(bool enabled);
```

Enables or disables global illumination for the scene.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetGlobalIlluminationMethod

```
void SetGlobalIlluminationMethod(EGlobalIlluminationMethod method);
```

Selects the global illumination method.

Parameter	Type	Meaning
method	EGlobalIlluminationMethod	Value supplied to configure the method property or operation.

Scene::IsGlobalIlluminationEnabled

```
bool IsGlobalIlluminationEnabled() const;
```

Returns the scene global illumination enabled state.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Scene::GetGlobalIlluminationMethod

```
EGlobalIlluminationMethod GetGlobalIlluminationMethod() const;
```

Returns the scene's selected global illumination method.

Scene::ResizeTexture

```
void ResizeTexture(int width, int height);
```

Resizes the off-screen render texture of the scene.

Parameter	Type	Meaning
width	int	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	int	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Scene::SetSkySphereEnabled

```
void SetSkySphereEnabled(bool enabled);
```

Enables or disables the procedural sky sphere for the scene.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetSkySphereColors

```
void SetSkySphereColors(float topR, float topG, float topB, float horizonR, float horizonG, float horizonB, float bottomR, float bottomG, float bottomB);
```

Sets the top, horizon and bottom colors of the procedural sky.

Parameter	Type	Meaning
-----------	------	---------

topR	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
topG	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
topB	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
horizonR	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
horizonG	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
horizonB	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
bottomR	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
bottomG	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
bottomB	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Scene::SetSkySphereRotation

```
void SetSkySphereRotation(float pitch, float yaw, float roll);
```

Rotates the sky sphere using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

Scene::SetSkySphereTexture

```
void SetSkySphereTexture(const Texture* texture);
```

Uses an equirectangular 2D texture as the sky-sphere image.

Parameter	Type	Meaning
texture	const Texture*	Reference or value identifying the associated HRL resource or wrapper object.

Scene::SetEnvironmentMappingEnabled

```
void SetEnvironmentMappingEnabled(bool enabled);
```

Enables or disables equirectangular environment mapping for 3D materials.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetEnvironmentMap

```
void SetEnvironmentMap(const Texture* texture);
```

Sets an equirectangular texture used for environment reflections.

Parameter	Type	Meaning
texture	const Texture*	Reference or value identifying the associated HRL resource or wrapper object.

Scene::EnableColorPickingBuffer

```
void EnableColorPickingBuffer(bool enabled);
```

Enables or disables the GPU color-picking buffer for the scene.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::GetHoveredObject

```
Mesh GetHoveredObject(int mouseX, int mouseY, EMeshType* meshType = nullptr);
```

Returns the mesh hovered at the given window-relative mouse coordinates.

Parameter	Type	Meaning
mouseX	int	Value supplied to configure the mouseX property or operation.
mouseY	int	Value supplied to configure the mouseY property or operation.
meshType	EMeshType*	Value supplied to configure the meshType property or operation.

Scene::SetFogEnabled

```
void SetFogEnabled(bool enabled);
```

Enables or disables the fog effect for the scene.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetFogMode

```
void SetFogMode(EFogType mode);
```

Sets the fog blending mode for the scene.

Parameter	Type	Meaning
mode	EFogType	Value supplied to configure the mode property or operation.

Scene::SetFogColor

```
void SetFogColor(float r, float g, float b);
```

Sets the fog color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Scene::SetFogDensity

```
void SetFogDensity(float density);
```

Sets the fog density for exponential fog modes.

Parameter	Type	Meaning
density	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetFogLinearRange

```
void SetFogLinearRange(float start, float end);
```

Sets the start and end distances for linear fog.

Parameter	Type	Meaning
start	float	Value supplied to configure the start property or operation.
end	float	Value supplied to configure the end property or operation.

Scene::CreateVolumetricFog

```
VolumetricFog CreateVolumetricFog();
```

Creates a localized volumetric fog volume attached to the scene.

Scene::SetGlobalVolumetricFogEnabled

```
void SetGlobalVolumetricFogEnabled(bool enabled);
```

Enables or disables scene-wide volumetric fog.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetGlobalVolumetricFogDensity

```
void SetGlobalVolumetricFogDensity(float density);
```

Sets the density of the scene-wide volumetric fog.

Parameter	Type	Meaning
density	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetGlobalVolumetricFogColor

```
void SetGlobalVolumetricFogColor(float r, float g, float b);
```

Sets the color of the scene-wide volumetric fog.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Scene::SetGlobalVolumetricFogSteps

```
void SetGlobalVolumetricFogSteps(uint steps);
```

Sets the ray-march sample count for scene-wide volumetric fog.

Parameter	Type	Meaning
steps	uint	Value supplied to configure the steps property or operation.

Scene::SetGodRaysEnabled

```
void SetGodRaysEnabled(bool enabled);
```

Enables or disables screen-space god rays.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetGodRaysPosition

```
void SetGodRaysPosition(float x, float y, float z);
```

Sets the world-space position from which god rays radiate.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Scene::SetGodRaysColor

```
void SetGodRaysColor(float r, float g, float b);
```

Sets the god-ray color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Scene::SetGodRaysDensity

```
void SetGodRaysDensity(float density);
```

Sets radial sampling density for god rays.

Parameter	Type	Meaning
density	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetGodRaysDecay

```
void SetGodRaysDecay(float decay);
```

Sets per-sample decay for god rays.

Parameter	Type	Meaning
decay	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetGodRaysWeight

```
void SetGodRaysWeight(float weight);
```

Sets the contribution weight for god rays.

Parameter	Type	Meaning
weight	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetGodRaysSamples

```
void SetGodRaysSamples(uint samples);
```

Sets the radial sample count for god rays.

Parameter	Type	Meaning
samples	uint	Index/count selector for an element or quality/iteration level in the associated collection.

Scene::SetAmbientOcclusionEnabled

```
void SetAmbientOcclusionEnabled(bool enabled);
```

Enables or disables screen-space ambient occlusion.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Scene::SetAmbientOcclusionStrength

```
void SetAmbientOcclusionStrength(float strength);
```

Sets the ambient occlusion strength.

Parameter	Type	Meaning
strength	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetAmbientOcclusionRadius

```
void SetAmbientOcclusionRadius(float radius);
```

Sets the ambient occlusion radius.

Parameter	Type	Meaning
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

Scene::SetAmbientOcclusionBias

```
void SetAmbientOcclusionBias(float bias);
```

Sets the ambient occlusion bias.

Parameter	Type	Meaning
-----------	------	---------

bias	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
------	-------	---

Scene::SetAmbientOcclusionPower

```
void SetAmbientOcclusionPower(float power);
```

Sets the ambient occlusion power.

Parameter	Type	Meaning
power	float	Value supplied to configure the power property or operation.

Scene::DrawAsDebugMode

```
void DrawAsDebugMode(EDebugView mode);
```

Overrides scene rendering with a diagnostic visualization mode.

Parameter	Type	Meaning
mode	EDebugView	Value supplied to configure the mode property or operation.

Scene::TakeScreenshot

```
void TakeScreenshot(const char* path) const;
```

Captures the rendered output of the scene and saves it as a PNG file.

Parameter	Type	Meaning
path	const char*	Value supplied to configure the path property or operation.

PostProcess

Post-processing resource attached to a viewport with an explicit material and priority.

PostProcess::PostProcess

```
explicit PostProcess(Viewport& viewport, const Material& material, int priority);
```

Creates and attaches a post-process pass to a viewport.

Parameter	Type	Meaning
viewport	Viewport&	Reference or value identifying the associated HRL resource or wrapper object.
material	const Material&	Reference or value identifying the associated HRL resource or wrapper object.
priority	int	Value supplied to configure the priority property or operation.

PostProcess::PostProcess

```
PostProcess(const PostProcess&) = delete;
```

Destroys the post-process pass.

PostProcess::PostProcess&

```
PostProcess& operator=(const PostProcess&) = delete;
```

Copy assignment is disabled.

PostProcess::PostProcess

```
PostProcess(PostProcess&& other) noexcept;
```

Move-constructs a post-process wrapper.

Parameter	Type	Meaning
other	PostProcess&&	Reference or value identifying the associated HRL resource or wrapper object.

PostProcess::PostProcess&

```
PostProcess& operator=(PostProcess&& other) noexcept;
```

Move-assigns a post-process wrapper.

Parameter	Type	Meaning
other	PostProcess&&	Reference or value identifying the associated HRL resource or wrapper object.

PostProcess::IsValid

```
bool IsValid() const;
```

Returns the underlying post-process ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

PostProcess::Release

```
void Release();
```

Destroys the owned post-process pass.

Shader

GPU shader program wrapper supporting synchronous or asynchronous compilation from vertex and fragment source/data.

Shader::Shader

```
Shader( const void* vertexData, std::size_t vertexSize, const void* fragmentData, std::size_t fragmentSize);
```

Compiles and links a shader from vertex and fragment source buffers.

Parameter	Type	Meaning
vertexData	const void*	Value supplied to configure the vertexData property or operation.
vertexSize	std::size_t	Value supplied to configure the vertexSize property or operation.
fragmentData	const void*	Value supplied to configure the fragmentData property or operation.
fragmentSize	std::size_t	Value supplied to configure the fragmentSize property or operation.

Shader::CreateAsync

```
static Shader CreateAsync( const void* vertexData, std::size_t vertexSize, const void* fragmentData, std::size_t fragmentSize);
```

Starts asynchronous shader source loading.

Parameter	Type	Meaning
vertexData	const void*	Value supplied to configure the vertexData property or operation.
vertexSize	std::size_t	Value supplied to configure the vertexSize property or operation.
fragmentData	const void*	Value supplied to configure the fragmentData property or operation.
fragmentSize	std::size_t	Value supplied to configure the fragmentSize property or operation.

Shader::Shader

```
Shader(const Shader&) = delete;
```

Destroys the shader program and frees its GPU resources.

Shader::Shader&

```
Shader& operator=(const Shader&) = delete;
```

Copy assignment is disabled.

Shader::Shader

```
Shader(Shader&& other) noexcept;
```

Move-constructs a shader wrapper.

Parameter	Type	Meaning
other	Shader&&	Reference or value identifying the associated HRL resource or wrapper object.

Shader::Shader&

```
Shader& operator=(Shader&& other) noexcept;
```

Move-assigns a shader wrapper.

Parameter	Type	Meaning
other	Shader&&	Reference or value identifying the associated HRL resource or wrapper object.

Shader::IsValid

```
bool IsValid() const;
```

Returns the underlying shader ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Shader::Release

```
void Release();
```

Destroys the owned shader and frees its GPU resources.

Shader::IsReady

```
bool IsReady() const;
```

Returns whether an asynchronous shader has finished compilation and linking.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Shader::Wait

```
void Wait();
```

Blocks until an asynchronous shader is compiled and linked.

Material

Named-parameter wrapper around a shader, including scalar/vector values, textures, and emissive color.

Material::Material

```
explicit Material(const Shader& shader);
```

Creates a material instance backed by a shader.

Parameter	Type	Meaning
shader	const Shader&	Reference or value identifying the associated HRL resource or wrapper object.

Material::~Material

```
Material(const Material&) = delete;
```

Destroys the material and its stored uniform data.

Material::Material&

```
Material& operator=(const Material&) = delete;
```

Copy assignment is disabled.

Material::Material

```
Material(Material&& other) noexcept;
```

Move-constructs a material wrapper.

Parameter	Type	Meaning
other	Material&&	Reference or value identifying the associated HRL resource or wrapper object.

Material::Material&

```
Material& operator=(Material&& other) noexcept;
```

Move-assigns a material wrapper.

Parameter	Type	Meaning
other	Material&&	Reference or value identifying the associated HRL resource or wrapper object.

Material::IsValid

```
bool IsValid() const;
```

Returns the underlying material ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Material::Release

```
void Release();
```

Destroys the material and releases its resources.

Material::SetInt

```
void SetInt(const char* name, int value);
```

Sets an integer uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.
value	int	Reference or value identifying the associated HRL resource or wrapper object.

Material::SetTexture

```
void SetTexture(const char* name, const Texture& texture);
```

Binds a texture to a named sampler uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.
texture	const Texture&	Reference or value identifying the associated HRL resource or wrapper object.

Material::SetBool

```
void SetBool(const char* name, bool value);
```

Sets a boolean uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.
value	bool	Reference or value identifying the associated HRL resource or wrapper object.

Material::SetFloat

```
void SetFloat(const char* name, float value);
```

Sets a float uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.
value	float	Reference or value identifying the associated HRL resource or wrapper object.

Material::SetVec2

```
void SetVec2(const char* name, float x, float y);
```

Sets a vec2 uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Material::SetVec3

```
void SetVec3(const char* name, float x, float y, float z);
```

Sets a vec3 uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.

x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Material::SetVec4

```
void SetVec4(const char* name, float x, float y, float z, float w);
```

Sets a vec4 uniform on the material.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
w	float	Value supplied to configure the w property or operation.

Material::SetEmissiveColor

```
void SetEmissiveColor(float r, float g, float b, float a);
```

Sets the emissive color tint of the material.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Viewport

Rectangular rendering target/view into a Scene with an optional camera.

Viewport::Viewport

```
Viewport(Scene& scene, const Camera* camera, float x, float y, float width, float height);
```

Creates a viewport that renders a scene through a camera into a screen region.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
camera	const Camera*	Reference or value identifying the associated HRL resource or wrapper object.
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
width	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Viewport::~Viewport

```
Viewport(const Viewport&) = delete;
```

Destroys the viewport.

Viewport::Viewport&

```
Viewport& operator=(const Viewport&) = delete;
```

Copy assignment is disabled.

Viewport::Viewport

```
Viewport(Viewport&& other) noexcept;
```

Move-constructs a viewport wrapper.

Parameter	Type	Meaning
other	Viewport&&	Reference or value identifying the associated HRL resource or wrapper object.

Viewport::Viewport&

```
Viewport& operator=(Viewport&& other) noexcept;
```

Move-assigns a viewport wrapper.

Parameter	Type	Meaning
other	Viewport&&	Reference or value identifying the associated HRL resource or wrapper object.

Viewport::IsValid

```
bool IsValid() const;
```

Returns the underlying viewport ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Viewport::Release

```
void Release();
```

Destroys the viewport.

Viewport::SetCamera

```
void SetCamera(const Camera* camera);
```

Reassigns the camera used by the viewport.

Parameter	Type	Meaning
camera	const Camera*	Reference or value identifying the associated HRL resource or wrapper object.

Viewport::SetRect

```
void SetRect(float x, float y, float width, float height);
```

Updates the screen-space rectangle of the viewport.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
width	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Camera

Scene camera defining projection type and position/orientation.

Camera::Camera

```
explicit Camera(Scene& scene, ECameraType type);
```

Creates a camera in a scene.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
type	ECameraType	Value supplied to configure the type property or operation.

Camera::Camera

```
Camera(const Camera&) = delete;
```

Destroys the camera.

Camera::Camera&

```
Camera& operator=(const Camera&) = delete;
```

Copy assignment is disabled.

Camera::Camera

```
Camera(Camera&& other) noexcept;
```

Move-constructs a camera wrapper.

Parameter	Type	Meaning
other	Camera&&	Reference or value identifying the associated HRL resource or wrapper object.

Camera::Camera&

```
Camera& operator=(Camera&& other) noexcept;
```

Move-assigns a camera wrapper.

Parameter	Type	Meaning
other	Camera&&	Reference or value identifying the associated HRL resource or wrapper object.

Camera::IsValid

```
bool IsValid() const;
```

Returns the underlying camera ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Camera::Release

```
void Release();
```

Destroys the camera.

Camera::SetType

```
void SetType(ECameraType type);
```

Changes the projection type of an existing camera at runtime.

Parameter	Type	Meaning
type	ECameraType	Value supplied to configure the type property or operation.

Camera::SetOrthoVertical

```
void SetOrthoVertical(float height);
```

Sets the vertical extent of an orthographic camera's view volume.

Parameter	Type	Meaning
height	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Camera::SetPerspectiveFov

```
void SetPerspectiveFov(float fov);
```

Sets the vertical field of view for a perspective camera.

Parameter	Type	Meaning
fov	float	Value supplied to configure the fov property or operation.

Camera::SetNearPlane

```
void SetNearPlane(float nearPlane);
```

Sets the near clipping plane distance.

Parameter	Type	Meaning
nearPlane	float	Value supplied to configure the nearPlane property or operation.

Camera::SetFarPlane

```
void SetFarPlane(float farPlane);
```

Sets the far clipping plane distance.

Parameter	Type	Meaning
farPlane	float	Value supplied to configure the farPlane property or operation.

Camera::SetLocation

```
void SetLocation(float x, float y, float z);
```

Sets the world-space position of the camera.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Camera::SetRotation

```
void SetRotation(float pitch, float yaw, float roll);
```

Sets the orientation of the camera using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

Font

Font resource constructed from in-memory font data.

Font::Font

```
Font(const void* data, std::size_t size);
```

Loads a TrueType font from a memory buffer.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

Font::~Font

```
Font(const Font&) = delete;
```

Destroys the font and releases its resources.

Font::Font&

```
Font& operator=(const Font&) = delete;
```

Copy assignment is disabled.

Font::Font

```
Font(Font&& other) noexcept;
```

Move-constructs a font wrapper.

Parameter	Type	Meaning
other	Font&&	Reference or value identifying the associated HRL resource or wrapper object.

Font::Font&

```
Font& operator=(Font&& other) noexcept;
```

Move-assigns a font wrapper.

Parameter	Type	Meaning
other	Font&&	Reference or value identifying the associated HRL resource or wrapper object.

Font::IsValid

```
bool IsValid() const;
```

Returns the underlying font ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Font::Release

```
void Release();
```

Destroys the font.

Widget

UI element attached to a viewport, with common layout/state controls and type-specific button, label, image, slider, checkbox, and progress-bar APIs.

Widget::Widget

```
Widget(Viewport& viewport, EWidgetType type);
```

Creates a widget in a viewport.

Parameter	Type	Meaning
viewport	Viewport&	Reference or value identifying the associated HRL resource or wrapper object.
type	EWidgetType	Value supplied to configure the type property or operation.

Widget::~Widget

```
Widget(const Widget&) = delete;
```

Destroys the widget.

Widget::Widget&

```
Widget& operator=(const Widget&) = delete;
```

Copy assignment is disabled.

Widget::Widget

```
Widget(Widget&& other) noexcept;
```

Move-constructs a widget wrapper.

Parameter	Type	Meaning
other	Widget&&	Reference or value identifying the associated HRL resource or wrapper object.

Widget::Widget&

```
Widget& operator=(Widget&& other) noexcept;
```

Move-assigns a widget wrapper.

Parameter	Type	Meaning
other	Widget&&	Reference or value identifying the associated HRL resource or wrapper object.

Widget::IsValid

```
bool IsValid() const;
```

Returns the underlying widget ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Widget::Release

```
void Release();
```

Destroys the widget.

Widget::SetPosition

```
void SetPosition(float x, float y);
```

Sets the widget position.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Widget::SetSize

```
void SetSize(float width, float height);
```

Sets the widget size.

Parameter	Type	Meaning
width	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
height	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Widget::SetAlpha

```
void SetAlpha(float alpha);
```

Sets the widget alpha value.

Parameter	Type	Meaning
alpha	float	Value supplied to configure the alpha property or operation.

Widget::SetAnchor

```
void SetAnchor(float ax, float ay);
```

Sets the widget anchor point.

Parameter	Type	Meaning
ax	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
ay	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Widget::SetVisible

```
void SetVisible(bool visible);
```

Enables or disables widget visibility.

Parameter	Type	Meaning
visible	bool	Boolean state controlling whether the associated feature is enabled or active.

Widget::SetEnabled

```
void SetEnabled(bool enabled);
```

Enables or disables widget interaction.

Parameter	Type	Meaning
-----------	------	---------

enabled	bool	Boolean state controlling whether the associated feature is enabled or active.
---------	------	--

Widget::SetZIndex

```
void SetZIndex(int zIndex);
```

Sets the widget Z index.

Parameter	Type	Meaning
zIndex	int	Value supplied to configure the zIndex property or operation.

Widget::IsHovered

```
bool IsHovered() const;
```

Returns whether the widget is currently hovered.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Widget::SetButtonClickable

```
void SetButtonClickable(bool clickable);
```

Enables or disables button clickability.

Parameter	Type	Meaning
clickable	bool	Boolean state controlling whether the associated feature is enabled or active.

Widget::SetButtonText

```
void SetButtonText(const char* text);
```

Sets the displayed button text.

Parameter	Type	Meaning
text	const char*	Value supplied to configure the text property or operation.

Widget::SetButtonTextSize

```
void SetButtonTextSize(float size);
```

Sets the button text size.

Parameter	Type	Meaning
size	float	Value supplied to configure the size property or operation.

Widget::SetButtonTextTintColor

```
void SetButtonTextTintColor( EWidgetState state, float r, float g, float b, float a);
```

Sets the button text tint color for a given interaction state.

Parameter	Type	Meaning
state	EWidgetState	Value supplied to configure the state property or operation.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetButtonTextFont

```
void SetButtonTextFont(const Font& font);
```

Sets the button text font.

Parameter	Type	Meaning
font	const Font&	Reference or value identifying the associated HRL resource or wrapper object.

Widget::SetButtonBackgroundTexture

```
void SetButtonBackgroundTexture(EWidgetState state, const Texture* texture);
```

Sets the button background texture for a given interaction state.

Parameter	Type	Meaning
state	EWidgetState	Value supplied to configure the state property or operation.
texture	const Texture*	Reference or value identifying the associated HRL resource or wrapper object.

Widget::SetButtonBackgroundTintColor

```
void SetButtonBackgroundTintColor(EWidgetState state, float r, float g, float b, float a);
```

Sets the button background tint color for a given interaction state.

Parameter	Type	Meaning
state	EWidgetState	Value supplied to configure the state property or operation.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetButtonPressedCallback

```
void SetButtonPressedCallback(ButtonPressedCallback callback, void* userData);
```

Registers the button pressed callback.

Parameter	Type	Meaning
callback	ButtonPressedCallback	Function pointer invoked by the corresponding event or state transition.
userData	void*	Value supplied to configure the userData property or operation.

Widget::SetLabelText

```
void SetLabelText(const char* text);
```

Sets the displayed label text.

Parameter	Type	Meaning
text	const char*	Value supplied to configure the text property or operation.

Widget::SetLabelTextSize

```
void SetLabelTextSize(float size);
```

Sets the label text size.

Parameter	Type	Meaning
size	float	Value supplied to configure the size property or operation.

Widget::SetLabelFont

```
void SetLabelFont(const Font& font);
```

Sets the label font.

Parameter	Type	Meaning
font	const Font&	Reference or value identifying the associated HRL resource or wrapper object.

Widget::SetLabelTintColor

```
void SetLabelTintColor(float r, float g, float b, float a);
```

Sets the label tint color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetImageTexture

```
void SetImageTexture(const Texture& texture);
```

Sets the image texture.

Parameter	Type	Meaning
texture	const Texture&	Reference or value identifying the associated HRL resource or wrapper object.

Widget::SetImageTintColor

```
void SetImageTintColor(float r, float g, float b, float a);
```

Sets the image tint color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetSliderRange

```
void SetSliderRange(float minimum, float maximum);
```

Sets the slider minimum and maximum range.

Parameter	Type	Meaning
minimum	float	Value supplied to configure the minimum property or operation.
maximum	float	Value supplied to configure the maximum property or operation.

Widget::SetSliderValue

```
void SetSliderValue(float value);
```

Sets the slider value.

Parameter	Type	Meaning
value	float	Reference or value identifying the associated HRL resource or wrapper object.

Widget::GetSliderValue

```
float GetSliderValue() const;
```

Returns the current slider value.

Return: float. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Widget::SetSliderOrientation

```
void SetSliderOrientation(ESliderOrientation orientation);
```

Sets the slider orientation.

Parameter	Type	Meaning
orientation	ESliderOrientation	Value supplied to configure the orientation property or operation.

Widget::SetSliderClickable

```
void SetSliderClickable(bool clickable);
```

Enables or disables slider clickability.

Parameter	Type	Meaning
clickable	bool	Boolean state controlling whether the associated feature is enabled or active.

Widget::SetSliderBackgroundColor

```
void SetSliderBackgroundColor(float r, float g, float b, float a);
```

Sets the slider background color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetSliderFillColor

```
void SetSliderFillColor(float r, float g, float b, float a);
```

Sets the slider fill color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetSliderHandleColor

```
void SetSliderHandleColor(float r, float g, float b, float a);
```

Sets the slider handle color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetSliderChangedCallback

```
void SetSliderChangedCallback(SliderChangedCallback callback, void* userData);
```

Registers the slider value-changed callback.

Parameter	Type	Meaning
callback	SliderChangedCallback	Function pointer invoked by the corresponding event or state transition.
userData	void*	Value supplied to configure the userData property or operation.

Widget::SetCheckboxChecked

```
void SetCheckboxChecked(bool checked);
```

Sets the checkbox checked state.

Parameter	Type	Meaning
checked	bool	Boolean state controlling whether the associated feature is enabled or active.

Widget::IsCheckboxChecked

```
bool IsCheckboxChecked() const;
```

Returns whether the checkbox is checked.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Widget::SetCheckboxClickable

```
void SetCheckboxClickable(bool clickable);
```

Enables or disables checkbox clickability.

Parameter	Type	Meaning
clickable	bool	Boolean state controlling whether the associated feature is enabled or active.

Widget::SetCheckboxBackgroundColor

```
void SetCheckboxBackgroundColor(float r, float g, float b, float a);
```

Sets the checkbox background color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetCheckboxCheckedColor

```
void SetCheckboxCheckedColor(float r, float g, float b, float a);
```

Sets the color used for the checked state.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetCheckboxChangedCallback

```
void SetCheckboxChangedCallback(CheckboxChangedCallback callback, void* userData);
```

Registers the checkbox value-changed callback.

Parameter	Type	Meaning
callback	CheckboxChangedCallback	Function pointer invoked by the corresponding event or state transition.
userData	void*	Value supplied to configure the userData property or operation.

Widget::SetProgressBarValue

```
void SetProgressBarValue(float value);
```

Sets the progress-bar value.

Parameter	Type	Meaning
value	float	Reference or value identifying the associated HRL resource or wrapper object.

Widget::GetProgressBarValue

```
float GetProgressBarValue() const;
```

Returns the progress-bar value.

Return: float. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Widget::SetProgressBarBackgroundColor

```
void SetProgressBarBackgroundColor(float r, float g, float b, float a);
```

Sets the progress-bar background color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Widget::SetProgressBarFillColor

```
void SetProgressBarFillColor(float r, float g, float b, float a);
```

Sets the progress-bar fill color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Gizmo

Interactive transform manipulator attached to a viewport, exposing axis/mode visibility, world/local space, colors, hit state and change callbacks.

Gizmo::Gizmo

```
explicit Gizmo(Viewport& viewport);
```

Creates an editor gizmo attached to a viewport.

Parameter	Type	Meaning
viewport	Viewport&	Reference or value identifying the associated HRL resource or wrapper object.

Gizmo::~Gizmo

```
Gizmo(const Gizmo&) = delete;
```

Destroys the gizmo.

Gizmo::Gizmo&

```
Gizmo& operator=(const Gizmo&) = delete;
```

Copy assignment is disabled.

Gizmo::Gizmo

```
Gizmo(Gizmo&& other) noexcept;
```

Move-constructs a gizmo wrapper.

Parameter	Type	Meaning
other	Gizmo&&	Reference or value identifying the associated HRL resource or wrapper object.

Gizmo::Gizmo&

```
Gizmo& operator=(Gizmo&& other) noexcept;
```

Move-assigns a gizmo wrapper.

Parameter	Type	Meaning
other	Gizmo&&	Reference or value identifying the associated HRL resource or wrapper object.

Gizmo::IsValid

```
bool IsValid() const;
```

Returns the underlying gizmo ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

Gizmo::Release

```
void Release();
```

Destroys the gizmo.

Gizmo::SetPosition

```
void SetPosition(float x, float y, float z);
```

Sets the gizmo position in world space.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Gizmo::GetPosition

```
void GetPosition(float& x, float& y, float& z) const;
```

Writes the gizmo position into caller-provided values.

Parameter	Type	Meaning
x	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Gizmo::SetRotation

```
void SetRotation(float pitch, float yaw, float roll);
```

Sets the gizmo rotation using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

Gizmo::GetRotation

```
void GetRotation(float& pitch, float& yaw, float& roll) const;
```

Writes the gizmo rotation into caller-provided values.

Parameter	Type	Meaning
pitch	float&	Value supplied to configure the pitch property or operation.
yaw	float&	Value supplied to configure the yaw property or operation.
roll	float&	Value supplied to configure the roll property or operation.

Gizmo::SetScale

```
void SetScale(float x, float y, float z);
```

Sets the gizmo scale.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Gizmo::GetScale

```
void GetScale(float& x, float& y, float& z) const;
```

Writes the gizmo scale into caller-provided values.

Parameter	Type	Meaning
x	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

Gizmo::SetMode

```
void SetMode(EGizmoMode mode);
```

Sets the gizmo transform mode flags.

Parameter	Type	Meaning
mode	EGizmoMode	Value supplied to configure the mode property or operation.

Gizmo::SetMode

```
void SetMode(uint mode);
```

Convenience overload for combined gizmo mode flags.

Parameter	Type	Meaning
mode	uint	Value supplied to configure the mode property or operation.

Gizmo::GetMode

```
EGizmoMode GetMode() const;
```

Returns the current gizmo transform mode flags.

Gizmo::SetSpace

```
void SetSpace(EGizmoSpace space);
```

Sets the gizmo coordinate space.

Parameter	Type	Meaning
space	EGizmoSpace	Value supplied to configure the space property or operation.

Gizmo::GetSpace

```
EGizmoSpace GetSpace() const;
```

Returns the gizmo coordinate space.

Gizmo::SetTranslateVisible

```
void SetTranslateVisible(bool visible);
```

Shows or hides the translation handles.

Parameter	Type	Meaning
visible	bool	Boolean state controlling whether the associated feature is enabled or active.

Gizmo::SetRotateVisible

```
void SetRotateVisible(bool visible);
```

Shows or hides the rotation handles.

Parameter	Type	Meaning
visible	bool	Boolean state controlling whether the associated feature is enabled or active.

Gizmo::SetScaleVisible

```
void SetScaleVisible(bool visible);
```

Shows or hides the scale handles.

Parameter	Type	Meaning
visible	bool	Boolean state controlling whether the associated feature is enabled or active.

Gizmo::SetTranslateAxes

```
void SetTranslateAxes(int axisMask);
```

Sets the visible translation axes using EGizmoAxis flags.

Parameter	Type	Meaning
axisMask	int	Value supplied to configure the axisMask property or operation.

Gizmo::SetRotateAxes

```
void SetRotateAxes(int axisMask);
```

Sets the visible rotation axes using EGizmoAxis flags.

Parameter	Type	Meaning
axisMask	int	Value supplied to configure the axisMask property or operation.

Gizmo::SetScaleAxes

```
void SetScaleAxes(int axisMask);
```

Sets the visible scale axes using EGizmoAxis flags.

Parameter	Type	Meaning
axisMask	int	Value supplied to configure the axisMask property or operation.

Gizmo::SetSize

```
void SetSize(float worldSize);
```

Sets the gizmo world-space size.

Parameter	Type	Meaning
worldSize	float	Value supplied to configure the worldSize property or operation.

Gizmo::SetScreenSize

```
void SetScreenSize(float pixels);
```

Sets the gizmo screen-space size in pixels.

Parameter	Type	Meaning
pixels	float	Value supplied to configure the pixels property or operation.

Gizmo::SetUseScreenSize

```
void SetUseScreenSize(bool useScreenSize);
```

Enables or disables screen-space sizing.

Parameter	Type	Meaning
useScreenSize	bool	Value supplied to configure the useScreenSize property or operation.

Gizmo::SetVisible

```
void SetVisible(bool visible);
```

Shows or hides the gizmo.

Parameter	Type	Meaning
visible	bool	Boolean state controlling whether the associated feature is enabled or active.

Gizmo::SetEnabled

```
void SetEnabled(bool enabled);
```

Enables or disables gizmo interaction.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

Gizmo::SetAxisColor

```
void SetAxisColor(int axis, float r, float g, float b, float a);
```

Sets the color of an axis.

Parameter	Type	Meaning
axis	int	Value supplied to configure the axis property or operation.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Gizmo::SetCenterColor

```
void SetCenterColor(float r, float g, float b, float a);
```

Sets the gizmo center color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Gizmo::SetHoverColor

```
void SetHoverColor(float r, float g, float b, float a);
```

Sets the gizmo hover color.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

Gizmo::GetHoveredPart

```
EGizmoPart GetHoveredPart() const;
```

Returns the currently hovered gizmo part.

Gizmo::GetActivePart

```
EGizmoPart GetActivePart() const;
```

Returns the currently active gizmo part.

Gizmo::GetHoveredOperation

```
EGizmoOperation GetHoveredOperation() const;
```

Returns the currently hovered gizmo operation.

Gizmo::GetActiveOperation

```
EGizmoOperation GetActiveOperation() const;
```

Returns the currently active gizmo operation.

Gizmo::SetRotateArcDegrees

```
void SetRotateArcDegrees(float degrees);
```

Sets the visible rotation arc size in degrees.

Parameter	Type	Meaning
degrees	float	Value supplied to configure the degrees property or operation.

Gizmo::SetChangedCallback

```
void SetChangedCallback(GizmoChangedCallback callback, void* userData);
```

Registers a gizmo interaction callback.

Parameter	Type	Meaning
callback	GizmoChangedCallback	Function pointer invoked by the corresponding event or state transition.
userData	void*	Value supplied to configure the userData property or operation.

VFXSystem

Container and timeline for particle emitters, transform, playback, looping and automatic update behavior.

VFXSystem::VFXSystem

```
explicit VFXSystem(Scene& scene);
```

Creates a VFX system in a scene.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.

VFXSystem::~VFXSystem

```
VFXSystem(const VFXSystem&) = delete;
```

Destroys the VFX system.

VFXSystem::VFXSystem&

```
VFXSystem& operator=(const VFXSystem&) = delete;
```

Copy assignment is disabled.

VFXSystem::VFXSystem

```
VFXSystem(VFXSystem&& other) noexcept;
```

Move-constructs a VFX system wrapper.

Parameter	Type	Meaning
other	VFXSystem&&	Reference or value identifying the associated HRL resource or wrapper object.

VFXSystem::VFXSystem&

```
VFXSystem& operator=(VFXSystem&& other) noexcept;
```

Move-assigns a VFX system wrapper.

Parameter	Type	Meaning
other	VFXSystem&&	Reference or value identifying the associated HRL resource or wrapper object.

VFXSystem::IsValid

```
bool IsValid() const;
```

Returns the underlying VFX system ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

VFXSystem::Release

```
void Release();
```

Destroys the VFX system.

VFXSystem::SetPosition

```
void SetPosition(float x, float y, float z);
```

Sets the VFX system world-space position.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXSystem::GetPosition

```
void GetPosition(float& x, float& y, float& z) const;
```

Retrieves the VFX system world-space position.

Parameter	Type	Meaning
x	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXSystem::SetRotation

```
void SetRotation(float pitch, float yaw, float roll);
```

Sets the VFX system rotation using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

VFXSystem::GetRotation

```
void GetRotation(float& pitch, float& yaw, float& roll) const;
```

Retrieves the VFX system rotation.

Parameter	Type	Meaning
pitch	float&	Value supplied to configure the pitch property or operation.
yaw	float&	Value supplied to configure the yaw property or operation.
roll	float&	Value supplied to configure the roll property or operation.

VFXSystem::SetScale

```
void SetScale(float x, float y, float z);
```

Sets the VFX system scale.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
---	-------	---

VFXSystem::GetScale

```
void GetScale(float& x, float& y, float& z) const;
```

Retrieves the VFX system scale.

Parameter	Type	Meaning
x	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float&	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXSystem::Play

```
void Play();
```

Starts VFX playback.

VFXSystem::Stop

```
void Stop();
```

Stops VFX playback.

VFXSystem::Pause

```
void Pause();
```

Pauses VFX playback.

VFXSystem::Reset

```
void Reset();
```

Resets the VFX system.

VFXSystem::SetLooping

```
void SetLooping(bool looping);
```

Enables or disables system looping.

Parameter	Type	Meaning
looping	bool	Boolean state controlling whether the associated feature is enabled or active.

VFXSystem::SetTimeScale

```
void SetTimeScale(float scale);
```

Sets the VFX time scale.

Parameter	Type	Meaning
-----------	------	---------

scale	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
-------	-------	---

VFXSystem::SetEnabled

```
void SetEnabled(bool enabled);
```

Enables or disables the VFX system.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

VFXSystem::SetAutoUpdate

```
void SetAutoUpdate(bool autoUpdate);
```

Enables or disables automatic update for the VFX system.

Parameter	Type	Meaning
autoUpdate	bool	Boolean state controlling whether the associated feature is enabled or active.

VFXSystem::SetDuration

```
void SetDuration(float duration);
```

Sets the system duration.

Parameter	Type	Meaning
duration	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

VFXSystem::CreateEmitter

```
VFXEmitter CreateEmitter();
```

Creates an emitter owned by this VFX system.

VFXEmitter

Configures particle spawning, motion, rendering, curves, collision and material/mesh presentation.

VFXEmitter::VFXEmitter

```
explicit VFXEmitter(VFXSystem& system);
```

Creates an emitter owned by a VFX system.

Parameter	Type	Meaning
system	VFXSystem&	Value supplied to configure the system property or operation.

VFXEmitter::~VFXEmitter

```
VFXEmitter(const VFXEmitter&) = delete;
```

Destroys the emitter.

VFXEmitter::VFXEmitter&

```
VFXEmitter& operator=(const VFXEmitter&) = delete;
```

Copy assignment is disabled.

VFXEmitter::VFXEmitter

```
VFXEmitter(VFXEmitter&& other) noexcept;
```

Move-constructs an emitter wrapper.

Parameter	Type	Meaning
other	VFXEmitter&&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::VFXEmitter&

```
VFXEmitter& operator=(VFXEmitter&& other) noexcept;
```

Move-assigns an emitter wrapper.

Parameter	Type	Meaning
other	VFXEmitter&&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::IsValid

```
bool IsValid() const;
```

Returns the underlying emitter ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

VFXEmitter::Release

```
void Release();
```

Destroys the emitter.

VFXEmitter::SetEnabled

```
void SetEnabled(bool enabled);
```

Enables or disables the emitter.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

VFXEmitter::SetPosition

```
void SetPosition(float x, float y, float z);
```

Sets the emitter world-space position.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXEmitter::SetRotation

```
void SetRotation(float pitch, float yaw, float roll);
```

Sets the emitter rotation using Euler angles in degrees.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

VFXEmitter::SetMaxParticles

```
void SetMaxParticles(uint maxParticles);
```

Sets the maximum particle count.

Parameter	Type	Meaning
maxParticles	uint	Value supplied to configure the maxParticles property or operation.

VFXEmitter::SetSpawnRate

```
void SetSpawnRate(float particlesPerSecond);
```

Sets the particle spawn rate.

Parameter	Type	Meaning
particlesPerSecond	float	Value supplied to configure the particlesPerSecond property or operation.

VFXEmitter::SetBurst

```
void SetBurst(uint count);
```

Sets the emitter's immediate burst count.

Parameter	Type	Meaning
count	uint	Index/count selector for an element or quality/iteration level in the associated collection.

VFXEmitter::AddBurst

```
void AddBurst(float time, uint count);
```

Adds a burst event at a given system time.

Parameter	Type	Meaning
time	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
count	uint	Index/count selector for an element or quality/iteration level in the associated collection.

VFXEmitter::ClearBursts

```
void ClearBursts();
```

Clears all scheduled burst events.

VFXEmitter::SetLifetime

```
void SetLifetime(float minLifetime, float maxLifetime);
```

Sets the minimum and maximum particle lifetime.

Parameter	Type	Meaning
minLifetime	float	Value supplied to configure the minLifetime property or operation.
maxLifetime	float	Value supplied to configure the maxLifetime property or operation.

VFXEmitter::SetSpawnShape

```
void SetSpawnShape(EVFXSpawnShape shape);
```

Sets the particle spawn shape.

Parameter	Type	Meaning
shape	EVFXSpawnShape	Value supplied to configure the shape property or operation.

VFXEmitter::SetShapeRadius

```
void SetShapeRadius(float radius);
```

Sets the radius used by the spawn shape.

Parameter	Type	Meaning
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

VFXEmitter::SetShapeSize

```
void SetShapeSize(float x, float y, float z);
```

Sets the size used by the spawn shape.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXEmitter::SetShapeAngle

```
void SetShapeAngle(float degrees);
```

Sets the angle used by the spawn shape.

Parameter	Type	Meaning
degrees	float	Value supplied to configure the degrees property or operation.

VFXEmitter::SetInitialVelocity

```
void SetInitialVelocity( float minX, float minY, float minZ, float maxX, float maxY, float maxZ);
```

Sets the initial velocity range.

Parameter	Type	Meaning
minX	float	Value supplied to configure the minX property or operation.
minY	float	Value supplied to configure the minY property or operation.
minZ	float	Value supplied to configure the minZ property or operation.
maxX	float	Value supplied to configure the maxX property or operation.
maxY	float	Value supplied to configure the maxY property or operation.
maxZ	float	Value supplied to configure the maxZ property or operation.

VFXEmitter::SetInitialSpeed

```
void SetInitialSpeed(float minSpeed, float maxSpeed);
```

Sets the initial speed range.

Parameter	Type	Meaning
minSpeed	float	Value supplied to configure the minSpeed property or operation.
maxSpeed	float	Value supplied to configure the maxSpeed property or operation.

VFXEmitter::SetInitialRotation

```
void SetInitialRotation( float minX, float minY, float minZ, float maxX, float maxY, float maxZ);
```

Sets the initial rotation range.

Parameter	Type	Meaning
minX	float	Value supplied to configure the minX property or operation.
minY	float	Value supplied to configure the minY property or operation.
minZ	float	Value supplied to configure the minZ property or operation.
maxX	float	Value supplied to configure the maxX property or operation.
maxY	float	Value supplied to configure the maxY property or operation.
maxZ	float	Value supplied to configure the maxZ property or operation.

VFXEmitter::SetAngularVelocity

```
void SetAngularVelocity( float minX, float minY, float minZ, float maxX, float maxY, float maxZ);
```

Sets the angular velocity range.

Parameter	Type	Meaning
minX	float	Value supplied to configure the minX property or operation.
minY	float	Value supplied to configure the minY property or operation.
minZ	float	Value supplied to configure the minZ property or operation.
maxX	float	Value supplied to configure the maxX property or operation.
maxY	float	Value supplied to configure the maxY property or operation.
maxZ	float	Value supplied to configure the maxZ property or operation.

VFXEmitter::SetGravity

```
void SetGravity(float x, float y, float z);
```

Sets the gravity vector.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXEmitter::SetDrag

```
void SetDrag(float drag);
```

Sets particle drag.

Parameter	Type	Meaning
drag	float	Value supplied to configure the drag property or operation.

VFXEmitter::SetForce

```
void SetForce(float x, float y, float z);
```

Sets an external force vector.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXEmitter::SetNoise

```
void SetNoise(float strength, float frequency, float scrollSpeed);
```

Sets the noise strength, frequency and scroll speed.

Parameter	Type	Meaning
strength	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
frequency	float	Value supplied to configure the frequency property or operation.
scrollSpeed	float	Value supplied to configure the scrollSpeed property or operation.

VFXEmitter::SetRenderMode

```
void SetRenderMode(EVFXRenderMode mode);
```

Sets the particle render mode.

Parameter	Type	Meaning
mode	EVFXRenderMode	Value supplied to configure the mode property or operation.

VFXEmitter::SetTexture

```
void SetTexture(const Texture& texture);
```

Sets the texture used by the emitter.

Parameter	Type	Meaning
texture	const Texture&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::SetMaterial

```
void SetMaterial(const Material& material);
```

Sets the material used by the emitter.

Parameter	Type	Meaning
material	const Material&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::SetMesh

```
void SetMesh(const Mesh& mesh);
```

Sets the mesh used by the emitter in mesh-rendering mode.

Parameter	Type	Meaning
mesh	const Mesh&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::SetMeshScale

```
void SetMeshScale(float x, float y, float z);
```

Sets the particle mesh scale.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXEmitter::SetMeshRotation

```
void SetMeshRotation(float pitch, float yaw, float roll);
```

Sets the particle mesh rotation.

Parameter	Type	Meaning
pitch	float	Value supplied to configure the pitch property or operation.
yaw	float	Value supplied to configure the yaw property or operation.
roll	float	Value supplied to configure the roll property or operation.

VFXEmitter::SetBlendMode

```
void SetBlendMode(EVFXBlendMode mode);
```

Sets the particle blending mode.

Parameter	Type	Meaning
-----------	------	---------

mode	EVFXBlendMode	Value supplied to configure the mode property or operation.
------	---------------	---

VFXEmitter::SetParticleSize

```
void SetParticleSize(float x, float y);
```

Sets the particle billboard size.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VFXEmitter::SetStretch

```
void SetStretch(float amount);
```

Sets the stretched-billboard amount.

Parameter	Type	Meaning
amount	float	Value supplied to configure the amount property or operation.

VFXEmitter::SetSimulationSpace

```
void SetSimulationSpace(EVFXSimulationSpace space);
```

Sets the particle simulation space.

Parameter	Type	Meaning
space	EVFXSimulationSpace	Value supplied to configure the space property or operation.

VFXEmitter::CreateColorCurve

```
VFXCurve CreateColorCurve();
```

Creates a color curve owned by the emitter.

VFXEmitter::CreateFloatCurve

```
VFXCurve CreateFloatCurve();
```

Creates a float curve owned by the emitter.

VFXEmitter::SetColorCurve

```
void SetColorCurve(const VFXCurve& curve);
```

Assigns a color curve to the emitter.

Parameter	Type	Meaning
curve	const VFXCurve&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::SetSizeCurve

```
void SetSizeCurve(const VFXCurve& curve);
```

Assigns a size curve to the emitter.

Parameter	Type	Meaning
-----------	------	---------

curve	const VFXCurve&	Reference or value identifying the associated HRL resource or wrapper object.
-------	-----------------	---

VFXEmitter::SetRotationCurve

```
void SetRotationCurve(const VFXCurve& curve);
```

Assigns a rotation curve to the emitter.

Parameter	Type	Meaning
curve	const VFXCurve&	Reference or value identifying the associated HRL resource or wrapper object.

VFXEmitter::SetCollisionEnabled

```
void SetCollisionEnabled(bool enabled);
```

Enables or disables VFX collision.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

VFXEmitter::SetCollisionRestitution

```
void SetCollisionRestitution(float restitution);
```

Sets collision restitution.

Parameter	Type	Meaning
restitution	float	Value supplied to configure the restitution property or operation.

VFXEmitter::SetCollisionFriction

```
void SetCollisionFriction(float friction);
```

Sets collision friction.

Parameter	Type	Meaning
friction	float	Value supplied to configure the friction property or operation.

VFXEmitter::SetCollisionScene

```
void SetCollisionScene(Scene& scene, bool enabled);
```

Enables or disables collision against a scene.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

VFXCurve

Keyframed scalar or color curve resource consumed by VFX emitters.

VFXCurve::Borrowed

```
static VFXCurve Borrowed(id value);
```

Creates a non-owning wrapper around an existing curve ID.

Parameter	Type	Meaning
value	id	Reference or value identifying the associated HRL resource or wrapper object.

VFXCurve::CreateColor

```
static VFXCurve CreateColor(VFXEmitter& emitter);
```

Creates a color curve owned by an emitter.

Parameter	Type	Meaning
emitter	VFXEmitter&	Reference or value identifying the associated HRL resource or wrapper object.

VFXCurve::CreateFloat

```
static VFXCurve CreateFloat(VFXEmitter& emitter);
```

Creates a float curve owned by an emitter.

Parameter	Type	Meaning
emitter	VFXEmitter&	Reference or value identifying the associated HRL resource or wrapper object.

VFXCurve::VFXCurve

```
VFXCurve(const VFXCurve&) = delete;
```

Destroys the curve.

VFXCurve::VFXCurve&

```
VFXCurve& operator=(const VFXCurve&) = delete;
```

Copy assignment is disabled.

VFXCurve::VFXCurve

```
VFXCurve(VFXCurve&& other) noexcept;
```

Move-constructs a curve wrapper.

Parameter	Type	Meaning
other	VFXCurve&&	Reference or value identifying the associated HRL resource or wrapper object.

VFXCurve::VFXCurve&

```
VFXCurve& operator=(VFXCurve&& other) noexcept;
```

Move-assigns a curve wrapper.

Parameter	Type	Meaning
other	VFXCurve&&	Reference or value identifying the associated HRL resource or wrapper object.

VFXCurve::IsValid

```
bool IsValid() const;
```

Returns the underlying curve ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

VFXCurve::Release

```
void Release();
```

Destroys the curve.

VFXCurve::AddColorKey

```
void AddColorKey(float time, float r, float g, float b, float a);
```

Adds a color key to the curve.

Parameter	Type	Meaning
time	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
a	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

VFXCurve::AddFloatKey

```
void AddFloatKey(float time, float value);
```

Adds a float key to the curve.

Parameter	Type	Meaning
time	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
value	float	Reference or value identifying the associated HRL resource or wrapper object.

VFXCurve::ClearColorKeys

```
void ClearColorKeys();
```

Removes all color keys from the curve.

VFXCurve::ClearFloatKeys

```
void ClearFloatKeys();
```

Removes all float keys from the curve.

VolumetricFog

Localized spherical volumetric fog volume attached to a scene.

VolumetricFog::VolumetricFog

```
explicit VolumetricFog(Scene& scene);
```

Creates a localized volumetric fog volume in a scene.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.

VolumetricFog::~VolumetricFog

```
VolumetricFog(const VolumetricFog&) = delete;
```

Destroys the volumetric fog volume.

VolumetricFog::VolumetricFog&

```
VolumetricFog& operator=(const VolumetricFog&) = delete;
```

Copy assignment is disabled.

VolumetricFog::VolumetricFog

```
VolumetricFog(VolumetricFog&& other) noexcept;
```

Move-constructs a volumetric fog wrapper.

Parameter	Type	Meaning
other	VolumetricFog&&	Reference or value identifying the associated HRL resource or wrapper object.

VolumetricFog::VolumetricFog&

```
VolumetricFog& operator=(VolumetricFog&& other) noexcept;
```

Move-assigns a volumetric fog wrapper.

Parameter	Type	Meaning
other	VolumetricFog&&	Reference or value identifying the associated HRL resource or wrapper object.

VolumetricFog::IsValid

```
bool IsValid() const;
```

Returns the underlying volumetric fog ID.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

VolumetricFog::Release

```
void Release();
```

Destroys the volumetric fog volume.

VolumetricFog::SetEnabled

```
void SetEnabled(bool enabled);
```

Enables or disables the volumetric fog volume.

Parameter	Type	Meaning
enabled	bool	Boolean state controlling whether the associated feature is enabled or active.

VolumetricFog::SetPosition

```
void SetPosition(float x, float y, float z);
```

Sets the center of the localized volumetric fog volume in world space.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

VolumetricFog::SetRadius

```
void SetRadius(float radius);
```

Sets the radius of the localized volumetric fog volume.

Parameter	Type	Meaning
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

VolumetricFog::SetDensity

```
void SetDensity(float density);
```

Sets the density of the localized volumetric fog volume.

Parameter	Type	Meaning
density	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

VolumetricFog::SetColor

```
void SetColor(float r, float g, float b);
```

Sets the color of the localized volumetric fog volume.

Parameter	Type	Meaning
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

VolumetricFog::SetSteps

```
void SetSteps(uint steps);
```

Sets the ray-march sample count for the localized volumetric fog.

Parameter	Type	Meaning
steps	uint	Value supplied to configure the steps property or operation.

FBXResources

Owning wrapper for decoded FBX material and embedded texture resources, independent of mesh creation.

FBXResources::FBXResources

```
FBXResources(const void* data, std::size_t size);
```

Loads FBX materials and textures without importing geometry.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

FBXResources::~FBXResources

```
FBXResources(const FBXResources&) = delete;
```

Destroys the decoded FBX resource collection.

FBXResources::FBXResources&

```
FBXResources& operator=(const FBXResources&) = delete;
```

Copy assignment is disabled.

FBXResources::FBXResources

```
FBXResources(FBXResources&& other) noexcept;
```

Move-constructs an FBX resource wrapper.

Parameter	Type	Meaning
other	FBXResources&&	Reference or value identifying the associated HRL resource or wrapper object.

FBXResources::FBXResources&

```
FBXResources& operator=(FBXResources&& other) noexcept;
```

Move-assigns an FBX resource wrapper.

Parameter	Type	Meaning
other	FBXResources&&	Reference or value identifying the associated HRL resource or wrapper object.

FBXResources::IsValid

```
bool IsValid() const;
```

Returns whether the FBX resource collection is valid.

Return: bool. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use INVALID_ID for an invalid/empty handle when applicable.

FBXResources::Release

```
void Release();
```

Releases the decoded FBX resource collection.

FBXResources::GetMaterialCount

```
std::size_t GetMaterialCount() const;
```

Returns the number of FBX materials.

Return: `std::size_t`. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use `INVALID_ID` for an invalid/empty handle when applicable.

FBXResources::GetMaterial

```
const FBXMaterialInfo* GetMaterial(std::size_t index) const;
```

Returns read-only material metadata.

Parameter	Type	Meaning
index	std::size_t	Index/count selector for an element or quality/iteration level in the associated collection.

FBXResources::FindMaterial

```
id FindMaterial(const char* name) const;
```

Finds a material by FBX name.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.

Return: `id`. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use `INVALID_ID` for an invalid/empty handle when applicable.

FBXResources::GetTextureCount

```
std::size_t GetTextureCount() const;
```

Returns the number of FBX textures.

Return: `std::size_t`. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use `INVALID_ID` for an invalid/empty handle when applicable.

FBXResources::GetTexture

```
const FBXTextureInfo* GetTexture(std::size_t index) const;
```

Returns read-only texture metadata and embedded bytes.

Parameter	Type	Meaning
index	std::size_t	Index/count selector for an element or quality/iteration level in the associated collection.

FBXResources::FindTexture

```
id FindTexture(const char* name) const;
```

Finds a texture by FBX name or filename.

Parameter	Type	Meaning
name	const char*	Null-terminated resource/member name used for lookup.

Return: `id`. Values and sentinels follow the associated C++ type and the HRL conventions; identifier-returning functions use `INVALID_ID` for an invalid/empty handle when applicable.

FBXResources::GetMaterialTexture

```
const FBXTextureInfo* GetMaterialTexture( std::size_t materialIndex, EFBXMaterialTextureSlot slot) const;
```

Returns the texture associated with a material texture slot.

Parameter	Type	Meaning
materialIndex	std::size_t	Index/count selector for an element or quality/iteration level in the associated collection.
slot	EFBXMaterialTextureSlot	Value supplied to configure the slot property or operation.

FBXResources::CreateTexture

```
Texture CreateTexture(std::size_t textureIndex) const;
```

Creates a GPU texture from an embedded FBX texture resource.

Parameter	Type	Meaning
textureIndex	std::size_t	Index/count selector for an element or quality/iteration level in the associated collection.

5. FBX and Free-Function Utilities

CreateMesh

```
HRL_CPP_API Mesh CreateMesh(Scene& scene, EMeshType type, const float* vertices);
```

Creates the requested child/resource object and returns it as an owning C++ wrapper.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
type	EMeshType	Value supplied to configure the type property or operation.
vertices	const float*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.

CreateMeshFromFile

```
HRL_CPP_API Mesh CreateMeshFromFile( Scene& scene, EMeshType type, const void* data, std::size_t size);
```

Creates the requested child/resource object and returns it as an owning C++ wrapper.

Parameter	Type	Meaning
scene	Scene&	Reference or value identifying the associated HRL resource or wrapper object.
type	EMeshType	Value supplied to configure the type property or operation.
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

CreateMaterialFromFBX

```
HRL_CPP_API Material CreateMaterialFromFBX(const void* data, std::size_t size);
```

Creates the requested child/resource object and returns it as an owning C++ wrapper.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

CreateMaterialFromFBXWithShader

```
HRL_CPP_API Material CreateMaterialFromFBXWithShader( const void* data, std::size_t size, const Shader& shader);
```

Creates the requested child/resource object and returns it as an owning C++ wrapper.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.
shader	const Shader&	Reference or value identifying the associated HRL resource or wrapper object.

CreateMaterialFromFBXIndexed

```
HRL_CPP_API Material CreateMaterialFromFBXIndexed( const void* data, std::size_t size, std::size_t materialIndex);
```

Creates the requested child/resource object and returns it as an owning C++ wrapper.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.
materialIndex	std::size_t	Index/count selector for an element or quality/iteration level in the associated collection.

CreateMaterialFromFBXIndexedWithShader

```
HRL_CPP_API Material CreateMaterialFromFBXIndexedWithShader( const void* data, std::size_t size, std::size_t materialIndex, const Shader& shader);
```

Creates the requested child/resource object and returns it as an owning C++ wrapper.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.
materialIndex	std::size_t	Index/count selector for an element or quality/iteration level in the associated collection.
shader	const Shader&	Reference or value identifying the associated HRL resource or wrapper object.

GetVertex3DFromFBX

```
HRL_CPP_API Vertex3D* GetVertex3DFromFBX( const void* data, std::size_t size, std::size_t& vertexCount);
```

Retrieves vertex3 d from f b x from the current HRL object.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.
vertexCount	std::size_t&	Value supplied to configure the vertexCount property or operation.

FreeVertex3DFromFBX

```
HRL_CPP_API void FreeVertex3DFromFBX(Vertex3D* vertices);
```

Performs the operation represented by this API entry on the associated HRL resource.

Parameter	Type	Meaning
vertices	Vertex3D*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.

GetSkeletalMeshFromFBX

```
HRL_CPP_API SkeletalMeshData* GetSkeletalMeshFromFBX( const void* data, std::size_t size);
```

Retrieves skeletal mesh from f b x from the current HRL object.

Parameter	Type	Meaning
data	const void*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.
size	std::size_t	Value supplied to configure the size property or operation.

FreeSkeletalMeshData

```
HRL_CPP_API void FreeSkeletalMeshData(SkeletalMeshData* data);
```

Performs the operation represented by this API entry on the associated HRL resource.

Parameter	Type	Meaning
data	SkeletalMeshData*	Pointer/reference to caller-provided data or an output destination, as indicated by the function signature.

UpdateSkeletalAnimations

```
HRL_CPP_API void UpdateSkeletalAnimations(float deltaSeconds);
```

Advances the corresponding time-dependent subsystem by the supplied elapsed time.

Parameter	Type	Meaning
deltaSeconds	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

UpdateVFX

```
HRL_CPP_API void UpdateVFX(float deltaSeconds);
```

Advances the corresponding time-dependent subsystem by the supplied elapsed time.

Parameter	Type	Meaning
deltaSeconds	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

WaitForAllAsyncResources

```
HRL_CPP_API void WaitForAllAsyncResources();
```

Performs the operation represented by this API entry on the associated HRL resource.

MouseMoved

```
HRL_CPP_API void MouseMoved(float x, float y);
```

Updates the renderer with the current mouse position in window-relative pixel coordinates.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

MouseButtonCallback

```
HRL_CPP_API void MouseButtonCallback(int button, int pressed);
```

Updates the renderer with a mouse button transition so interactive UI, gizmos, and picking can react to input.

Parameter	Type	Meaning
button	int	Value supplied to configure the button property or operation.
pressed	int	Boolean state controlling whether the associated feature is enabled or active.

IsGlobalIlluminationMethodSupported

```
HRL_CPP_API bool IsGlobalIlluminationMethodSupported( EGlobalIlluminationMethod method);
```

Queries whether the requested global-illumination method is supported by the active rendering backend.

Parameter	Type	Meaning
method	EGlobalIlluminationMethod	Value supplied to configure the method property or operation.

GetGlobalIlluminationSupportedMethods

```
HRL_CPP_API uint GetGlobalIlluminationSupportedMethods();
```

Retrieves global illumination supported methods from the current HRL object.

ClearScreen

```
HRL_CPP_API void ClearScreen();
```

Clears the current framebuffer using the renderer clear state.

SetAntialiasingMode

```
HRL_CPP_API void SetAntialiasingMode(EAntialiasingMode mode);
```

Configures antialiasing mode for this resource. Calling the function updates the corresponding runtime state used by HRL rendering, simulation, UI, or resource processing.

Parameter	Type	Meaning
mode	EAntialiasingMode	Value supplied to configure the mode property or operation.

SetAntialiasingMode

```
HRL_CPP_API void SetAntialiasingMode(uint mode);
```

Configures antialiasing mode for this resource. Calling the function updates the corresponding runtime state used by HRL rendering, simulation, UI, or resource processing.

Parameter	Type	Meaning
mode	uint	Value supplied to configure the mode property or operation.

GetProjectionMatrix

```
HRL_CPP_API void GetProjectionMatrix(float (&matrix)[16]);
```

Retrieves projection matrix from the current HRL object.

GetViewMatrix

```
HRL_CPP_API void GetViewMatrix(float (&matrix)[16]);
```

Retrieves view matrix from the current HRL object.

GetModelMatrix

```
HRL_CPP_API void GetModelMatrix(const Mesh& mesh, float (&matrix)[16]);
```

Retrieves model matrix from the current HRL object.

Parameter	Type	Meaning
mesh	const Mesh&	Reference or value identifying the associated HRL resource or wrapper object.

SetDebugLineThickness

```
HRL_CPP_API void SetDebugLineThickness(float thickness);
```

Configures debug line thickness for this resource. Calling the function updates the corresponding runtime state used by HRL rendering, simulation, UI, or resource processing.

Parameter	Type	Meaning
thickness	float	Value supplied to configure the thickness property or operation.

DrawDebugSegment

```
HRL_CPP_API void DrawDebugSegment( const Scene& scene, float ax, float ay, float az, float bx, float by, float bz, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
ax	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
ay	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
az	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bx	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
by	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bz	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugPolygon

```
HRL_CPP_API void DrawDebugPolygon( const Scene& scene, EDebugRenderingType mode, const float* xs, const float* ys, const float* zs, int count, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
mode	EDebugRenderingType	Value supplied to configure the mode property or operation.
xs	const float*	Value supplied to configure the xs property or operation.
ys	const float*	Value supplied to configure the ys property or operation.
zs	const float*	Value supplied to configure the zs property or operation.
count	int	Index/count selector for an element or quality/iteration level in the associated collection.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugCircle

```
HRL_CPP_API void DrawDebugCircle( const Scene& scene, EDebugRenderingType mode, float cx, float cy, float cz, float radius, int segments, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
mode	EDebugRenderingType	Value supplied to configure the mode property or operation.
cx	float	Value supplied to configure the cx property or operation.
cy	float	Value supplied to configure the cy property or operation.
cz	float	Value supplied to configure the cz property or operation.
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
segments	int	Index/count selector for an element or quality/iteration level in the associated collection.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugCapsule

```
HRL_CPP_API void DrawDebugCapsule( const Scene& scene, EDebugRenderingType mode, float ax, float ay, float az, float bx, float by, float bz, float radius, int segments, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
-----------	------	---------

scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
mode	EDebugRenderingType	Value supplied to configure the mode property or operation.
ax	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
ay	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
az	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bx	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
by	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bz	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
segments	int	Index/count selector for an element or quality/iteration level in the associated collection.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugPoint

```
HRL_CPP_API void DrawDebugPoint( const Scene& scene, float x, float y, float z, float size, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
size	float	Value supplied to configure the size property or operation.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
---	-------	--

6. Frame, Input, Debug and Capability Functions

Frame and input

UpdateSkeletalAnimations

```
/** * @brief Advances all playing skeletal mesh animations by deltaSeconds. */ /*
===== * UPDATE & INPUT *
===== */ HRL_CPP_API void
UpdateSkeletalAnimations(float deltaSeconds);
```

Advances the corresponding time-dependent subsystem by the supplied elapsed time.

Parameter	Type	Meaning
deltaSeconds	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

UpdateVFX

```
/** * @brief Advances VFX systems that have AutoUpdate disabled. */ HRL_CPP_API void UpdateVFX(float deltaSeconds);
```

Advances the corresponding time-dependent subsystem by the supplied elapsed time.

Parameter	Type	Meaning
deltaSeconds	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).

WaitForAllAsyncResources

```
/** * @brief Waits for all queued asynchronous resources and uploads their completed GPU objects. */ HRL_CPP_API void
WaitForAllAsyncResources();
```

Performs the operation represented by this API entry on the associated HRL resource.

MouseMoved

```
/** * @brief Updates the renderer's mouse position. * * Coordinates are window-relative pixels. */ HRL_CPP_API void
MouseMoved(float x, float y);
```

Updates the renderer with the current mouse position in window-relative pixel coordinates.

Parameter	Type	Meaning
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.

MouseButtonCallback

```
/** * @brief Notifies HRL of a mouse button transition. * * @param button One of MOUSE_BUTTON_LEFT,
MOUSE_BUTTON_RIGHT or MOUSE_BUTTON_MIDDLE. * @param pressed One of MOUSE_RELEASE or MOUSE_PRESS. */ HRL_CPP_API void
MouseButtonCallback(int button, int pressed);
```

Updates the renderer with a mouse button transition so interactive UI, gizmos, and picking can react to input.

Parameter	Type	Meaning
button	int	Value supplied to configure the button property or operation.

pressed	int	Boolean state controlling whether the associated feature is enabled or active.
---------	-----	--

Capability queries

IsGlobalIlluminationMethodSupported

```
/* ===== * GLOBAL ILLUMINATION QUERIES *
===== */ /** * @brief Returns whether the
active backend implements the requested GI method. */ HRL_CPP_API bool IsGlobalIlluminationMethodSupported(
EGlobalIlluminationMethod method);
```

Queries whether the requested global-illumination method is supported by the active rendering backend.

Parameter	Type	Meaning
method	EGlobalIlluminationMethod	Value supplied to configure the method property or operation.

GetGlobalIlluminationSupportedMethods

```
/** * @brief Returns the bitmask of global illumination methods supported by the active backend. * * The bit
corresponding to a method is represented by (1u << method). */ HRL_CPP_API uint
GetGlobalIlluminationSupportedMethods();
```

Performs the operation represented by this API entry on the associated HRL resource.

Global rendering

ClearScreen

```
/** * @brief Clears the entire screen to its default clear color. * * Useful when no scene covers the full
framebuffer. */ HRL_CPP_API void ClearScreen();
```

Clears the current framebuffer using the renderer clear state.

SetAntialiasingMode

```
/** * @brief Enables or changes the global multisample anti-aliasing mode. */ HRL_CPP_API void
SetAntialiasingMode(EAntialiasingMode mode);
```

Configures antialiasing mode for this resource. Calling the function updates the corresponding runtime state used by HRL rendering, simulation, UI, or resource processing.

Parameter	Type	Meaning
mode	EAntialiasingMode	Value supplied to configure the mode property or operation.

Matrices

GetProjectionMatrix

```
/* ===== * MATRICES *
===== */ /** * @brief Writes the current
projection matrix into a caller-provided array. * * The matrix is column-major and contiguous. */ HRL_CPP_API void
GetProjectionMatrix(float (&matrix)[16]);
```

Retrieves projection matrix from the current HRL object.

GetViewMatrix

```
/** * @brief Writes the current view matrix into a caller-provided array. * * The matrix is column-major and
contiguous. */ HRL_CPP_API void GetViewMatrix(float (&matrix)[16]);
```

Retrieves view matrix from the current HRL object.

GetModelMatrix

```
/** * @brief Writes the model matrix of a specific mesh into a caller-provided array. * * The matrix is column-major and contiguous. */ HRL_CPP_API void GetModelMatrix(const Mesh& mesh, float (&matrix)[16]);
```

Retrieves model matrix from the current HRL object.

Parameter	Type	Meaning
mesh	const Mesh&	Reference or value identifying the associated HRL resource or wrapper object.

Debug geometry

SetDebugLineThickness

```
/* ===== * DEBUG GEOMETRY * ===== */
/** * @brief Sets the line thickness used by all debug draw calls. * * The exact visual result depends on backend line rendering support. */ HRL_CPP_API void SetDebugLineThickness(float thickness);
```

Configures debug line thickness for this resource. Calling the function updates the corresponding runtime state used by HRL rendering, simulation, UI, or resource processing.

Parameter	Type	Meaning
thickness	float	Value supplied to configure the thickness property or operation.

DrawDebugSegment

```
/** * @brief Draws a debug line segment for the current frame. * * Must be called every frame to persist the rendering. * The segment is expressed in world space and the color components are in [0,1]. */ HRL_CPP_API void DrawDebugSegment(const Scene& scene, float ax, float ay, float az, float bx, float by, float bz, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
ax	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
ay	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
az	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bx	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
by	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bz	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugPolygon

```
/** * @brief Draws a debug polygon for the current frame. * * Vertices are specified as separate X, Y and Z arrays of
the same length. */ HRL_CPP_API void DrawDebugPolygon( const Scene& scene, EDebugRenderingType mode, const float* xs,
const float* ys, const float* zs, int count, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
mode	EDebugRenderingType	Value supplied to configure the mode property or operation.
xs	const float*	Value supplied to configure the xs property or operation.
ys	const float*	Value supplied to configure the ys property or operation.
zs	const float*	Value supplied to configure the zs property or operation.
count	int	Index/count selector for an element or quality/iteration level in the associated collection.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugCircle

```
/** * @brief Draws a debug circle for the current frame. */ HRL_CPP_API void DrawDebugCircle( const Scene& scene,
EDebugRenderingType mode, float cx, float cy, float cz, float radius, int segments, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
mode	EDebugRenderingType	Value supplied to configure the mode property or operation.
cx	float	Value supplied to configure the cx property or operation.
cy	float	Value supplied to configure the cy property or operation.
cz	float	Value supplied to configure the cz property or operation.
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
segments	int	Index/count selector for an element or quality/iteration level in the associated collection.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugCapsule

```
/** * @brief Draws a debug capsule for the current frame. * * The capsule is a cylinder with hemispherical caps. */
HRL_CPP_API void DrawDebugCapsule( const Scene& scene, EDebugRenderingType mode, float ax, float ay, float az, float
bx, float by, float bz, float radius, int segments, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
mode	EDebugRenderingType	Value supplied to configure the mode property or operation.
ax	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
ay	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
az	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bx	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
by	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
bz	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
radius	float	Scalar control value; units depend on the method and are described by the method context (for example seconds, world units, pixels, degrees, or a normalized factor).
segments	int	Index/count selector for an element or quality/iteration level in the associated collection.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

DrawDebugPoint

```
/** * @brief Draws a debug point for the current frame. * * The point size is expressed in pixels. */ HRL_CPP_API
void DrawDebugPoint( const Scene& scene, float x, float y, float z, float size, float r, float g, float b);
```

Queues a diagnostic primitive for the current frame. Debug geometry is transient and should be submitted again on subsequent frames when persistent visualization is required.

Parameter	Type	Meaning
scene	const Scene&	Reference or value identifying the associated HRL resource or wrapper object.
x	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
y	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
z	float	Dimension or coordinate, expressed in pixels for framebuffer/UI geometry or in world/object space where the surrounding API operates in 3D space.
size	float	Value supplied to configure the size property or operation.
r	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
g	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.

b	float	Color component. Unless otherwise stated by the surrounding API, color values are interpreted as normalized floating-point components.
----------	--------------	--

7. Recommended Usage Patterns

The following examples are documentation patterns derived from the resource relationships in the C++ interface. They illustrate how the wrappers are intended to fit together without introducing additional APIs.

Minimal frame loop

```
#include "hrl.hpp"

hrl::Instance renderer(hrl::HRL_OPENGL_33, width, height, loader);
hrl::Scene scene(true);

while (!shouldClose) {
    renderer.BeginFrame();
    // Update scene state here.
    renderer.EndFrame();
}
```

Static mesh + material

```
hrl::Shader shader(vertexBytes, vertexSize, fragmentBytes, fragmentSize);
hrl::Material material(shader);
hrl::StaticMesh mesh(scene, vertices, vertexCount, indices, indexCount);

mesh.SetMaterial(material);
mesh.SetLocation(0.0f, 0.0f, -3.0f);
mesh.SetRotation(0.0f, 45.0f, 0.0f);
```

Asynchronous shader

```
hrl::Shader shader = hrl::Shader::CreateAsync(
    vertexBytes, vertexSize, fragmentBytes, fragmentSize);

if (!shader.IsReady()) {
    shader.Wait();
}

hrl::Material material(shader);
```

FBX resource inspection

```
hrl::FBXResources resources(fbxBytes, fbxSize);

std::size_t materials = resources.GetMaterialCount();
for (std::size_t i = 0; i < materials; ++i) {
    const hrl::FBXMaterialInfo* material = resources.GetMaterial(i);
    if (material) {
        // Inspect material->baseColor, roughness, metallic, etc.
    }
}
```

Skeletal animation

```

hrl::SkeletalMeshData* data = hrl::GetSkeletalMeshFromFBX(fbxBytes, fbxSize);
if (data) {
    hrl::SkeletalMesh character(scene, *data);
    character.PlaySkeletalAnimation(0);
    hrl::UpdateSkeletalAnimations(deltaSeconds);
    hrl::FreeSkeletalMeshData(data);
}

```

VFX emitter

```

hrl::VFXSystem system(scene);
hrl::VFXEmitter emitter = system.CreateEmitter();

emitter.SetSpawnRate(40.0f);
emitter.SetLifetime(0.4f, 1.0f);
emitter.SetInitialSpeed(1.0f, 3.0f);
emitter.SetGravity(0.0f, -9.81f, 0.0f);
emitter.SetRenderMode(hrl::HRL_VFX_RENDER_BILLBOARD);
system.Play();

```

UI + callbacks

```

hrl::Widget button(viewport, hrl::HRL_WIDGET_BUTTON);
button.SetPosition(30.0f, 30.0f);
button.SetSize(180.0f, 48.0f);
button.SetButtonText("Click me");
button.SetButtonClickable(true);
button.SetButtonPressedCallback(onButton, userData);

```

Version note

This combined reference is labeled API version 26.6 as requested. The C API reference precedes this section; the C++ reference begins here and follows the declarations present in `hrl.hpp`.